

Pairing-Based SNARGs with Two Group Elements

Gal Arnon
galarnon42@gmail.com
Bocconi University

Jesko Dujmovic
mail@ind-jesko.net
Northeastern University

Eylon Yogev
eylon.yogev@biu.ac.il
Bar-Ilan University

February 12, 2026

Abstract

SNARGs are cryptographic primitives that allow a prover to demonstrate membership in an NP language while sending a proof that is much smaller than the witness. In this work, we focus on the succinctness of publicly-verifiable group-based SNARGs, analyzed in a model that combines both a generic (asymmetric) bilinear group ($\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$) and a random oracle (the GGM + ROM).

We construct the first publicly-verifiable SNARG in the GGM + ROM where the proof consists of exactly 2 elements of $\mathbb{G}_1$ and no additional bits, achieving the smallest proof size among all known publicly verifiable group-based SNARGs. Our security analysis is tight, ensuring that the construction incurs no hidden security losses. Concretely, when instantiated with the BLS12-381 curve for 128-bit security, our scheme yields a proof size of 768 bits, nearly a $2\times$ improvement over the best known pairing-based SNARG. While our scheme is not yet concretely efficient, it demonstrates the feasibility of ultra-short proofs and opens the door to future practical instantiations.

Complementing this construction, we establish a new lower bound for group-based SNARGs. We prove that under mild and natural restrictions on the verifier (which are satisfied by all known schemes) no SNARG exists in the Maurer GGM + ROM with a proof that consists of a single group element (assuming one-way functions). This substantially strengthens the lower bound of Groth, which was more restrictive and did not extend to settings with a random oracle.

Keywords: succinct arguments; interactive proofs; generic group model; pairing-based cryptography

Contents

1	Introduction	1
1.1	Our results	2
1.2	Related work	4
1.3	Open problems	5
2	Techniques	6
2.1	SNARGs with two group elements	6
2.2	Lower bound for SNARGs with one group element	11
3	Preliminaries	19
3.1	Idealized models	19
3.2	Succinct non-interactive arguments	20
4	Linear map commitments	21
5	SNARGs with two group elements	26
5.1	Linear PCPs	26
5.2	SNARGs from PCPs with small accepting set	29
5.3	Proof of Theorems 5.1 and 5.2	35
6	Lower bound for SNARGs with one group element	38
6.1	Removing the ROM from one group element SNARGs	39
6.2	Ruling out one group element SNARGs in the pure GGM	48
	Acknowledgments	54
	References	54

1 Introduction

Succinct non-interactive arguments (of knowledge), known as SNARGs (or SNARKs) are cryptographic primitives that enable a prover to demonstrate membership in an NP language while sending only a small proof, much smaller than the witness itself. Their succinctness and efficiency have made SNARGs a central tool in applications such as verifiable delegation of computation, zero-knowledge proofs for privacy-preserving protocols, and scalability solutions in blockchain systems. Recent years have witnessed substantial progress in the construction of SNARGs, leading to proof systems with concretely short arguments and improved efficiency of both prover and verifier, which are widely used and deployed in practice.

The focus of this work is the succinctness of publicly verifiable group-based SNARGs. This class also captures pairing-based constructions and schemes that additionally rely on the random oracle model. Group-based SNARGs are among the most succinct and practically efficient proof systems known, and they are the ones most widely deployed in real-world applications. Our work is motivated by the following fundamental question:

What is the smallest possible (publicly verifiable) group-based SNARG proof size?

The quest for minimal proofs. A long sequence of works, beginning with Groth’s construction [Gro10], has aimed to minimize the concrete proof size of SNARGs for NP. Most of these works are built over (asymmetric) pairing-friendly bilinear groups $(\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T)$, with soundness typically analyzed in the generic group model [Sho97; Mau05; Nec94]. Groth’s initial construction achieved proofs consisting of 42 bilinear group elements; this was later reduced to 39 elements by Lipmaa [Lip12]. Subsequently, Gennaro et al. [GGPR13] constructed SNARGs with proofs of just 7 group elements.

Bitansky et al. [BCIOP13] introduced the notion of a *linear* PCP, where a verifier issues a small number of inner-product queries to a proof vector, and described a general compiler from linear PCPs to SNARGs using the notion of linear-only encryption. A similar compiler was implicit in [GGPR13], and both works followed the high-level blueprint introduced in [IKO05; Gro10]. Later, Danezis et al. [DFGK14] constructed succinct arguments with proofs consisting of just 4 bilinear group elements.

Finally, Groth [Gro16] presented a construction with only 3 bilinear group elements (amounting to just over 1500 bits in practice) and with highly efficient verification. The succinctness and efficiency of this last scheme made it particularly appealing in practice, and it has since been widely implemented and deployed. For a long time, it remained an open question whether it is possible to construct publicly verifiable SNARGs over asymmetric bilinear groups with fewer than three group elements.

We note that while such two group element proofs are achievable in the symmetric pairing setting [Gro16], our work explicitly targets *asymmetric* bilinear groups. This distinction is critical since symmetric groups are much larger than their asymmetric counterparts [BGJT13; Gal14]. Consequently, real-world SNARK deployments rely almost exclusively on asymmetric pairings (e.g., BLS12-381), making this the primary setting of interest.

Smaller proofs in the GGM + ROM. Recently, two improvements have been made over the construction of Groth [Gro16] by additionally leveraging a random oracle beyond the generic group (which we will refer to as the GGM + ROM). Lipmaa leveraged this model [Lip24] to reduce the proof size by designing a SNARG consisting of only three $\mathbb{G}_1$ group elements (the typically smaller

group in a bilinear map) and one field element. This optimization roughly halves the proof length compared to [Gro16] (for some parameter settings).

Subsequently, Dellepere et al. [DMS24] introduced the “PARI” scheme, a SNARG that breaks the three-group-element barrier (also in the GGM+ROM). PARI has a proof size of just two $\mathbb{G}_1$ elements and two field elements, representing the smallest proof size achieved by any known publicly verifiable SNARG.

Groth [Gro16] showed that there are no group-based SNARGs for NP with proofs consisting of a single group element, as long as the verifier admits a certain structure. However, the lower bound does not extend to the ROM model, a gap explicitly highlighted by Lipmaa [Lip24]. At present, no lower bounds are known in this combined model for group-based SNARGs. It is also worth noting that allowing a single round of interaction enables proofs of just one group element [BIOW20].

Designated verifiers. One approach to reducing proof size is to consider the relaxed *designated-verifier* SNARG (dv-SNARG) model. In this setting, the verifier privately holds a verification key generated during setup, and soundness is lost if this key is ever leaked. Such schemes are considerably less applicable in practice compared to their publicly verifiable counterparts.

Barta et al. [BIOW20] constructed a dv-SNARG with proofs of just two group elements in a pairing-free setting. However, their construction achieves only inverse-polynomial (rather than negligible) soundness error. More recently, Arnon et al. [ADI25] presented a dv-SNARG consisting of a single group element plus $O(\lambda)$ bits for λ -bit security, achieving negligible soundness error. However, their scheme does not satisfy the standard notion of reusable soundness, restricting its applicability.¹

Non-group-based SNARGs. There are also several constructions of SNARGs that do not rely on bilinear groups (or on groups at all). Perhaps the most prominent examples are hash-based SNARGs, whose security is proven in the random oracle model without additional assumptions. These schemes enjoy many desirable practical properties (and are post-quantum secure), but their proofs are comparatively large. Moreover, there is strong evidence that such schemes cannot achieve proofs significantly smaller than $\Theta(\lambda^2)$ for λ -bit security [HNY22].

If one is willing to rely on stronger cryptographic assumptions, such as general-purpose obfuscation, then it is possible to construct SNARGs with optimal proof length [SW14; WW24; WZ24; WW25]. However, today these schemes remain far from practical and, moreover, they satisfy the stronger notion of knowledge soundness only in elaborate ideal models (e.g., [JLLW23]). While obfuscation-based SNARGs achieve even smaller proofs than ours, they do not offer a clear path toward practical implementations. Finally, lattice-based proof systems, e.g., [BISW17; BS23; SSEKYZ24; AFLN24], are not currently competitive with group-based systems in terms of concrete succinctness.

1.1 Our results

We make significant progress towards pinpointing the minimal proof size in group-based SNARGs in terms of both new constructions and improved lower bounds.

The 2-element upper bound. We construct a pairing-based, publicly verifiable SNARG (with

¹Reusable soundness allows the same setup to be used to verify multiple proofs, where the prover learns whether each proof was accepted or rejected by the verifier and chooses the next instance accordingly. A scheme that lacks this property suffers a degradation of security (e.g., a reduced security parameter) with each reuse, thus limiting it to a small number of uses.

imperfect completeness) with proofs consisting of exactly *two* $\mathbb{G}_1$ elements, achieving the smallest proof size among all known publicly verifiable SNARGs and answering an open question raised in [DMS24]. Our analysis provides tight security guarantees in the combined generic group and random oracle models, ensuring that the construction incurs no hidden security losses.

We emphasize that our focus is not on optimizing prover efficiency, and the scheme is therefore not intended to compete with concretely efficient constructions. Nevertheless, our results demonstrate the feasibility of achieving publicly verifiable SNARGs with proofs of just two group elements, opening the door to future practical instantiations.

Theorem 1.1 ($2\mathbb{G}_1$ SNARG; informal). *Let λ be a security parameter, and assume that $\lambda \leq \log^c n$ for some constant c , where n is the instance size. Let $\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a bilinear group (of prime order $p \geq 2^\lambda$). Every language in NP has a (publicly verifiable) SNARG in the GGM+ROM where the proof consists of $2 \mathbb{G}_1$ elements.*

See Theorem 5.1 for a formal statement.

Instantiating the group with the widely used BLS12-381 curve (for 128-bit security) yields a proof size of $2 \cdot 384 = 768$ bits—nearly a $2\times$ improvement over the best known pairing-based SNARGs. This proof size also closely matches that of the best designated-verifier SNARG [ADI25]. We provide a detailed comparison of our proof size against existing publicly verifiable group-based SNARGs in Section 1.1.

Scheme	Model	Proof Structure	Size (Bits)
Groth16 [Gro16]	GGM	$2 \mathbb{G}_1 + 1 \mathbb{G}_2$	1536
Lipmaa [Lip24]	GGM + ROM	$3 \mathbb{G}_1 + 1 \mathbb{F}$	≈ 1408
PARI [DMS24]	GGM + ROM	$2 \mathbb{G}_1 + 2 \mathbb{F}$	≈ 1280
This Work	GGM + ROM	$2 \mathbb{G}_1$	768

Table 1: Comparison of proof sizes for publicly verifiable group-based SNARGs over asymmetric pairing groups. Bit sizes are estimated for the BLS12-381 curve (≈ 381 -bit field), assuming point compression ($|\mathbb{G}_1| \approx 384$ bits, $|\mathbb{G}_2| \approx 768$ bits).

The theorem above (Theorem 1.1) guarantees λ bits of security for the reasonable parameter regime where $\lambda \leq \log^c n$. For instance, in a typical setting with $\lambda = 100$ and $n = 2^{20}$, the assumption already holds with $c = 2$. As a more fundamental question, we also examine the general parameter regime.

We obtain a two-group-element SNARG for NP (similarly to Theorem 1.1) that has negligible soundness and completeness error also for the case of $\lambda = \omega(\text{polylog } n)$, but the construction additionally relies on the *unique games conjecture* (UGC). The UGC, introduced by Khot [Kho02], posits that it is computationally hard to approximate certain constraint satisfaction problems known as *unique games*. This conjecture has become a central assumption in computational complexity and hardness-of-approximation results.

Theorem 1.2 ($2\mathbb{G}_1$ UGC based SNARG; informal). *Let λ be a security parameter, and assume the unique games conjecture holds. Let $\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a bilinear group (of prime order $p \geq 2^\lambda$). Every language in NP has a (publicly verifiable) SNARG in the GGM+ROM where proof consists of $2 \mathbb{G}_1$ elements.*

See Theorem 5.2 for a formal statement.

The 1-element lower bound. As we have mentioned, there are no known group-based SNARGs with a single group element. In [Gro16], Groth gave strong limitations on the feasibility of pairing-based SNARGs that consist of only a single group element. He ruled out a class of SNARGs, where the verifier is a conjunction of pairing equation checks.

Complementing our construction, we give strong lower bounds for group-based SNARGs with single-element proofs. First, we strengthen Groth’s lower bound by extending it to all SNARGs in the Maurer [Mau05] generic group model (GGM). Next, we extend this bound to the setting where the prover and verifier have access to both the generic group and a random oracle (GGM+ROM). The domain and range of the oracle are flexible: they may consist of group elements, pure strings, or a mixture of both. Our lower bound applies under the restriction that the verifier’s random-oracle queries are the output of predefined pairing functions of the CRS and the proof (while the verifier may still issue arbitrary GGM queries and is otherwise unrestricted). Notably, all known SNARG constructions satisfy this restriction.

Theorem 1.3 (No 1G SNARG in the GGM+ROM; informal). *If one-way functions exist (in the plain model), there are no SNARGs for NP in the (Maurer) Bilinear GGM with a random oracle, with the following properties:*

- *The proof consists of a single group element;*
- *The completeness and soundness errors are negligible;*
- *The verifier performs queries to the ROM that are non-adaptive²;*
- *The setup algorithm does not perform random oracle queries.*

See Theorem 6.1 for a formal statement.

1.2 Related work

Pairing-free. There has been a line of work [BCIOP13; BCCGP16; BBBPWM18; BLOW20; BHIRW24] which constructs SNARGs using generic pairing-free groups, namely in the standard GGM. The simpler structure gives hope for more conservative group instantiations with better concrete parameters. Unlike pairing-based SNARGs, the most succinct GGM-based SNARGs apply only in the designated-verifier setting.

Pure random oracle model. There is a long line of work constructing SNARGs that rely solely on a random oracle (a.k.a., hash-based SNARGs). The best trait of these constructions is a fast prover, since the prover performs only “symmetric key cryptography” operations (oracle calls, or field operations), whereas group-based operations are typically heavier. At a technical level, these SNARGs combine the blueprint of Kilian [Kil92] and Micali [Mic00] with an interactive variant of classical PCPs known as an IOP [BCS16]. See [CY24] for further details. However, even the most succinct hash-based SNARGs [AHIV17; BBHR19; STW23; GLSTW23; ZCF24; BBHR18; ACY23; ACFY24; ACFY25] have proofs with tens to hundreds of kilobytes in size. Moreover, there is strong evidence that such schemes cannot achieve proofs significantly smaller than $\Theta(\lambda^2)$ for λ -bit security [HNY22].

²More precisely, the verifier can be split into two parts; The first part does not do any pairing equation checks or ROM calls but it produces the call to the ROM, the second part receives the responses to the ROM call but can not do any further ROM calls.

1.3 Open problems

While our work establishes tight bounds for publicly verifiable SNARGs in the combined GGM+ROM, several intriguing problems remain open.

- **Removing the Random Oracle.** Our construction relies on the random oracle to compress the proof down to two elements. In the standard Generic Group Model (without a random oracle), the state-of-the-art remains the 3-element construction of Groth [Gro16]. *Do publicly verifiable SNARGs with proofs of exactly two group elements exist in the standard GGM?*
- **Perfect Completeness.** Our scheme achieves negligible completeness error rather than perfect completeness. This limitation stems from our use of single-query linear PCPs, which inherently has imperfect completeness (e.g., [BHIRW24, Appendix A] and [BIOW20, Remark 4.9]). While acceptable in theory, perfect completeness is a desirable property in practice. *Can one construct a group-based SNARG in the GGM+ROM with two group elements and perfect completeness?*
- **Single-Element Designated Verifiers.** Our lower bound rules out single-element proofs for *publicly verifiable* schemes. However, in the designated-verifier setting, Arnon et al. [ADI25] recently constructed a SNARG consisting of a single group element plus a few auxiliary bits. *Do designated-verifier SNARGs with a proof consisting of exactly one group element exist in the GGM+ROM?*
- **Unrestricted Lower Bounds.** Our impossibility result assumes a natural structural restriction on the verifier (specifically, that ROM queries are non-adaptive). While this captures all known group-based SNARGs, a fully general lower bound would be theoretically satisfying. *Can we prove a lower bound for single-element SNARGs in the GGM+ROM without any structural assumptions on the verifier?*

2 Techniques

We give a high-level overview of our techniques:

- In Section 2.1, we give an overview of our constructions (namely, Theorem 1.1 and Theorem 1.2).
- In Section 2.2, we give an overview of our lower bound (namely, Theorem 1.3).

2.1 SNARGs with two group elements

In this section we sketch the proofs of Theorems 1.1 and 1.2, describing the construction of a SNARG in the bilinear generic group plus random oracle model with argument size of two group elements.

2.1.1 SNARGs from linear PCPs

The starting point of our construction is the work of Lai and Malavolta [LM19] who built a SNARG by combining a linear PCP with a linear map commitment. Before describing the construction, we first give informal definitions of these two building blocks:

- A (non-adaptive) linear PCP $(\mathbf{P}, \mathbf{V})$ over a finite field $\mathbb{F}$ generalizes the standard notion of a PCP by allowing the verifier to query linear functions of the proof, rather than individual coordinates as in a standard PCP. In more detail, the prover generates a proof $\pi \in \mathbb{F}^\ell$, and the verifier, given an instance $\mathbf{x}$ and randomness ρ , produces a matrix $M \in \mathbb{F}^{\ell \times q}$ (which we interpret as q linear queries). The verifier then receives as an answer the vector $\alpha = M\pi \in \mathbb{F}^q$, and makes its decision. Completeness and soundness are defined analogously to those in a standard PCP.
- A linear map commitment $(\text{Setup}, \text{Com}, \text{Open}, \text{Verify})$ is a commitment scheme with the following properties. After a setup phase **Setup** that generates a common reference string **crs**, the sender generates a commitment $\mathbf{c} = \text{Com}(\text{crs}, \pi)$ to a string $\pi \in \mathbb{F}^\ell$. Given a matrix $M \in \mathbb{F}^{q \times \ell}$, the sender can produce an answer and a decommitment proof $(\alpha, \text{pf}) = \text{Open}(\text{crs}, \pi, M)$ attesting to the fact that the vector π committed to in $\mathbf{c}$ satisfies $\alpha = M\pi$, which is verified using $\text{Verify}(\text{crs}, \mathbf{c}, M, \alpha, \text{pf})$.

Lai and Malavolta [LM19] combine linear PCPs with linear map commitments and the Fiat-Shamir transformation [FS86] to construct a SNARG $(\mathcal{S}, \mathcal{P}, \mathcal{V})$ in the random oracle model, where all parties are given access to a random function $\mathcal{O}$. For a security parameter λ , their scheme works as follows:

- $\mathcal{S}(1^n, 1^\lambda)$: Generate reference string **crs** using **Setup**.
- $\mathcal{P}(\text{crs}, \mathbf{x}, \mathbf{w})$:
 1. Compute $\pi = \mathbf{P}(\mathbf{x}, \mathbf{w})$ and a commitment $\mathbf{c} = \text{Com}(\text{crs}, \pi)$.
 2. Query $\rho = \mathcal{O}(\mathbf{x}, \mathbf{c})$ and use $\mathbf{x}$ and ρ to compute the PCP verifier query matrix M .
 3. Compute $(\alpha, \text{pf}) = \text{Open}(\text{crs}, \pi, M)$, a decommitment proof to the answer $\alpha = M\pi$.
 4. Output the proof $\Pi = (\mathbf{c}, \alpha, \text{pf})$.
- $\mathcal{V}(\text{crs}, \mathbf{x}, \Pi = (\mathbf{c}, \alpha, \text{pf}))$:
 1. Query $\rho = \mathcal{O}(\mathbf{x}, \mathbf{c})$ and use $\mathbf{x}$ and ρ to compute the PCP verifier query matrix M .
 2. Accept if and only if the PCP verifier accepts given $\mathbf{x}$, randomness ρ , and answer α , and if $\text{Verify}(\text{crs}, \mathbf{c}, M, \alpha, \text{pf}) = 1$.

The above scheme is a SNARG³ with argument size $|c| + |pf| + |\alpha|$, with completeness error σ and soundness error $\beta' = \text{poly}(\lambda) \cdot \beta + \text{negl}(\lambda)$ against $\text{poly}(\lambda)$ -size adversaries, where σ and β are the completeness and soundness errors of the PCP respectively (as we will discuss later on in this section, [LM19] do not do a tight security analysis for their scheme). Thus, in order to construct SNARGs with short proof length using this paradigm, we need a linear map commitment with small commitment and opening sizes, and a PCP with small answer size (which, since $\alpha \in \mathbb{F}^q$, means working over a small field with small query complexity q).

Lai and Malavolta construct a linear map commitment in the bilinear generic group model $(\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T)$, where the groups have prime order $p \geq 2^\lambda$. Their commitment scheme satisfies $|c| = |pf| = 1\mathbb{G}_1$, and the underlying field is $\mathbb{F} = \mathbb{Z}_p$. Consequently, the resulting SNARG has size $2\mathbb{G}_1 + q \cdot \lambda$ bits and achieves at most $\lambda/2$ bits of security, since discrete logarithm attacks run in time proportional to the square root of the group size.

It seems that we have reached an impasse. Despite extensive research on improving the parameters of linear PCPs (e.g., [ALMSS98; IKO07; BCIOP13; BIOW20; BHIRW24]), even under a dream scenario with a 1-query PCP defined over a field of size 2^λ (so as to achieve λ bits of security), the resulting proof would still consist of two group elements (one for the commitment and one for the opening proof) plus λ additional bits (for the answer α). Thus, our central goal of obtaining SNARGs with exactly two group elements remains out of reach.

2.1.2 Reducing to two group elements

We realize the goal of only two group elements as follows. We have the prover *not send the query answer* α ; instead, the verifier computes α locally by itself! At first glance, this seems paradoxical: how can a PCP be verified without receiving the query result? If the verifier can compute α by itself, maybe it does not need the prover at all, rendering the language to be easily decidable. We observe, however, that this is not a contradiction. Even if the verifier can predict α in advance, it still “learns new information” by making the specified query; namely, that the prover has committed to a string π such that $M\pi = \alpha$.

Another way to describe the property we require from the PCP is via its *accepting set*, which we want to be minimal—specifically, of size 1. Formally, for an instance $\mathfrak{x}$ and verifier randomness ρ , the accepting set $S_{\mathfrak{x},\rho}$ consists of all answers that would make the verifier accept. The accepting set size s is defined as the maximum size of $S_{\mathfrak{x},\rho}$ over all instances and randomness.⁴ This parameter provides a way to minimize communication: if the PCP has accepting set size s , only $\log s$ bits are needed to encode the message sent to the verifier.⁵ Pushing this idea to the extreme, if $s = 1$, then *no bits* need to be communicated.

The above discussion motivates optimizing linear PCPs for accepting set size, rather than for query complexity or field size—once the length of α no longer affects communication, these traditionally important parameters become far less critical. This naturally leads to the following question:

Do linear PCPs with accepting set size 1 exist?

Linear PCPs with accepting set size 1. We answer this question in the affirmative. In particular, the linear PCP for the NP-complete language **GapMWSP** described in [BIOW20]—previously

³In fact, the transformation results in a SNARK if the PCP satisfies knowledge soundness.

⁴The value $\log s$ is sometimes called the “free bit complexity” of the PCP.

⁵This assumes an efficient mapping exists between the accepting set and the bit encoding.

used to construct 2-message laconic arguments—satisfies this property. We now sketch their PCP and explain why its accepting set has size 1:

Lemma 1 (informal). *There is a constant $c > 1$ such that there exists a linear PCP for GapMWSP with completeness error $1/\log n$, soundness error $n^{-\log^c n} + 1/|\mathbb{F}|$ and accepting set size $s = 1$.*

Proof sketch. We first (informally) describe the GapMWSP language: given $(A, b, d) \in \mathbb{F}^{\ell \times n} \times \mathbb{F}^\ell \times \mathbb{N}$ decide whether (in the Yes case) there exists x such that $Ax = b$ and has Hamming weight at most d , or (in the No case) every x with $Ax = b$ has Hamming weight at least $d \cdot \text{polylog}(n)$.

We now sketch the linear PCP $(\mathbf{P}, \mathbf{V})$ for this language, due to [BIOW20], and explain why its accepting set size is 1. See [BIOW20] for an analysis of completeness and soundness.

- $\mathbf{P}((A, b, d), x)$: let $\pi = x$ be the proof string.
- $\mathbf{V}(A, b, d)$: Choose a random vector r and a sparse vector e , and make the query $M = (r^T A + e^T)$ (in this case the matrix M is a vector). Given an answer $\alpha = Mx$, accept if and only if $\alpha = r^T b$.

Observe that the verifier accepts if and only if the answer α that it receives is equal to $r^T b$, where this target value $r^T b$ can be computed by the verifier given the input and its randomness. Thus, the PCP has accepting set $\{r^T b\}$ of size 1. $\square$

Our main scheme (Theorem 1.1) is obtained by instantiating the previously described construction with the PCP from Lemma 1. The resulting SNARG has soundness error $\text{poly}(\lambda) \cdot (n^{-\text{polylog } n} + 1/|\mathbb{F}|) + \text{negl}(\lambda)$ for $|\mathbb{F}| \approx 2^\lambda$, and thus it satisfies soundness $\text{negl}(\lambda)$ when $\lambda \leq \text{polylog } n$. This suffices for most realistic settings.

The Samorodnitsky and Trevisan PCP. Outside of this parameter regime, this PCP does not suffice. Note that standard repetition of the PCP does help in this case: since the completeness error of the PCP is nonzero, the amplified verifier’s decision will need to be equal to the majority decision of each execution, which increases the accepting set size.

To fulfill Theorem 1.2, we turn to a second PCP. Samorodnitsky and Trevisan [ST06] describe a PCP that, assuming the *unique games conjecture* (UGC),⁶ for a parameter k satisfies: completeness error 2^{-k} , soundness error $O(k/2^k)$, query complexity $O(k)$, and accepting set size k . To reduce the accepting set size from k to 1, we apply the following transformation: the verifier guesses a random element in the accepting set and accepts only if the answer is equal to its guess. This converts a PCP with accepting set size k and completeness error σ into one with accepting set size 1 and completeness error $\sigma + (1 - 1/k)$, while leaving soundness unchanged.

By instantiating this PCP with $k = \lambda$, we obtain (assuming the UGC) a 2-group-element SNARG with negligible soundness error in any parameter regime, thereby proving Theorem 1.2. An interesting open problem is to construct linear PCPs with inverse exponential soundness error and accepting set size 1, without relying on additional assumptions (we stress that the unique games conjecture is used here for completeness rather than soundness, as it is a conjecture on NP reductions rather than a security assumption).

Amplifying completeness. The SNARG resultant from the construction (roughly) preserves the completeness and soundness errors of the PCP. While we have been careful to ensure that the soundness errors of our PCPs are negligible, the completeness errors are highly inadequate: the

⁶See Section 1.1 for a brief description of the unique games conjecture.

[BIOW20] PCP has error $1/\log n$ and the PCP derived from [ST06] has error $1 - \Theta(1/\lambda)$. As mentioned previously, standard repetition of the PCP cannot be used to decrease the completeness error, as it increases the accepting set size.

We instead get around this issue by altering the SNARG construction rather than the PCP: we have the honest prover “resample” the query matrix M until the verifier accepts. This must be done without affecting the argument size. We describe the new prover for a parameter t , assuming already that the PCP has accepting set size 1, and so the query answer is not sent:

- $\mathcal{P}(\text{crs}, \mathbf{x}, \mathbf{w})$:
 1. Compute $\pi = \mathbf{P}(\mathbf{x}, \mathbf{w})$.
 2. For $i = 1$ to t :
 - (a) Compute $\mathbf{c}_i$ to the string $\pi_i = (\pi||i)$.⁷
 - (b) Query $\rho_i = \mathcal{O}(\mathbf{x}, \mathbf{c}_i)$ and use $\mathbf{x}$ and ρ_i to compute the PCP verifier query matrix M_i .
 - (c) Check whether the PCP verifier accepts given answer $\alpha_i = M_i\pi$.
 - (d) If the above check passes then compute $\mathbf{pf}$, a decommitment proof to the answer $\alpha_i = M_i\pi$ and output SNARG proof $\Pi = (\mathbf{c}_i, \mathbf{pf})$.⁸
 - (e) Otherwise, go to the next iteration.
 3. Output $\perp$.

The verifier is identical to the verifier described in the beginning of this section, except that it pads the matrix M with zeroes so that it has the same number of rows as π_i .

The key idea in the above construction is that, since each $\mathbf{c}_i$ commits to a distinct vector, the security of the commitment scheme guarantees (up to negligible probability) that all $\mathbf{c}_i$ are unique (as otherwise, one could break binding). The Fiat–Shamir hash (the random oracle $\mathcal{O}$) applied to unique commitments, yields a fresh uniformly random PCP randomness. This effectively allows the prover to resample randomness for the PCP verifier, providing multiple independent attempts to make the verifier accept. If the underlying PCP has completeness error σ , then the completeness error of the resulting SNARG is approximately σ^t . As the verifier remains essentially unchanged, this transformation boosts completeness without affecting the proof size or the soundness error relative to the original scheme.

2.1.3 Tight analysis via multi-extraction

In order to give tight security guarantees for our SNARGs, we require a tight analysis that minimizes error losses. We describe how the analysis of [LM19] works, identify the sources of security loss at each step, and explain how we modified the analysis to achieve tightness.

1. *Prove function binding of the linear map commitment.* Roughly, function binding means that for a commitment $\mathbf{c}$ no efficient sender can supply $(\alpha_1, M_1, \mathbf{pf}_1)$ and $(\alpha_2, M_2, \mathbf{pf}_2)$ so that verification passes on both using commitment $\mathbf{c}$, but there exists no vector π such that simultaneously $M_1\pi = \alpha_1$ and $M_2\pi = \alpha_2$. In [LM19], this analysis is done loosely, describing a $\text{negl}(\lambda)$ error for polynomial-time adversaries.

⁷For simplicity of this exposition we treat i an element in the field $\mathbb{F}$ of the linear PCP.

⁸Formally, since the commitment is to π_i , the decommitment attests to $\alpha_i = M'_i\pi_i$ where M'_i is M_i padded with zeroes.

We give a tight analysis in the GGM, showing that the linear map commitment has function binding error (roughly) $\varepsilon = s^2 n/p$ against s -size adversaries where p is the order of the GGM group and n is the committed vector length (actually we show that it satisfies a stronger notion called multi-extraction which we discuss below).

2. *Straightline extraction.* The [LM19] scheme is the application of the Fiat–Shamir [FS86] transformation on the interactive protocol where the prover sends $\mathbf{c}$, the verifier chooses a uniformly random challenge ρ , and the prover replies with $(\alpha, \mathbf{pf})$. By function-binding of the linear map commitment, there must be a single string π that is consistent with all valid answers. Lai and Malavolta show that this interactive protocol is sound by running the prover up until it sends $\mathbf{c}$, and repeatedly generating matrix-answer pairs until an entire PCP proof π can be derived. This reduces soundness to that of the PCP. Here, extracting π by repeatedly running the prover is very costly.

We reduce this error by showing that the linear map commitment is *straightline extractable*. Roughly speaking, the linear-map commitment is straightline extractable if the committed to string can be extracted given only a commitment and the query–answer trace made by the adversary to the GGM oracle. This bypasses the need to rewind the interactive protocol, as π can be extracted given a single commitment. Using straightline extraction, it can be shown that the interactive protocol derived from the scheme prior to Fiat–Shamir has tight soundness error $\beta' = \beta + \varepsilon$ where β is the soundness error of the PCP, and ε is the same binding error as described above.

3. *Fiat–Shamir via multi-extraction.* Lai and Malavolta apply the Fiat–Shamir transformation generically to the interactive protocol above. Against s -size adversaries, incurs error (roughly) $s \cdot \beta'$. Intuitively, this is because the prover can try s different first messages (i.e., $\mathbf{c}$), each of which will generate verifier randomness, thus giving it another attempt to attack the PCP. Thus, even if we use the tight analysis described above with straightline extraction, we would get an error of $s \cdot \beta' \approx s \cdot \beta + s^3 n/p$. While the $s \cdot \beta$ term is to be expected and tight (since the prover can try multiple PCPs), the term $s^3 n/p$ is not. Since the linear map commitment is secure in the GGM, the random oracle calls do not grant the prover any power, and so we would expect to see a factor of $s^2 n/p$ rather than $s^3 n/p$.

To resolve this issue and get a tight bound, we strengthen the security of the linear map commitment, showing that it is *multi-extractable*. At a high-level, multi-extraction considers the prover’s success probability at breaking the commitment in a setting where it has many attempts at different commitments (in our case, due to multiple Fiat–Shamir queries). An analog of this definition for Merkle trees was given in [CY24] in the context of standard PCPs.

We provide an informal yet detailed definition:

Definition 2.1 (Multi-extraction, informal). *A linear map commitment scheme satisfies multi-extraction with error ε if there exists an efficient extractor such that the following holds for any poly-size multi-step adversary $\mathcal{A} = (\mathcal{A}_1, \dots, \mathcal{A}_n, \mathcal{A}')$:*

$$\Pr \left[\begin{array}{l} \left(\begin{array}{l} \text{Com.Verify}(\text{crs}, \mathbf{c}_{i^*}, M, \alpha, \mathbf{pf}) = 1 \\ \wedge Mx_{i^*} \neq \alpha \end{array} \right) \\ \vee \\ \left(\begin{array}{l} \exists i_1, i_2 \in [n] : \\ \mathbf{c}_{i_1} = \mathbf{c}_{i_2} \wedge x_{i_1} \neq x_{i_2} \end{array} \right) \end{array} \right] \left| \begin{array}{l} \text{crs} \leftarrow \text{Com.Setup}(1^\lambda) \\ \text{aux}_0 := \perp \\ \forall i \in [n] : \left(\begin{array}{l} (\mathbf{c}_i, \text{aux}_i) \leftarrow \mathcal{A}_i(\text{crs}, \text{aux}_{i-1}) \\ x_i \leftarrow \text{Ext}(\mathcal{A}_1, \dots, \mathcal{A}_i, \text{crs}) \end{array} \right) \\ (i^*, M, \alpha, \mathbf{pf}) \leftarrow \mathcal{A}'(\text{crs}, \text{aux}_n) \end{array} \right. \right] \leq \varepsilon.$$

We show that the [LM19] linear map commitment has (straightline) multi-extraction error roughly $\varepsilon = s^2 n/p$ against s -size adversaries, where p is the order of the GGM.

We show tight bounds for our SNARG construction assuming a linear map commitment with multiextraction:

Lemma 2 (informal). *Given a linear PCP with completeness error σ and soundness error β , and a linear-map commitment with multiextraction error ε against s -size adversaries, the SNARG described in this section has completeness error $\sigma + \varepsilon$ and soundness error $s \cdot \beta + \varepsilon$ against s -size adversaries.*

See Lemma 5.11 for a formal statement.

Given all of the above, we prove that if the PCP has soundness error β and the GGM has order $p > 2^\lambda$, then for an adversary of size s and instance size n , the SNARG has error (roughly) $O(s \cdot \beta + s^2 n/2^\lambda)$, thus deriving tight error bounds for our SNARGs. For example, for our SNARG for GapMWSP, when plugging in $\beta = n^{-\log^c n} + 1/|\mathbb{F}| = O(2^{-\lambda})$ (in our case we have $\lambda \leq \log^c n$) this becomes $O(s^2 n/2^\lambda)$. For comparison, the best-known soundness bounds for the [Gro16] SNARG is $O(n^3/2^\lambda + s^2 n/2^\lambda)$ [FKL18, Corollary 7.3]. In both cases the $O(s^2 n/2^\lambda)$ term is the dominant factor.

2.2 Lower bound for SNARGs with one group element

In this section, we outline the proof of Theorem 1.3, showing a lower-bound for pairing-based one group element SNARGs in the GGM ($\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$) + ROM (denoted by $\mathcal{O}$) where (1) the setup algorithm does not use the random oracle, and (2) the verifier performs only non-adaptive queries to the random oracle prior to any GGM queries (i.e., no query depends on the answer of a different query or, equivalently, all of the verifier's queries are made at the same time, and this is prior to any GGM equation checks). In this overview, we consider a simplified case compared to Theorem 1.3:

- the SNARG proof is an element in $\mathbb{G}_1$,
- the random oracle receives a group element and outputs a bit (i.e., $\mathcal{O}: \mathbb{G}_1 \cup \mathbb{G}_2 \cup \mathbb{G}_T \rightarrow \{0, 1\}$),
- the common reference string includes the generators of $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ (this is without loss of generality), and
- the verifier is deterministic.⁹

We stress that the full proof supports SNARGs where the (single) group element in the proof is an element of either of the groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ and where the random oracle receives any number of group elements or bits and outputs bits and group elements (e.g., $\mathcal{O}: (\{0, 1\} \cup \mathbb{G}_1 \cup \mathbb{G}_2 \cup \mathbb{G}_T)^* \rightarrow \{0, 1\} \times \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T$). Before giving a high-level description of our proof, we identify two different types of ROM queries (in fact, these are types of group elements that the verifier has access to).

Types of ROM queries. Consider a SNARG in the GGM + ROM with the simplifications described above. We observe that in the Maurer GGM, since the verifier cannot depend on the description of group elements that it receives, the group element that the verifier uses to query the

⁹SNARG verifiers are sometimes considered to be deterministic. However, we can rule out SNARGs with a randomized verifier. In fact, the transformations that we will see later on in this section will have the verifier end up being randomized.

random oracle can be described in terms of linear functions of the group elements in the common reference string and the element sent as the SNARG proof Π .

To explain in more detail, we first need some notation. Here, we use additive notation for the group. Specifically, a $\mathbb{G}_1$ element with discrete logarithm q will be denoted by $[q]_1$. Similarly, $[q]_2$ and $[q]_T$ are group elements in $\mathbb{G}_2$ and $\mathbb{G}_T$, respectively. If it is not relevant which of the groups an element is in we write $[q]_*$. In this notation the following operations are efficient:

- For $z \in \{1, 2, T\}$ and $x, y \in \mathbb{Z}_p$ two $\mathbb{G}_z$ elements $[a]_z, [b]_z$ can be linearly combined

$$x[a]_z + y[b]_z = [xa + yb]_z$$

- A $\mathbb{G}_1$ element $[a]_1$ and a $\mathbb{G}_2$ element $[b]_2$ can be multiplied $[a]_1 \cdot [b]_2 = [ab]_T$.

We call the $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{G}_T$ elements in the common reference string $[\text{crs}_1]_1$, $[\text{crs}_2]_2$, and $[\text{crs}_T]_T$, respectively. Observe that the common reference string includes the generators of $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$, and so these vectors all also have the value $1 \in \mathbb{Z}_p$.

Consider a query $[q]_z$ for $z \in \{1, 2, T\}$ that the verifier makes to the random oracle. As we are in the Maurer GGM, the only things that the verifier can do to generate group elements are linear operations on existing group elements and pair $\mathbb{G}_1$ with $\mathbb{G}_2$ elements. We have three cases depending on whether the query is a $\mathbb{G}_1$, $\mathbb{G}_2$, or $\mathbb{G}_T$ element:

- $\mathbb{G}_1$: $[q]_1 = a \cdot [\Pi]_1 + \langle \mathbf{b}, [\text{crs}_1]_1 \rangle$
- $\mathbb{G}_2$: $[q]_2 = \langle \mathbf{b}, [\text{crs}_2]_2 \rangle$
- $\mathbb{G}_T$: $[q]_T = [\Pi]_1 \cdot \langle \mathbf{a}, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}, [\text{crs}_T]_T \rangle$

Above, $a \in \mathbb{Z}_p$ and $\mathbf{a}$, $\mathbf{b}$ and $\mathbf{c}$ are vectors of $\mathbb{Z}_p$ elements.

We say that a query is *proof-dependent* if (1) it is a $\mathbb{G}_1$ element and $a \neq 0$, or (2) it is a $\mathbb{G}_T$ element and $\langle \mathbf{a}, [\text{crs}_2]_2 \rangle \neq [0]_2$. Any query that is not proof-dependent (which includes all $\mathbb{G}_2$ element queries) is considered *proof-independent*.

Proof overview. The main idea of our proof is to first remove the random oracle, thereby transforming the SNARG in the GGM + ROM to one simply in the GGM, and then showing that such SNARGs do not exist. The next sections are as follows:

1. In Section 2.2.1 we show how to remove the ROM queries in the case where they are all proof-dependent.
2. In Section 2.2.2 we explain how to extend the previous item, showing how to remove all ROM queries whether they be proof-dependent or independent.
3. In Section 2.2.3 we show that there do not exist one group element SNARGs in the GGM.

Put together, they conclude the proof of Theorem 1.3. See Section 6 for a full formal theorem statement and proof.

2.2.1 Proof-dependent ROM query

We show how to transform a SNARG in the GGM + ROM, where all of the verifier's queries are proof-dependent, into one in the GGM without a random oracle. Our construction removes the random oracle by having each party separately (and with no coordination) lazily sample a random oracle. We first describe the transformation and then discuss further. Let $(\mathcal{S}^{\mathcal{G}}, \mathcal{P}^{\mathcal{G}, \mathcal{O}}, \mathcal{V}^{\mathcal{G}, \mathcal{O}})$ be a SNARG in the GGM + ROM where all of the verifier's ROM queries are proof-dependent. The new SNARG is as follows:

- $\mathcal{S}'^{\mathcal{G}}(1^n, 1^\lambda)$: Compute $\text{crs} \leftarrow \mathcal{S}^{\mathcal{G}}(1^n, 1^\lambda)$. Output crs .
- $\mathcal{P}'^{\mathcal{G}}(\text{crs}, \mathbb{x}, \mathbb{w})$: Compute $[\Pi]_1 \leftarrow \mathcal{P}^{\mathcal{G}, \mathcal{O}_{\mathcal{P}}}(\text{crs}, \mathbb{x}, \mathbb{w})$, where $\mathcal{O}_{\mathcal{P}}$ is a lazily sampled random oracle. Output Π .
- $\mathcal{V}'^{\mathcal{G}}(\text{crs}, \mathbb{x}, \mathbb{w})$: Run $\mathcal{V}^{\mathcal{G}, \mathcal{O}_{\mathcal{V}}}(\text{crs}, \mathbb{x})$, where $\mathcal{O}_{\mathcal{V}}$ is a lazily sampled random oracle. Accept if and only if $\mathcal{V}$ accepts.

By construction, the new SNARG does not rely on a random oracle. Furthermore, it is immediate that soundness still holds as the original was sound even when the malicious prover had query access to the random oracle seen by the verifier. However, it is not clear at all that the new protocol satisfies *completeness*.

To thoroughly understand why the protocol remains complete, we first consider the case where the verifier makes a single query to the random oracle (the prover may still make many queries). Let p be the probability that the prover and verifier both make the same query to the random oracle. We are in one of two cases:

1. *The probability p is small*: Since the probability that $\mathcal{P}$ and $\mathcal{V}$ make the same query is small, their view (in the original SNARG) of the random oracle with significant probability is indistinguishable from two independent random oracles. Thus, in this case, our transformation changes little, and completeness does not change when prover and verifier queries are distinct.
2. *The probability p is large*: In this case the prover makes the query $[q]_*$ with high probability. We henceforth condition on the prover making the same query as the verifier. Let $f(\text{crs}, [\Pi]_1)$ be the function computing the verifier's query $[q]_*$ from the reference string and the proof. Then, informally writing the verifier as receiving the query answer to make its decision, the verifier's acceptance probability can be written as:

$$\Pr_{\text{crs}, \mathcal{O}} \left[\mathcal{V}^{\mathcal{G}}(\text{crs}, \mathbb{x}, \alpha) = 1 \left| \begin{array}{l} [\Pi]_1 \leftarrow \mathcal{P}^{\mathcal{G}, \mathcal{O}}(\text{crs}, \mathbb{x}, \mathbb{w}) \\ [q]_* = f([\Pi]_1, \text{crs}) \\ \alpha = \mathcal{O}([q]_*) \end{array} \right. \right].$$

For every i , let $\mathcal{P}_i^{\mathcal{G}, \mathcal{O}}(\text{crs}, \mathbb{x}, \mathbb{w})$ be the prover algorithm run up until it makes the i -th ROM query, and let this query be its output. Let i^* be the random variable representing the index of the prover ROM query that is equal to the verifier's query, so that $[q]_* = \mathcal{P}_{i^*}^{\mathcal{G}, \mathcal{O}}(\text{crs}, \mathbb{x}, \mathbb{w})$. We will show below that f is a *permutation* with respect to $[\Pi]_1$, so that there exists f^{-1} where $f^{-1}([q]_*, \text{crs}) = [\Pi]_1$. Thus, we can rewrite the above probability as:

$$= \Pr_{\text{crs}, \mathcal{O}} \left[\mathcal{V}^{\mathcal{G}}(\text{crs}, \mathbb{x}, \alpha) = 1 \left| \begin{array}{l} [q]_* = \mathcal{P}_{i^*}^{\mathcal{G}, \mathcal{O}}(\text{crs}, \mathbb{x}, \mathbb{w}) \\ [\Pi]_1 = f^{-1}([q]_*, \text{crs}) \\ \alpha = \mathcal{O}([q]_*) \end{array} \right. \right].$$

Observe that the verifier is oblivious to all of the ROM queries made by the prover up to index i^* . Moreover, the prover's algorithm does not depend on the answer to the query $[q]_*$. Thus, we can separate the single random oracle into two independent random oracles:

$$= \Pr_{\text{crs}, \mathcal{O}_{\mathcal{P}}, \mathcal{O}_{\mathcal{V}}} \left[\mathcal{V}^{\mathcal{G}}(\text{crs}, \mathbb{x}, \alpha) = 1 \left| \begin{array}{l} [q]_* = \mathcal{P}_{i^*}^{\mathcal{G}, \mathcal{O}_{\mathcal{P}}}(\text{crs}, \mathbb{x}, \mathbb{w}) \\ [\Pi]_1 = f^{-1}([q]_*, \text{crs}) \\ \alpha = \mathcal{O}_{\mathcal{V}}([q]_*) \end{array} \right. \right].$$

Now we can reverse the process we did before to bring back the prover running in full, but now where the two parties run with independent oracles:

$$= \Pr_{\text{crs}, \mathcal{O}_{\mathcal{P}}, \mathcal{O}_{\mathcal{V}}} \left[\mathcal{V}^{\mathcal{G}}(\text{crs}, \mathbb{X}, \alpha) = 1 \left| \begin{array}{l} [\Pi]_1 \leftarrow \mathcal{P}^{\mathcal{G}, \mathcal{O}_{\mathcal{P}}}(\text{crs}, \mathbb{X}, \mathbb{W}) \\ [q]_* = f([\Pi]_1) \\ \alpha = \mathcal{O}_{\mathcal{V}}([q]_*) \end{array} \right. \right],$$

which is precisely the SNARG following the transformation.

Showing that f is a permutation. The only thing we are missing is to show that f is a permutation. To see this, let us consider how q is computed. We describe the case of the query being a $\mathbb{G}_1$ element, where the $\mathbb{G}_T$ case will be analogous, when replacing a with $\langle \mathbf{a}, \text{crs}_2 \rangle$ and $\langle \mathbf{b}, \text{crs}_1 \rangle$ with $\langle \mathbf{b}, \text{crs}_1 \otimes \text{crs}_2 \rangle + \langle \mathbf{c}, \text{crs}_T \rangle$.

$$q = a \cdot \Pi + \langle \mathbf{b}, \text{crs}_1 \rangle.$$

Recall that we are under the assumption that $[q]_*$ is proof-dependent, meaning that $a \neq 0$. Thus, we can rewrite the above as

$$\Pi = \frac{q - \langle \mathbf{b}, \text{crs}_1 \rangle}{a}.$$

In other words, the proof string Π is uniquely defined by $[q]_*$ and by the common reference string, and so f is a permutation. Observe that it is in the final step, where we claim that Π is uniquely defined by $[q]_*$, that we crucially use the fact that the proof contains a single group element, as otherwise the proof could not be uniquely defined by one dependent query.

Handling multiple queries. Extending the analysis from a single ROM query to multiple queries follows a similar high-level structure, but requires more technical work for controlling the intersection between the prover's and verifier's queries. We show that, despite this complication, the core ideas carry over and suffice to establish the complete proof for the general case.

Furthermore, extending to a ROM that accepts multiple group elements as inputs adds complexity to the proof, but the same general ideas still apply, and we give a formal proof for this general case as well.

2.2.2 Handling any type of ROM query

In the previous section, we showed how to transform one group element SNARGs in the GGM + ROM with only proof-dependent queries into SNARGs in the pure GGM. However, this transformation fails when the verifier makes proof-independent queries. In this section, we describe how to remove all proof-independent queries without affecting the proof-dependent ones. Thus, the full transformation is:

- Transform $(\mathcal{S}^{\mathcal{G}}, \mathcal{P}^{\mathcal{G}, \mathcal{O}}, \mathcal{V}^{\mathcal{G}, \mathcal{O}})$ into a new SNARG scheme $(\mathcal{S}'^{\mathcal{G}}, \mathcal{P}'^{\mathcal{G}, \mathcal{O}}, \mathcal{V}'^{\mathcal{G}, \mathcal{O}})$ in the GGM + ROM where, for any common reference string and for any proof in the image of the prover, the verifier only makes proof-dependent queries to the random oracle.
- Transform $(\mathcal{S}'^{\mathcal{G}}, \mathcal{P}'^{\mathcal{G}, \mathcal{O}}, \mathcal{V}'^{\mathcal{G}, \mathcal{O}})$ into a new SNARG scheme $(\mathcal{S}''^{\mathcal{G}}, \mathcal{P}''^{\mathcal{G}}, \mathcal{V}''^{\mathcal{G}})$ in the GGM (without the random oracle).

The high-level idea of our transformation to get rid of proof-independent queries is that, since a ROM query is independent of the proof, it is essentially fixed once the common reference string and the instance are fixed. Thus, the answer is also fixed and outside the prover's control. Thus, there is no difference between it being chosen as the output of a random oracle and it being written down as part of the common reference string.

We begin by describing the transformation in the case that all queries are proof-independent, and then explain how the construction must change if, as in the general case, the queries can be either proof-dependent or proof-independent.

All queries are proof-independent. Consider a SNARG $(\mathcal{S}^{\mathcal{G}}, \mathcal{P}^{\mathcal{G}, \mathcal{O}}, \mathcal{V}^{\mathcal{G}, \mathcal{O}})$ where the verifier makes t ROM queries, all of which are proof-independent. We construct a new SNARG where all of these queries are removed (in fact, since these are all of the verifier's ROM queries, the random oracle is completely removed):

- $\mathcal{S}'^{\mathcal{G}}(1^n, 1^\lambda)$:
 1. Let $\text{crs} \leftarrow \mathcal{S}^{\mathcal{G}}(1^n, 1^\lambda)$.
 2. For $i \in [t]$ sample $r_i \leftarrow \{0, 1\}$ uniformly at random.
 3. Output $\text{crs}' = (\text{crs}, r_1, \dots, r_t)$.
- $\mathcal{P}'^{\mathcal{G}}(\text{crs}' = (\text{crs}, r_1, \dots, r_t), \mathbb{x}, \mathbb{w})$:
 1. Let $[\Pi']_1$ be an arbitrary $\mathbb{G}_1$ element (for concreteness we can pick the generator of $\mathbb{G}_1$.)
 2. Run $\mathcal{V}(\text{crs}, \mathbb{x}, [\Pi']_1)$ where, on its i -th ROM query $[q_i]_*$, answer with r_i .
 3. Run $[\Pi]_1 = \mathcal{P}^{\mathcal{G}, \mathcal{O}'}(\text{crs}, \mathbb{x}, \mathbb{w})$, where $\mathcal{O}'$ returns r_i on query $[q_i]_*$ and is a lazily sampled random oracle at all other points.
 4. Output $[\Pi]_1$.
- $\mathcal{V}'^{\mathcal{G}}(\text{crs}' = (\text{crs}, r_1, \dots, r_t), \mathbb{x}, [\Pi]_1)$:
 1. Simulate $\mathcal{V}(\text{crs}, \mathbb{x}, [\Pi]_1)$, answering the i -th random oracle query $[q_i]_*$ with answer r_i .
 2. Accept if and only if $\mathcal{V}$ accepts.

Completeness and soundness of this new scheme follow by construction since all of the verifier's queries are proof-independent, putting the query answers into the reference string grants all parties an identical view of the verifier's query/answer pairs as in the ROM. Moreover, since these queries are independent of the proof, the prover gets the same queries when running the verifier with $[\Pi']_1$ as are made with the real proof $[\Pi]_1$.

Handling mixed query types. Recall our assumption that the verifier is non-adaptive with respect to the random oracle. This means that, without loss of generality, it can be thought of as making all queries at once (i.e., both proof-dependent and proof-independent ones). However, as the argument above fails for proof-dependent queries, we can only replace the proof-independent ones with randomness from the common reference string. However, which query is proof-dependent and which is proof-independent may depend on crs and $\mathbb{x}$, and so we cannot tell “at design time” which query is which. This initially seems difficult, as proof dependence is a property that depends on the (possibly hidden) exponent of the group elements that comprise the queries.

However, upon closer inspection, both the verifier (and therefore also the prover) can recognize whether a query is proof-dependent or not. Recall that all $\mathbb{G}_2$ queries are proof-independent. Any $\mathbb{G}_1$ query is of the form

$$[q]_1 = a \cdot [\Pi]_1 + \langle \mathbf{b}, [\text{crs}_1]_1 \rangle ,$$

where proof dependence means that $a \neq 0$. It is easy to check whether $a \neq 0$.

Finally, in the $\mathbb{G}_T$ case, the query has the form

$$[q]_T = [\Pi]_1 \cdot \langle \mathbf{a}, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}, [\text{crs}_T]_T \rangle ,$$

and proof-dependence means that $\langle \mathbf{a}, [\text{crs}_2]_2 \rangle \neq 0$. This test is equivalent to checking $\langle \mathbf{a}, [\text{crs}_2]_2 \rangle \neq [0]_2$, which can be done without having to compute any discrete logarithms.

2.2.3 One group element SNARGs do not exist

At this point, following all of the previous sections, we have completely removed the random oracle from the SNARG, and we are left with one group element SNARG in the GGM. We show Theorem 1.3: assuming one-way functions in the standard model, there are no one group element SNARGs in the Maurer GGM. More specifically, letting G be a PRG, we show that a one group element SNARG for the relation $\mathcal{R} = \{(G(s), s)\}$ can be used to distinguish between the distributions $\mathcal{D}_U$, which is a uniformly random string, and $\mathcal{D}_G$, which is the output of the PRG with a uniformly random seed. We note that in order to give a more intuitive explanation, the high-level overview described in this section differs slightly from that of the formal proof in the main body of the paper (see proof of Lemma 6.11).

This proof can be viewed as an extension to Groth's lower bound [Gro16], who proves a similar theorem under the restriction that the verifier is a *conjunction* of pairing-equality checks.¹⁰ We do not rely on the verifier being a conjunction. However, as we show below, we prove that any verifier must have a weaker structure that is sufficient for ruling out such SNARGs.

On the structure of the verifier. In the Maurer GGM, the verifier's only way of obtaining useful information is through GGM zero-test queries (equivalently, equality tests). Concretely, given a group element $[q]_*$, the verifier may query an oracle to determine whether q is zero. To analyze the structure of the verifier's algorithm, we once again use the notion of proof-dependent queries, previously applied to characterize queries to the ROM, and here adapted to capture zero-test oracle queries.

Recall that a proof-dependent query has one of the following two structures, depending on which group it belongs to:

- $\mathbb{G}_1$: $[q]_1 = a \cdot [\Pi]_1 + \langle \mathbf{b}, [\text{crs}_1]_1 \rangle$ where $a \neq 0$
- $\mathbb{G}_T$: $[q]_T = [\Pi]_1 \cdot \langle \mathbf{a}, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}, [\text{crs}_T]_T \rangle$ where $\langle \mathbf{a}, [\text{crs}_2]_2 \rangle \neq 0$.

Observe that in both cases there is a single and unique proof Π so that $q = 0$ and the zero-test returns 1 (e.g., in the $\mathbb{G}_1$ case $[\Pi]_1 = \frac{\langle \mathbf{b}, [\text{crs}_1]_1 \rangle}{a}$).

Using these observations, we show that in order for the verifier to accept an instance sampled from $\mathcal{D}_G$, there must be a proof-dependent zero-check that passes. As we will see later, this is enough structure to distinguish $\mathcal{D}_U$ from $\mathcal{D}_G$.

¹⁰Actually, Groth shows that there do not exist SNARGs of any length with such verifiers in the case where the entire proof is comprised of elements of the same group (e.g., the proof is only $\mathbb{G}_1$ elements). This, in particular, rules out the one group element case.

Claim 2.2 (informal). *For $\mathbf{x} \leftarrow \mathcal{D}_G$ and crs sampled as in the SNARG, if there exists $[\Pi]_1$ so that $\mathcal{V}(\text{crs}, \mathbf{x}, [\Pi]_1) = 1$ then (except with negligible probability) there exists a proof-dependent zero-test query for which the answer is 1.*

Proof sketch. We show that for almost every $\mathbf{x}$ in the support of $\mathcal{D}_U$ (i.e., instances $\mathbf{x} \notin L(\mathcal{R})$) the verifier rejects if all proof-dependent queries evaluate to 0 with overwhelming probability. The claim follows as it is hard to distinguish $\mathcal{D}_U$ and $\mathcal{D}_G$.¹¹

Fix $\mathbf{x} \notin L(\mathcal{R})$ and some common reference string crs consider what it means for the verifier to accept when all of the proof-dependent queries evaluate to 0. Every proof-dependent query $[q_i]_*$ returns 1 for exactly one group element $[\Pi_i]_1$. Since the verifier accepts if all these queries return 0, a malicious prover will cause the verifier to accept if it sends $[\Pi]_1$ that is different from all of the elements $[\Pi_i]_1$. Since the verifier makes at most $\text{poly}(\lambda)$ queries, a prover that chooses a random group element for $[\Pi]_1$ causes the verifier to accept with probability $= 1 - \text{negl}(\lambda)$.

Since the SNARG is sound, we conclude that the verifier accepts when all proof-dependent queries evaluate to 0 with negligible probability.

For $\mathbf{x}' \leftarrow \mathcal{D}_G$, too, the verifier accepts when all proof-dependent queries evaluate to 0 with negligible probability. That is because $\mathbf{x} \leftarrow \mathcal{D}_U$ and $\mathbf{x}' \leftarrow \mathcal{D}_G$ are computationally indistinguishable. If the verifier acted significantly different on $\mathbf{x}'$ then the PRG distinguisher can simulate the SNARG and detect the difference. $\square$

Ruling out one group element SNARGs. Having established some structural understanding of the SNARG verifier, we now leverage this insight to construct a distinguisher against the PRG. To this end, we introduce a polynomial-time subroutine $\mathbf{A}_V$, which takes as input a common reference string crs , an instance $\mathbf{x}$, and a function f as a polynomial with one of the following forms (matching to the relevant group):

- $\mathbb{G}_1$: $f([\Pi]_1, [\text{crs}_1]_1) = a \cdot [\Pi]_1 + \langle \mathbf{b}, [\text{crs}_1]_1 \rangle$ where $a \neq 0$
- $\mathbb{G}_T$: $f([\Pi]_1, [\text{crs}_1]_1, [\text{crs}_2]_2, [\text{crs}_T]_T) = [\Pi]_1 \cdot \langle \mathbf{a}, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}, [\text{crs}_T]_T \rangle$ where $\langle \mathbf{a}, [\text{crs}_2]_2 \rangle \neq [0]_2$.

The algorithm $\mathbf{A}_V(\text{crs}, \mathbf{x}, f)$ outputs a group element $[\Pi]_1$ that passes the zero-test if plugged in to a query of the form of f , i.e., in the $\mathbb{G}_1$ case, such that $f([\Pi]_1, [\text{crs}_1]_1) = 0$.

In the case where f is a query made by the verifier and $\mathbf{x}$ is sampled from $\mathcal{D}_G$, such an algorithm can be derived using ideas similar to those used in Groth's lower-bound [Gro16]. The algorithm essentially uses the fact that the verifier's queries form a linear subspace to progressively learn which answers will cause this query to pass. In the full proof (see Lemma 6.11) we give full details, but we note that they don't have the same high-level of abstraction as in this overview.

We now have all the tools we need in order to break the PRG:

Lemma 2.3. *If there exists a one group element SNARG in the Maurer GGM, then there exists an efficient distinguisher between $\mathcal{D}_U$ and $\mathcal{D}_G$.*

Proof sketch. We describe the distinguisher $\mathbf{D}$. Suppose that the verifier makes at most t queries. Given an instance $\mathbf{x}$ the distinguisher $\mathbf{D}$ runs as follows:

¹¹Observe that the view of the verifier of $[\Pi]_1$ is only in the proof-dependent queries, so we can efficiently simulate this behaviour without knowing $[\Pi]_1$. Thus, showing that the PRG fools this efficient algorithm implies the same behavior for any $[\Pi]_1$ that induces the same verifier view.

1. Sample crs as in the SNARG.
2. For every $i = 1$ to t :
 - (a) Run the verifier up to the i -th query (recall that we have established in prior sections that proof-dependent queries can be recognized), answering all previous proof-dependent queries with 0.
 - (b) Compute $[\Pi_i]_1 = \mathbf{A}_{\mathcal{V}}(\text{crs}, \mathbb{x}, f_i)$ where f_i is the structure of the i -th query (which can be efficiently derived by tracking all of the verifier's GGM queries).
3. If there exists i such that $\mathcal{V}(\text{crs}, \mathbb{x}, [\Pi_i]_1) = 1$ then output 1. Otherwise, output 0.

We show that $\mathbf{D}$ distinguishes between the distributions by showing that, for instances sampled from $\mathcal{D}_b$, the distinguisher outputs the bit b with overwhelming probability.

Consider $\mathbb{x} \leftarrow \mathcal{D}_{\mathbf{G}}$. By completeness, there must be $[\Pi]_1$ that causes the verifier to accept. As shown in Claim 2.2, in order for this to be the case, it must be that $[\Pi]_1$ satisfies one of the proof-dependent zero-test queries. Let i^* be the index of the first proof-dependent zero test that passes. Since this element is unique, for $i = i^*$ the distinguisher will get from $\mathbf{A}_{\mathcal{V}}$ the output $[\Pi_{i^*}]_1 = [\Pi]_1$, and so $\mathcal{V}$ will accept on $[\Pi_{i^*}]_1$. Thus, in this case $\mathbf{D}$ will output 1.

Now consider $\mathbb{x} \leftarrow \mathcal{D}_U$. Since $\mathbf{D}$ is efficient and, after generating the common reference string, makes no further use of its randomness, it can be simulated by a malicious SNARG prover. By soundness of the SNARG, the verifier must therefore reject all candidate proofs $[\Pi_i]_1$ produced by $\mathbf{D}$. We conclude that the distinguisher outputs 0 for such instances. $\square$

3 Preliminaries

3.1 Idealized models

The SNARGs in this paper are proved secure in the generic group model with an additional random oracle. We describe these oracles:

Definition 3.1 (Maurer’s generic group). *Maurer’s generic (asymmetric) group model (GGM) introduces group elements as an atomic unit of information. A group element can be from one of three prime- p order groups $\mathbb{G}_1$, $\mathbb{G}_2$, or $\mathbb{G}_T$, generated by elements g_1 , g_2 , and g_T respectively, which every party implicitly receives. In this model, every algorithm can only interact with these group elements by using one of the three oracles $\mathcal{G} = (\mathcal{G}_l, \mathcal{G}_e, \mathcal{G}_c)$ defined as:*

1. $\mathcal{G}_l$ takes as input two group elements $g = g_i^a, h = g_i^b$ from the same group $\mathbb{G}_i$ with $i \in \{1, 2, T\}$ and $a, b \in \mathbb{Z}_p$ and outputs g_i^{a+b} in $\mathbb{G}_i$,
2. $\mathcal{G}_e$ takes as input two group elements $g = g_1^a \in \mathbb{G}_1, h = g_2^b \in \mathbb{G}_2$ for $a, b \in \mathbb{Z}_p$ and outputs $g_T^{a \cdot b}$ in $\mathbb{G}_T$,
3. $\mathcal{G}_c$ takes as input one group elements $g = g^a$ for some $a \in \mathbb{Z}_p$ from the group $\mathbb{G}_i$ with $i \in \{1, 2, T\}$ and outputs 1 if $a = 0$ and outputs 0 otherwise.

Notice that, in this model, group elements do not have a bit representation. This way the only way an algorithm can learn bits from a group element is by querying $\mathcal{G}_c$.

Definition 3.2 (Random oracle in the presence of a generic group). *Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be a generic group as defined above. Let $\text{RO}(1^\lambda)$ be the set of all functions*

$$\mathcal{O}: (\{0, 1\} \cup \mathbb{G}_1 \cup \mathbb{G}_2 \cup \mathbb{G}_T)^* \rightarrow \{0, 1\}^\lambda \times \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T.$$

A construction in the GGM + ROM is a construction where all parties have oracle access to a uniformly random $\mathcal{O} \leftarrow \text{RO}(1^\lambda)$.

Use of GGM + ROM in this paper. In this paper we use the GGM + ROM differently in our constructions compared to our lower bound. We briefly explain how it is used in either case:

- Constructions (Section 5): In our construction we only require the random oracle to receive as input n bits (where n is the instance size) and a single $\mathbb{G}_1$ element and output λ bits. That is, we use the GGM + ROM where $\text{RO}(1^\lambda)$ is only required to include functions of the form $\mathcal{O}: \{0, 1\}^n \times \mathbb{G}_1 \rightarrow \{0, 1\}^\lambda$.
- Lower-bound (Section 6): Our lower-bound holds for the full class of GGM + ROM as defined above. However, for convenience the proof uses oracles that output a single symbol. That is, it is described for the class $\text{RO}(1)$ of functions of the form

$$\mathcal{O}: (\{0, 1\} \cup \mathbb{G}_1 \cup \mathbb{G}_2 \cup \mathbb{G}_T)^* \rightarrow \{0, 1\} \times \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T.$$

This is without loss of generality, as all parties can always append each query with $\log \lambda$ bits to generate λ different outputs. This affects the running times and query complexity of the parties by a polynomial factor (as λ is polynomially related to all other parameters). These factors do not come into play in the proof of our lower bound, and so it is safe to work with the assumption that the output length is 1.

3.2 Succinct non-interactive arguments

Definition 3.3. A succinct non-interactive argument (SNARG) for a relation $\mathcal{R} = \{(\mathbf{x}, \mathbf{w})\}$ is defined by a triple $(\mathcal{S}, \mathcal{P}, \mathcal{V})$ as follows:

- Syntax.
 - The setup algorithm $\mathcal{S}$ receives an input size parameter 1^n and a security parameter 1^λ . It outputs a common reference string crs .
 - The (honest) prover algorithm $\mathcal{P}$ receives as input a common reference string crs , instance $\mathbf{x} \in \{0, 1\}^n$, and witness $\mathbf{w} \in \{0, 1\}^m$. It outputs a proof Π .
 - The verifier algorithm $\mathcal{V}$ receives as an instance $\mathbf{x} \in \{0, 1\}^n$, and a proof Π . It outputs a bit $b \in \{0, 1\}$.
- Completeness. A SNARG has completeness error σ if for all $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ and $\lambda \in \mathbb{N}$:

$$\Pr \left[\mathcal{V}(\mathbf{x}, \Pi) = 1 \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}(1^{|\mathbf{x}|}, 1^\lambda) \\ \Pi \leftarrow \mathcal{P}(\text{crs}, \mathbf{x}, \mathbf{w}) \end{array} \right] \geq 1 - \sigma(\lambda, |\mathbf{x}|).$$

- Soundness. A SNARG has (adaptive) soundness with error β if for every $\lambda \in \mathbb{N}$, $n \in \mathbb{N}$, and s -size prover $\mathcal{P}'$:

$$\Pr \left[\begin{array}{l} |\mathbf{x}| \leq n \\ \mathbf{x} \notin L(\mathcal{R}) \\ \wedge \mathcal{V}(\text{st}, \mathbf{x}, \Pi) = 1 \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}(1^n, 1^\lambda) \\ (\mathbf{x}, \Pi) \leftarrow \mathcal{P}'(\text{crs}) \end{array} \right] \leq \beta(\lambda, n, s).$$

- Succinctness. For every large enough λ , $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ and crs in the image of $\mathcal{S}(1^{|\mathbf{x}|}, 1^\lambda)$, we have $|\Pi| = o(|\mathbf{w}|)$ for $\Pi = \mathcal{P}(\text{crs}, \mathbf{x}, \mathbf{w})$.

A SNARG with the following additional knowledge property is known as a SNARK:

- Knowledge soundness. A SNARG has (adaptive) knowledge soundness (in which case we refer to it as a SNARK) with knowledge κ if there exists an expected polynomial time PPT extractor $\mathcal{E}$ so that for every $\lambda \in \mathbb{N}$, $n \in \mathbb{N}$, and s -size prover $\mathbf{P}'$,

$$\Pr \left[\begin{array}{l} |\mathbf{x}| \leq n \\ (\mathbf{x}, \mathbf{w}) \notin \mathcal{R} \\ \wedge \mathcal{V}(\text{st}, \mathbf{x}, \Pi) = 1 \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}(1^n, 1^\lambda) \\ (\mathbf{x}, \Pi) \leftarrow \mathcal{P}'(\text{crs}) \\ \mathbf{w} \leftarrow \mathcal{E}(\mathcal{P}', \text{crs}) \end{array} \right] \leq \kappa(\lambda, n, s).$$

4 Linear map commitments

In this section we define linear map commitments, describe a construction in the generic group model due to Lai and Malavolta [LM19] and give a tight analysis of the errors in their linear map commitment. In fact, we prove a stronger security notion than the one proven in [LM19]; we prove multi-extraction.

We begin by defining linear map commitments:

Definition 4.1. *A linear map commitment over a finite field $\mathbb{F}$ is a tuple of algorithms*

$$(\text{Com.Setup}, \text{Com.Commit}, \text{Com.Open}, \text{Com.Verify}),$$

with the following properties:

- **Correctness.** *The linear map commitment has (perfect) correctness if for every $\lambda, \ell, q \in \mathbb{N}$, vector $x \in \mathbb{F}^\ell$ and matrix $M \in \mathbb{F}^{\ell \times q}$:*

$$\Pr \left[\text{Com.Verify}(\text{crs}, c, M, \alpha, \text{pf}) = 1 \mid \begin{array}{l} \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell, q) \\ (c, \text{aux}) \leftarrow \text{Com.Commit}(\text{crs}, x) \\ \text{pf} \leftarrow \text{Com.Open}(\text{crs}, c, \text{aux}, M) \\ \alpha = Mx \end{array} \right] = 1.$$

- **Multi-extraction.** *The linear map commitment has multi-extraction error ε if there exists an extractor for every $\lambda, \ell, q, n, s \in \mathbb{N}$, there exists a polynomial-time extractor Ext such that for any s -size $n + 1$ -stage adversary $\mathcal{A} = (\mathcal{A}_1, \dots, \mathcal{A}_n, \mathcal{A}')$:*

$$\Pr \left[\begin{array}{l} \left(\begin{array}{l} \text{Com.Verify}(\text{crs}, c_{i^*}, M, \alpha, \text{pf}) = 1 \\ \wedge Mx_{i^*} \neq \alpha \end{array} \right) \\ \vee \\ \left(\begin{array}{l} \exists i_1, i_2 \in [n] : \\ c_{i_1} = c_{i_2} \wedge x_{i_1} \neq x_{i_2} \end{array} \right) \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell, q) \\ \text{aux}_0 := \perp \\ \forall i \in [n] : \left(\begin{array}{l} (c_i, \text{aux}_i) \leftarrow \mathcal{A}_i(\text{crs}, \text{aux}_{i-1}) \\ x_i \leftarrow \text{Ext}(\mathcal{A}_1, \dots, \mathcal{A}_i, \text{crs}) \end{array} \right) \\ (i^*, M, \alpha, \text{pf}) \leftarrow \mathcal{A}'(\text{crs}, \text{aux}_n) \end{array} \right] \leq \varepsilon(\lambda, \ell, q, n, s).$$

Remark 4.2. The linear map commitment that we use will be secure in the generic group model. In this case, we refer to errors as a function of the number of queries made by the adversary rather than its circuit size.

We now show that the [LM19] linear map commitment has the above properties and give a tight analysis of security:

Theorem 4.3. *There is a linear map commitment in the bilinear GGM with prime order p with $p \geq 2^\lambda$ over the field $\mathbb{F} = \mathbb{Z}_p$ with multi-extraction error $\frac{4\ell s^2}{2^\lambda}$ against s -query adversaries (as long as $s > 60\ell^2 q^2$). The commitment and opening sizes are both 1 element of $\mathbb{G}_1$. The committing time is ℓ source group operations, the opening time is $q\ell^2$ source group operations, and the verification entails 3 pairing operations and $\ell(q + 1)$ additional group operations. The extraction time is $t_{\text{ext}}(s) = O(s)$.*

Proof. We begin by describing the construction:

Construction 4.4 ([LM19]). The linear map commitment is as follows. We assume all algorithms have access to a GGM oracle $\mathcal{G}$ with known prime order p , and all those that receive crs parse it as $\text{crs} = (\{G_j\}_{j \in [\ell]}, \{H_{a,i,j}\}_{a \in \{1,2\}, i \in [q], j \in [2\ell] \setminus \{\ell+1\}})$.

- **Com.Setup** $^{\mathcal{G}}(1^\lambda, \ell, q)$:
 1. $\beta, z_1, \dots, z_q \leftarrow \mathbb{Z}_p$.
 2. For all $j \in [\ell]$ set $G_j = g_1^{\beta^j}$.
 3. For all $a \in \{1, 2\}$, $i \in [q]$, and $j \in [2\ell]$ set $H_{a,i,j} = g_a^{z_i \beta^j}$.
 4. Output $\text{crs} = (\{G_j\}_{j \in [\ell]}, \{H_{a,i,j}\}_{a \in \{1,2\}, i \in [q], j \in [2\ell] \setminus \{\ell+1\}})$.
- **Com.Commit** $^{\mathcal{G}}(\text{crs}, x)$: Return $(c, \text{aux}) = (\prod_{j \in [\ell]} G_j^{x_j}, x)$.
- **Com.Open** $^{\mathcal{G}}(\text{crs}, c, \text{aux}, M)$: Parse $\text{aux} = x$ and return pf defined as follows:

$$\text{pf} = \prod_{i \in [q]} \prod_{j \in [\ell]} \prod_{j' \in [\ell] \setminus \{j\}} H_{1,i,\ell+1+j'-j}^{M_{i,j} \cdot x_{j'}}.$$

- **Com.Verify** $^{\mathcal{G}}(\text{crs}, c, M, \alpha, \text{pf})$: Accept if and only if:

$$e \left(c, \prod_{i \in [q]} \prod_{j \in [\ell]} H_{2,i,\ell+1-j}^{M_{i,j}} \right) = e \left(G_1, \prod_{i \in [q]} H_{2,i,\ell}^{\alpha_i} \right) \cdot e(\text{pf}, g_2).$$

We show that the scheme above has multi-extraction. We describe an extractor Ext such that:

$$\Pr \left[\begin{array}{l} \left(\begin{array}{l} \text{Com.Verify}(\text{crs}, c_{i^*}, M, \alpha, \text{pf}) = 1 \\ \wedge Mx_{i^*} \neq \alpha \end{array} \right) \\ \vee \\ \left(\begin{array}{l} \exists i_1, i_2 \in [n] : \\ c_{i_1} = c_{i_2} \wedge x_{i_1} \neq x_{i_2} \end{array} \right) \end{array} \right] \left[\begin{array}{l} \mathcal{G} \leftarrow \text{GGM}(1^\lambda) \\ \text{crs} \leftarrow \text{Com.Setup}^{\mathcal{G}}(1^\lambda, \ell, q) \\ \text{aux}_0 := \perp \\ \forall i \in [n] : \left(\begin{array}{l} (c_i, \text{aux}_i) \leftarrow \mathcal{A}_i^{\mathcal{G}}(\text{crs}, \text{aux}_{i-1}) \\ x_i \leftarrow \text{Ext}^{\mathcal{G}}(\mathcal{A}_1, \dots, \mathcal{A}_i, \text{crs}) \end{array} \right) \\ (i^*, M, \alpha, \text{pf}) \leftarrow \mathcal{A}'^{\mathcal{G}}(\text{crs}) \end{array} \right] \leq \frac{4\ell s^2}{p} \leq \frac{4\ell s^2}{2^\lambda}.$$

We describe the extractor $\text{Ext}^{\mathcal{G}}$ on inputs $\mathcal{A}_1, \dots, \mathcal{A}_i$ and $\text{crs} = (\{G_j\}_{j \in [\ell]}, \{H_{a,i,j}\}_{a \in \{1,2\}, i \in [q], j \in [2\ell] \setminus \{\ell+1\}})$.¹²

1. Initialize lists Λ_1 , Λ_2 , and Λ_T . These lists will hold entries of the form (g, f) where for $a \in \{1, 2, T\}$, $g \in \mathbb{G}_a$ and f is a polynomial over formal variables to be defined in the following item. Λ_1 is initiated as $\{(g_1, 1)\}$, Λ_2 is initiated as $\{(g_2, 1)\}$, and Λ_T is initially empty.
2. Let $X, Z_1, \dots, Z_q$ be formal variables. For every $j \in [\ell]$ add (G_j, X^j) to Λ_1 , and for every $a \in \{1, 2\}$, $i \in [q]$, and $j \in [2\ell]$ add $(H_{a,i,j}, Z_i \cdot X^j)$ to Λ_a .
3. Run $\mathcal{A}_1^{\mathcal{G}}(\text{crs})$ up to $\mathcal{A}_i^{\mathcal{G}}(\text{crs})$ in order, passing the auxiliary values between algorithms appropriately. Keep track of queries to the generic group as follows:
 - For a product query $h_1 \cdot h_2$ where $h_1, h_2 \in \mathbb{G}_a$ search Λ_a for entries (h_1, f_1) and (h_2, f_2) . If no such entries exist, output $\perp$. If they exist and there is no entry of the form $(h', f_1 + f_2)$ then add $(h_1 \cdot h_2, f_1 + f_2)$ to Λ_a .

¹² Ext only needs the trace of oracle calls made by $\mathcal{A}_1, \dots, \mathcal{A}_i$, and not additional access to the GGM oracle, but we write it in this way for simplicity.

- For a pairing query $e(h_1, h_2)$ where $(h_1, h_2) \in \mathbb{G}_1 \times \mathbb{G}_2$, search Λ_1 an entry (h_1, f_1) and Λ_2 for an entry (h_2, f_2) . If no such entries exist, output $\perp$. If they exist and there is no entry in Λ_T of the form $(h', f_1 \cdot f_2)$ then add $(e(h_1, h_2), f_1 \cdot f_2)$ to Λ_T .
4. The adversary $\mathcal{A}_i$ eventually outputs c_i . If there is no entry (c_i, f) in Λ_1 then output $\perp$. Otherwise, due to how we are adding elements Λ_1 , the polynomial f is over the variable X and has the form:
- $$f(X) = \gamma_0 + \sum_{j \in [\ell]} \gamma_j \cdot X^j + \sum_{i \in [q], j \in [2\ell] \setminus \{\ell+1\}} \gamma_{i,j} \cdot Z_i X^j,$$
- where $\gamma_0, \gamma_j, \gamma_{i,j} \in \mathbb{Z}$ for every i, j .
5. Output $x_i = (\gamma'_1, \dots, \gamma'_\ell) \in \mathbb{Z}_p^\ell$ where $\gamma'_j = \gamma_j \pmod p$.

We additionally consider the following oracle $\mathcal{G}'$ which has an identical interface to the standard GGM oracle, except that it keeps lists of polynomials over formal variables in exactly the same way as done by **Ext**, and generates a fresh label whenever a polynomial appears which has not been seen before.

1. Initialize lists Λ_1 , Λ_2 , and Λ_T . These lists will hold entries of the form (g, f) where for $a \in \{1, 2, T\}$, $g \in \mathbb{G}_a$ and f is a polynomial over formal variables to be defined in the following item.
2. Choose a random label g_1 and add $(g_1, 1)$ to Λ_1 . Similarly, add $(g_2, 1)$ to Λ_2 for a random label g_2 .
3. Let $X, Z_1, \dots, Z_q$ be formal variables. For every $j \in [\ell]$ add (G_j, X^j) to Λ_1 where G_j is a random label not previously used, and for every $a \in \{1, 2\}$, $i \in [q]$ and $j \in [2\ell]$ add $(H_{a,i,j}, Z_i \cdot X^j)$ to Λ_a where $H_{b,i,j}$ is a random label not previously used. When **Com.Setup** initiates **crs**, answer with the G_j and $H_{b,i,j}$ labels as appropriate.
4. For any query made by $\mathcal{A} = (\mathcal{A}_1, \dots, \mathcal{A}_n, \mathcal{A}')$, **Com.Verify**, or **Ext**, answer the query as follows:
 - For a product query $h_1 \cdot h_2$ where $h_1, h_2 \in \mathbb{G}_a$ search Λ_a for entries (h_1, f_1) and (h_2, f_2) . If no such entries exist, output $\perp$. If they exist and there is no entry of the form $(h', f_1 + f_2)$ then choose a random unused label h' and add $(h', f_1 + f_2)$ to Λ_a .
 - For a pairing query $e(h_1, h_2)$ where $(h_1, h_2) \in \mathbb{G}_1 \times \mathbb{G}_2$, search Λ_1 an entry (h_1, f_1) and Λ_2 for an entry (h_2, f_2) . If no such entries exist, output $\perp$. If they exist and there is no entry in Λ_T of the form $(h', f_1 \cdot f_2)$ then generate a random unused label h' and add $(h', f_1 \cdot f_2)$ to Λ_T .

We in show in Claim 4.5 that using $\mathcal{G}'$ in place of $\mathcal{G}$ yields the same success probability for the adversary up to a probability of $\frac{4\ell s^2}{p}$, and, in Claim 4.6, we show that when using $\mathcal{G}'$ the adversary will never succeed in the experiment. Together these proving multi-extraction of the scheme.

Claim 4.5.

$$\begin{aligned}
& \Pr \left[\begin{array}{l} \left(\text{Com.Verify}^{\mathcal{G}}(\text{crs}, \mathbf{c}_{i^*}, M, \alpha, \text{pf}) = 1 \right) \\ \wedge Mx_{i^*} \neq \alpha \\ \vee \\ \left(\begin{array}{l} \exists i_1, i_2 \in [n] : \\ \mathbf{c}_{i_1} = \mathbf{c}_{i_2} \wedge x_{i_1} \neq x_{i_2} \end{array} \right) \end{array} \right] \left[\begin{array}{l} \mathcal{G} \leftarrow \text{GGM}(1^\lambda) \\ \text{crs} \leftarrow \text{Com.Setup}^{\mathcal{G}}(1^\lambda, \ell, q) \\ \text{aux}_0 := \perp \\ \forall i \in [n] : \left((\mathbf{c}_i, \text{aux}_i) \leftarrow \mathcal{A}_i^{\mathcal{G}}(\text{crs}, \text{aux}_{i-1}) \right) \\ (i^*, M, \alpha, \text{pf}) \leftarrow \mathcal{A}'^{\mathcal{G}}(\text{crs}) \end{array} \right] \\
& - \Pr \left[\begin{array}{l} \left(\text{Com.Verify}^{\mathcal{G}'}(\text{crs}, \mathbf{c}_{i^*}, M, \alpha, \text{pf}) = 1 \right) \\ \wedge Mx_{i^*} \neq \alpha \\ \vee \\ \left(\begin{array}{l} \exists i_1, i_2 \in [n] : \\ \mathbf{c}_{i_1} = \mathbf{c}_{i_2} \wedge x_{i_1} \neq x_{i_2} \end{array} \right) \end{array} \right] \left[\begin{array}{l} \text{crs} \leftarrow \text{Com.Setup}^{\mathcal{G}'}(1^\lambda, \ell, q) \\ \text{aux}_0 := \perp \\ \forall i \in [n] : \left((\mathbf{c}_i, \text{aux}_i) \leftarrow \mathcal{A}_i^{\mathcal{G}'}(\text{crs}, \text{aux}_{i-1}) \right) \\ (i^*, M, \alpha, \text{pf}) \leftarrow \mathcal{A}'^{\mathcal{G}'}(\text{crs}) \end{array} \right] \\
& \leq \frac{4\ell s^2}{p}.
\end{aligned}$$

Proof. The view of all parties in the experiments only differs if there exist two polynomials f, f' added by $\mathcal{G}'$ to the list such that f and f' are not equivalent, and yet $\mathcal{G}$ gives them the same label.

Fix some f, f' in the lists of $\mathcal{G}'$. Since f, f' represent the values in $\mathcal{G}$ up to plugging in of the variables, this occurs only if

$$f(\beta, z_1, \dots, z_q) = f'(\beta, z_1, \dots, z_q),$$

which, by the polynomial identity lemma, occurs with probability at most $\frac{\max\{\deg(f), \deg(f')\}}{p}$, where $\deg$ denotes the total degree of the polynomial.

Observe that polynomials in Λ_1 have degree at most $2\ell + 1$, polynomials in Λ_2 have degree at most $2\ell + 1$, and polynomials in Λ_T have degree at most $4\ell + 2$. Thus, for f, f' , this probability is at most $\frac{4\ell+2}{p}$.

Finally, the number of equality queries done to $\mathcal{G}'$ is bounded as follows: $\ell + 2q \cdot (2\ell - 1)$ by the setup algorithm, s queries made by $\mathcal{A}$, none made by Ext (as it only makes the same queries as the stages in $\mathcal{A}$), and one in the verification test. Together, the total number of queries is bounded by $5q\ell + s$. Thus, the probability that there exists a pair of polynomials whose description differs in $\mathcal{G}'$ but is the same for $\mathcal{G}$ is bounded by

$$\begin{aligned}
\binom{5q\ell + s}{2} \cdot \frac{4\ell + 2}{p} &< \frac{50q^2\ell^3 + 25q^2\ell^2 + 20q\ell^2s + 10q\ell s + 2\ell s^2 + s^2}{p} \\
&< \frac{55q^2\ell^3 + 25q\ell^2s + 3\ell s^2}{p} \\
&< \frac{60q^2\ell^3s + 3\ell s^2}{p} \\
&< \frac{\ell s^2 + 3\ell s^2}{p} = \frac{4\ell s^2}{p}.
\end{aligned}$$

The inequalities above hold since $60\ell^2q^2 < s$, and using the mild assumption that $\ell \geq 5$. $\square$

Finally, we show that in the $\mathcal{G}'$ world, the multiextraction error is 0:

Claim 4.6.

$$\Pr \left[\begin{array}{l} \left(\text{Com.Verify}^{\mathcal{G}'}(\text{crs}, \mathbf{c}_{i^*}, M, \alpha, \text{pf}) = 1 \right) \\ \wedge Mx_{i^*} \neq \alpha \\ \vee \\ \left(\begin{array}{l} \exists i_1, i_2 \in [n] : \\ \mathbf{c}_{i_1} = \mathbf{c}_{i_2} \wedge x_{i_1} \neq x_{i_2} \end{array} \right) \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \text{Com.Setup}^{\mathcal{G}'}(1^\lambda, \ell, q) \\ \text{aux}_0 := \perp \\ \forall i \in [n] : \left(\begin{array}{l} (\mathbf{c}_i, \text{aux}_i) \leftarrow \mathcal{A}_i^{\mathcal{G}'}(\text{crs}, \text{aux}_{i-1}) \\ x_i \leftarrow \text{Ext}^{\mathcal{G}'}(\mathcal{A}_1, \dots, \mathcal{A}_i, \text{crs}) \end{array} \right) \\ (i^*, M, \alpha, \text{pf}) \leftarrow \mathcal{A}'^{\mathcal{G}'}(\text{crs}) \end{array} \right] = 0.$$

Proof. We consider each part in the event separately and show that they both cannot hold when interacting with $\mathcal{G}'$:

- “ $\text{Com.Verify}^{\mathcal{G}'}(\text{crs}, \mathbf{c}_{i^*}, M, \alpha, \text{pf}) = 1 \wedge Mx_{i^*} \neq \alpha$ ”: Consider $(i^*, M, \alpha, \text{pf})$ chosen by $\mathcal{A}'$ and the matching commitment $\mathbf{c}_{i^*}$. Let f be the polynomial matching the label $\mathbf{c}_{i^*}$ and h be the polynomial matching the label pf . Suppose that $\text{Com.Verify}^{\mathcal{G}'}(\text{crs}, \mathbf{c}_{i^*}, M, \alpha, \text{pf}) = 1$. This implies that the following two polynomials F_1, F_2 get the same label:

$$F_1(X, Z_1, \dots, Z_q) := f(X, Z_1, \dots, Z_q) \cdot \sum_{i \in [q]} \sum_{j \in [\ell]} M_{i,j} \cdot Z_i X^{\ell+1-j}$$

$$F_2(X, Z_1, \dots, Z_q) := \sum_{i \in [q]} \alpha_i \cdot Z_i X^{\ell+1} + h(X, Z_1, \dots, Z_q).$$

By construction, the only way that F_1 and F_2 can share a label is if $F_1 \equiv F_2$, and so we assume this to be the case.

Observe that by construction, any polynomial in Λ_1 (which includes $\mathbf{c}_{i^*}$) is of degree ℓ in X and monomials of the form $Z_i X^j$ for $i \in [q]$ and $j \in [2\ell] \setminus \{\ell+1\}$, and any polynomial in Λ_2 has monomials only of the form $Z_i X^j$ for $i \in [q]$ and $j \in [2\ell] \setminus \{\ell+1\}$. As a result, f and h have the following forms:

$$f(X) = \gamma_0 + \sum_{j \in [\ell]} \gamma_j X^j + \sum_{i \in [q]} \sum_{j \in [2\ell] \setminus \{\ell+1\}} \gamma_{i,j} Z_i X^j$$

$$h(X, Z_1, \dots, Z_q) = \sigma + \sum_{i \in [q]} \sum_{j \in [2\ell] \setminus \{\ell+1\}} \sigma_{i,j} Z_i X^j,$$

for some integers $\gamma_j, \gamma_{i,j}, \sigma$ and, $\sigma_{i,j}$.

Thus, for every $i \in [q]$, the coefficient of $Z_i X^{\ell+1}$ is $\sum_{j \in \ell} M_{i,j} \gamma_j$ for F_1 and α_i for F_2 . Since $F_1 \equiv F_2$, we get that $\sum_{j \in \ell} M_{i,j} \gamma_j = \alpha_i \pmod{p}$, or, in other words, the vector $x_{i^*} = (\gamma'_1, \dots, \gamma'_\ell)$ where $\gamma'_i = \gamma_i \pmod{p}$ has $Mx_{i^*} = \alpha$. Finally, we observe that Ext and $\mathcal{G}'$ both have exactly the same definitions and view of $\Lambda_1, \Lambda_2, \Lambda_T$, and so Ext will derive the same vector x_{i^*} .

- “ $\exists i_1, i_2 \in [n] : \mathbf{c}_{i_1} = \mathbf{c}_{i_2} \wedge x_{i_1} \neq x_{i_2}$ ”: by construction, there is a single polynomial f for every $\mathbf{c}$ such that $(\mathbf{c}, f) \in \Lambda_1$ (in both the extractor and $\mathcal{G}'$). Thus, for any i_1, i_2 with $\mathbf{c}_{i_1} = \mathbf{c}_{i_2}$, the extractor will choose the same polynomial f from which to derive x_{i_1} and x_{i_2} . Thus, $x_{i_1} = x_{i_2}$.

□

□

5 SNARGs with two group elements

In this section we show that there exist SNARGs for NP in the GGM + ROM with only two group elements by combining linear PCPs with a linear map commitment scheme. We present two different SNARGs based on two different PCPs, where the main difference between the SNARGs is the soundness error.

Our first SNARG has error roughly $O(s/n^{\text{polylog } n} + \text{poly}(n, s)/2^\lambda)$, and so is negligible in λ when $\lambda \leq \text{polylog } n$:

Theorem 5.1. *Let $c > 0$ be a constant. The NP-complete problem GapMWSP_m for $m = \log^{c+2} n$ has a SNARK in the bilinear GGM + ROM with group size 2^λ and ROM output length λ with the following parameters for inputs $(A, b, d) \in \mathbb{F}^{n \times \ell} \times \mathbb{F}^\ell \times \mathbb{N}$:*

- *Argument size:* $2 \mathbb{G}_1$.
- *Completeness error:* $\frac{65n^5 \lambda^2}{2^\lambda \log \log(n)^2}$.
- *Soundness/knowledge errors:* $\frac{s+1}{n^{\log^c n}} + \frac{5ns^2}{2^\lambda}$ with extraction time $O(n+s)$ against s -size adversaries.
- *Reference string length:* $3n - 1 \mathbb{G}_1 + 2n - 1 \mathbb{G}_2$.
- *Running times (in group operations):* proving $O(\ell n + n^2 + \lambda \log \log(n)^2)$, verification $O(\ell n)$ (3 pairings, 1 ROM query).

The above error bounds hold for $n \geq 256$ and $s > 60n^2$.

The following theorem improves upon the soundness error of Theorem 5.1 but requires the additional assumption of the unique games conjecture:

Theorem 5.2. *Assume the unique games conjecture. There is a SNARK for NP in the bilinear GGM + ROM with group size 2^λ and ROM output length λ and the following parameters for inputs of size n :*

- *Argument size:* $2 \mathbb{G}_1$.
- *Completeness error:* $\frac{\lambda^4 \text{poly}(n)}{2^\lambda}$.
- *Soundness error:* $\frac{s^2 \text{poly}(n)}{2^\lambda}$.
- *Reference string length:* $\text{poly}(n) \mathbb{G}_1$ and $\mathbb{G}_2$ elements.
- *Running times (in group operations):* proving $\text{poly}(n, \lambda)$, verification $\text{poly}(n)$ (3 pairings, 1 ROM query).

The above error bounds hold for $n \geq 256$, $s > 60n^2$, and $s > \lambda + 38$.

This section is organized as follows: (1) in Section 5.1 we define linear PCPs and describe the PCPs used in this paper, as well as a generic way to transform PCPs with a large accepting set into one with a small accepting set, (2) in Section 5.2 we show how to combine a linear PCP with a linear map commitment to construct a SNARK, and (3) in Section 5.3 we put these tools together to prove Theorems 5.1 and 5.2.

5.1 Linear PCPs

In this section we define linear PCPs. We define them in a slightly non-standard way as we will specifically refer to the accepting set of the PCP verifier.

Definition 5.3. A (non-adaptive) linear PCP over a finite field $\mathbb{F}$ for a relation $\mathcal{R}$ is a tuple of algorithms $(\mathbf{P}, \mathbf{V})$ where the prover outputs a string $\pi \in \mathbb{F}^\ell$, and the verifier outputs a matrix $M \in \mathbb{F}^{q \times \ell}$ and a set $S \subseteq \mathbb{F}^q$ (for which membership is efficiently decidable) such that the following properties hold:

- **Completeness.** The PCP has completeness error σ if for every $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$,

$$\Pr \left[(M\pi) \in S \mid \begin{array}{l} \pi = \mathbf{P}(\mathbf{x}, \mathbf{w}) \\ \rho \leftarrow \{0, 1\}^r \\ (M, S) \leftarrow \mathbf{V}(\mathbf{x}, \rho) \end{array} \right] \geq 1 - \sigma.$$

- **Soundness.** The PCP has soundness error β if for every $\mathbf{x} \notin L(\mathcal{R})$ and $\pi \in \mathbb{F}^\ell$:

$$\Pr \left[(M\pi) \in S \mid \begin{array}{l} \rho \leftarrow \{0, 1\}^r \\ (M, S) \leftarrow \mathbf{V}(\mathbf{x}, \rho) \end{array} \right] \leq \beta.$$

- **Knowledge soundness.** The PCP has knowledge soundness error κ with extractor running time $\mathbf{et}$ if there exists a $\mathbf{et}$ -time extractor $\mathbf{Ext}$ such that for every instance $\mathbf{x}$ and proof π , if

$$\Pr \left[(M\pi) \in S \mid \begin{array}{l} \rho \leftarrow \{0, 1\}^r \\ (M, S) \leftarrow \mathbf{V}(\mathbf{x}, \rho) \end{array} \right] \geq \kappa,$$

then $(\mathbf{x}, \mathbf{Ext}(\mathbf{x}, \pi)) \in \mathcal{R}$.

The proving time $\mathbf{pt}$ is the time to run $\pi = \mathbf{P}(\mathbf{x}, \mathbf{w})$, and the verification time $\mathbf{vt}$ is the combined time to generate M, S and to test membership in S . We say that the accepting set size of the PCP is s if for every $\mathbf{x}$ and ρ : $|S| \leq s$. In this paper we consider PCPs with an injective map $\varphi_S: S \rightarrow [|S|]$. We let $\mathbf{t}_\varphi$ be the time to compute φ_S and φ_S^{-1} .

The following claim shows that any PCP can be transformed into one with small accepting set size at a high cost to the completeness error:

Claim 5.4. Suppose $\mathcal{R}$ has a linear PCP with completeness error σ , randomness complexity $\mathbf{r}$, and accepting set size s . Then $\mathcal{R}$ has a linear PCP with completeness error $\sigma + \frac{s-1}{s}$, randomness complexity $\mathbf{r} + \log s$, and accepting set size 1. The soundness error, knowledge soundness error, and length of the PCP are preserved, and the running times are preserved up to an additive factor of $O(\log s)$.

Proof. Let $(\mathbf{P}, \mathbf{V})$ be the PCP for $\mathcal{R}$. We construct a new PCP verifier $\mathbf{V}'$ that interacts with the same honest prover $\mathbf{P}$. The new verifier $\mathbf{V}'$, given $(\mathbf{x}, (\rho, z))$ where $z \in [s]$ computes $(M, S) \leftarrow \mathbf{V}(\mathbf{x}, \rho)$, and outputs M and accepting set $S' = \{\varphi_S^{-1}(z)\}$.

If $\mathbf{x}$ is in the relation, then the verifier rejects either when it rejects in the original PCP (with probability at most σ) or if it chooses an incorrect answer from S , which happens with probability $\frac{s-1}{s}$. Thus, the completeness error is $\sigma + \frac{s-1}{s}$. Soundness does not degrade as M comes from the exact same distribution in $\mathbf{V}$ and $\mathbf{V}'$ and since $S' \subseteq S$. $\square$

5.1.1 PCP and linear PCP constructions

We describe the PCPs used to derive Theorems 5.1 and 5.2. We begin by defining an NP-complete language:

Definition 5.5. Fix a finite field $\mathbb{F}$. For an approximation factor m , and instance of GapMWSP_m over $\mathbb{F}$ is a triple (A, b, d) where $A \in \mathbb{F}^{n \times \ell}$, $b \in \mathbb{F}^\ell$, and $d \in \mathbb{N}$ such that:

- (A, b, d) is a YES instance if there exists $x \in \mathbb{F}^n$ such that $Ax = b$ and x has Hamming weight at most d .
- (A, b, d) is a NO instance if every solution $x \in \mathbb{F}^n$ where $Ax = b$ has Hamming weight at least $m \cdot d$.

It is known that for certain values of m , the problem GapMWSP_m is NP-hard:

Lemma 5.6 ([BIOW20], Lemma 4.2). For any constant $c > 0$, $m = \log^c n$ and finite field $\mathbb{F}$ where $\log |\mathbb{F}| = \text{poly}(n)$ there exists a Karp–Levin reduction¹³ from SAT to GapMWSP_m , where the reduction algorithm takes a target field $\mathbb{F}$ as an explicit input and outputs an instance (A, b, d) over $\mathbb{F}$ in time $\text{poly}(n, \log |\mathbb{F}|)$.

[BIOW20] further construct a PCP for GapMWSP_m with accepting size 1:

Theorem 5.7 ([BIOW20], Corollary 4.6 reformulated). Let $\sigma > 0$ be a completeness parameter and $m > 0$ be an approximation parameter. There is a 1-query linear PCP for the GapMWSP_m problem with accepting set size $s = 1$,¹⁴ completeness error σ and soundness error $1/|\mathbb{F}| + e^{-\sigma m}$. The prover time is $O(n)$ and the verifier time is $O(n\ell)$.

In order to get better parameters, we also use a PCP that is proven under the unique games conjecture. A *unique game* is a tuple $(V, E, \Sigma, f_{(x,y) \in E})$ where $V \subseteq \Sigma$, $E \subseteq V \times V$, and for every $(x, y) \in E$, $f_{x,y}: \Sigma \rightarrow \Sigma$ is a permutation. The value of assignment $A: V \rightarrow \Sigma$ is the fraction of pairs $(x, y) \in E$ where $A(y) = f_{x,y}(A(x))$, and the value of the unique game is the value of the optimum assignment. The unique games conjecture is as follows:

Conjecture 5.8. For every $\gamma > 0$ there is $\sigma = \sigma(\gamma)$ such that it is NP-hard to distinguish unique games of value $\geq 1 - \gamma$ from unique games of value γ , even when restricted to instances where $|\Sigma| \leq \sigma$ and where the constraint graph is bipartite.

We can now state the PCP in question:

Theorem 5.9 ([ST06], Theorem 29). Assuming the unique games conjecture, for every $k \geq 2$ there exists a PCP for NP with completeness error 2^{-2^k} , soundness error $(2^k + 2)/2^{2^k}$, proof length $\text{poly}(n)$, query complexity $2^k - 1$, and accepting set size 2^k .

Proof. Theorem 29 of [ST06] states that if the unique games conjecture holds then for every $\delta > 0$ and $k \geq 2$ we have there exists a PCP for NP with completeness error δ , soundness error $(2^k + 1)/2^{2^k} + \delta$, proof length $\text{poly}(n)$, and query complexity $2^k - 1$. We use $\delta = 2^{-2^k}$ and observe that the accepting set is of size 2^k .

¹³Here, by Karp–Levin reduction from $\mathcal{R}$ to $\mathcal{R}'$, we mean that there exist a pair of efficiently computable functions f, g so that if $(x, w) \in \mathcal{R}$ then $(f(x), g(x, w)) \in \mathcal{R}'$, and if $x \notin L(\mathcal{R})$ then $f(x) \notin L(\mathcal{R}')$.

¹⁴[BIOW20] do not explicitly say that their PCP has accepting set size 1, but this follows by their construction.

All queries their verifier does are in the context of a δ -noisy hypergraph test (see Definition 26 of [ST06]). This hypergraph test receives $2^k - 1$ functions $(g^{(s)} : \{0, 1\}^n \rightarrow \{0, 1\})_{s \subset [k], s \neq \emptyset}$. The verifier samples uniform k bitstrings $x_1, \dots, x_k$ and $2^k - 1$ sparse bitstrings $(\eta^{(s)})_{s \subset [k], s \neq \emptyset}$. The verifier queries $g^{(s)}(\eta^{(s)} + \sum_{i \in s} x_i)$. The verifier accepts if and only if for all $\emptyset \neq s \subset [k]$ we have

$$\sum_{i \in s} g^{(\{i\})}(x_i + \eta^{(\{i\})}) = g^{(s)}(\eta^{(s)} + \sum_{i \in s} x_i).$$

Notice, that on the right-hand side of the equation is just response to one term and the left-hand side of these equations are only ever terms $g^{(\{i\})}(x_i + \eta^{(\{i\})})$ for $i \in [k]$. Therefore, the response to these k uniquely identify one accepting response and the accepting set is size $\leq 2^k$. $\square$

By applying Claim 5.4 to Theorem 5.9 and setting $\lambda = 2^k$ we get the following PCP with negligible soundness error and accepting set size 1:

Corollary 5.10. *Assuming the unique games conjecture, for every $\lambda > 8$, there exists a PCP for NP with completeness error $1 - 1/2\lambda$, soundness error $(\lambda + 2)/2^\lambda$, proof length $\text{poly}(n)$, query complexity $\lambda - 1$, and accepting set size 1.*

5.2 SNARGs from PCPs with small accepting set

Given a linear PCP (ideally with small accepting set) and a linear map commitment, we construct a SNARG. This construction is similar to those in [LM19; CGKY25], except that we save in proof length by considering the accepting set size, and that we provide a technique for reducing completeness error at no runtime costs to the verifier.

Lemma 5.11. *Let $\mathbb{F}$ be a finite field and $t \in \mathbb{N}$ be a completeness error amplification parameter. Consider the following:*

- *A linear PCP for a relation $\mathcal{R}$ over $\mathbb{F}$ with completeness error σ , soundness error β , knowledge soundness error κ , accepting set size s , mapping time $\mathbf{t}_\varphi$, proving time $\mathbf{pt}$ and verifier time $\mathbf{vt}$, and extraction time $\mathbf{et}$.*
- *A linear map commitment over $\mathbb{F}$ with multi-extraction ε , committing time $\mathbf{t}_{\text{cc}}$, opening time $\mathbf{t}_{\text{co}}$, verification time $\mathbf{t}_{\text{cv}}$, and extraction time $\mathbf{t}_{\text{ext}}$.*

Then there is SNARK in the ROM for $\mathcal{R}$ with:

- *Argument size: $s_{\text{c}} + s_{\text{pf}} + \log s$*
- *Completeness error: $\sigma^t + t^2 \cdot \varepsilon(\lambda, \ell', 1, 1, O(\mathbf{pt} + \mathbf{t}_{\text{cc}} + \mathbf{t}_{\text{co}}))$*
- *Soundness error: $(s + 1) \cdot \beta + \varepsilon(\lambda, \ell', q, s + 1, s + \mathbf{vt} + \mathbf{t}_\varphi)$ against s -size adversaries*
- *Knowledge error: $(s + 1) \cdot \kappa + \varepsilon(\lambda, \ell', q, s + 1, s + \mathbf{vt} + \mathbf{t}_\varphi)$ against s -size adversaries with extraction time $O(\mathbf{pt} + \mathbf{et} + \mathbf{t}_{\text{ext}}(s))$*
- *Random oracle queries: 1*
- *Proving time $O(\mathbf{pt} + \mathbf{t}_\varphi + t \cdot (\mathbf{t}_{\text{cc}} + \mathbf{vt}) + \mathbf{t}_{\text{co}})$*
- *Verification time: $O(\mathbf{vt} + \mathbf{t}_\varphi + \mathbf{t}_{\text{cv}})$*

Above, $\ell' = \ell + \lceil \log_{|\mathbb{F}|} t \rceil$.

Remark 5.12. When using a standard PCP (i.e., a linear PCP where the verifier makes only point queries), the linear map commitment in the above theorem can be relaxed to a subvector commitment scheme.

Remark 5.13. If, as is the case in this paper, the linear map commitment is secure in the generic group model, and the multi-extraction error is a function of GGM queries rather than circuit size (see Remark 4.2), then the errors in Theorem 5.1 can be improved as follows:

- Completeness error: $\sigma^t + t^2 \cdot \varepsilon(\lambda, \ell', 1, 1, 2 \cdot \mathbf{t}_{cc} + \mathbf{t}_{co})$
- Soundness error: $(s + 1) \cdot \beta + \varepsilon(\lambda, \ell', q, s + 1, s + \mathbf{t}_{cv})$
- Knowledge error: $(s + 1) \cdot \kappa + \varepsilon(\lambda, \ell', q, s + 1, s + \mathbf{t}_{cv})$

This follows from the fact that all of the reductions of adversaries made by our proofs do not make any GGM queries unless they explicitly call one of the algorithms of the commitment scheme.

Proof. Let $\ell' = \ell + \lceil \log_{|\mathbb{F}|} t \rceil$. For simplicity, we assume that the linear PCP verifier receiver λ bits of randomness. The scheme can be easily adapted if this is not the case by the random oracle outputting the appropriate number of bits. We begin by describing the construction:

Construction 5.14. The SNARG $(\mathcal{S}, \mathcal{P}, \mathcal{V})$ is as follows:

- $\mathcal{S}(1^n, 1^\lambda)$: Output $\text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q)$.
- $\mathcal{P}^\mathcal{O}(\text{crs}, \mathbb{x}, \mathbb{w})$:
 1. $\pi \leftarrow \mathbf{P}(\mathbb{x}, \mathbb{w})$.
 2. For $i = 1$ to t :
 - (a) Set $\pi' = (\pi || i)$ (where we treat i as $\lceil \log_{|\mathbb{F}|} t \rceil$ field elements).
 - (b) $(\mathbf{c}, \text{aux}) \leftarrow \text{Com.Commit}(\text{crs}, \pi')$.
 - (c) $\rho \leftarrow \mathcal{O}(\mathbb{x}, \mathbf{c})$.
 - (d) $(M, S) \leftarrow \mathbf{V}(\mathbb{x}, \rho)$.
 - (e) If $(M\pi) \in S$ then:
 - i. Let $M' \in \mathbb{F}^{q \times \ell'}$ be defined as $M'[j, k] = \begin{cases} M[j, k] & j \in [\ell] \\ 0 & j > \ell \end{cases}$.
 - ii. $(\alpha, \text{pf}) \leftarrow \text{Com.Open}(\text{crs}, \mathbf{c}, \text{aux}, M')$.
 - iii. $z = \varphi_S(\alpha)$.
 - iv. Output $\Pi = (\mathbf{c}, z, \text{pf})$
 3. If this point has been reached, output “FAILED”.
- $\mathcal{V}^\mathcal{O}(\text{crs}, \mathbb{x}, \Pi = (\mathbf{c}, z, \text{pf}))$:
 1. Compute $\rho \leftarrow \mathcal{O}(\mathbb{x}, \mathbf{c})$.
 2. Let $(M, S) \leftarrow \mathbf{V}(\mathbb{x}, \rho)$, let $\alpha = \varphi_S^{-1}(z)$, and set $M' \in \mathbb{F}^{q \times \ell'}$ is defined as in the honest prover algorithm.
 3. Accept if $\alpha \in S$ and $\text{Com.Verify}(\text{crs}, \mathbf{c}, M', \alpha, \text{pf}) = 1$ and reject otherwise.

We first show completeness and then soundness and knowledge soundness.

Claim 5.15. *The SNARK has completeness error at most $\sigma^t + t^2 \cdot \varepsilon(\lambda, \ell', 1, 1, O(\mathbf{pt} + \mathbf{t}_{cc} + \mathbf{t}_{co}))$.*

Proof. Fix $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ and $\pi = \mathbf{P}(\mathbb{x}, \mathbb{w})$. For iteration i , we denote $(\mathbf{c}_i, \text{aux}_i) = \text{Com.Commit}(\text{crs}, (\pi || i))$, $\rho_i \leftarrow \mathcal{O}(\mathbb{x}, \mathbf{c}_i)$, and $(M_i, S_i) \leftarrow \mathbf{V}(\mathbb{x}, \rho_i)$. Observe that ρ_i and ρ_j are independently distributed uniformly random strings whenever $\mathbf{c}_i \neq \mathbf{c}_j$. Thus, by completeness of the linear PCP and correctness

of the linear map commitment, if for every $i \in [t]$, $\mathbf{c}_i$ is unique, the verifier rejects with probability at most σ^t . In other words:

$$\Pr[\text{Verifier Rejects}] \leq \Pr \left[\exists i, j \in [t], i \neq j \wedge \mathbf{c}_i = \mathbf{c}_j \mid \begin{array}{l} \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ \forall i \in [t], (\mathbf{c}_i, \text{aux}_i) = \text{Com.Commit}(\text{crs}, (\pi||i)) \end{array} \right] + \sigma^t$$

All that remains is to show that:

$$\Pr \left[\exists i, j \in [t], i \neq j \wedge \mathbf{c}_i = \mathbf{c}_j \mid \begin{array}{l} \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ \forall i \in [t], (\mathbf{c}_i, \text{aux}_i) = \text{Com.Commit}(\text{crs}, (\pi||i)) \end{array} \right] < \varepsilon.$$

We show that this holds by the multi-extraction property of the linear map commitment. Consider the following PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}')$ to function binding of the LMC. Let Ext be the extractor of the commitment scheme:

- $\mathcal{A}_1(\text{crs}, \text{aux} = \perp)$:
 1. Compute $\pi = \mathbf{P}(\mathbf{x}, \mathbf{w})$.
 2. Choose $(i, j) \leftarrow \binom{[t]}{2}$.
 3. For $a \in \{i, j\}$, let $\pi_a = (\pi||a)$ (where we treat a as $\lceil \log_{|\mathbb{F}|} t \rceil$ field elements) and compute $(\mathbf{c}_a, \text{aux}_a) = \text{Com.Commit}(\text{crs}, \pi_a)$.
 4. If $\mathbf{c}_i = \mathbf{c}_j$ then output $(\mathbf{c}_i, \text{aux}' = (i, j, \mathbf{c}_i, \text{aux}_i, \text{aux}_j))$. Otherwise output $\perp$.
- $\mathcal{A}'(\text{crs}, \text{aux}' = (i, j, \mathbf{c}_i, \text{aux}_i, \text{aux}_j))$:
 1. Choose $a \leftarrow \{i, j\}$ uniformly at random.
 2. Let $k \in [\ell']$ be such that $\pi_i[k] \neq \pi_j[k]$. One such k exists since the last $\lceil \log_{|\mathbb{F}|} t \rceil$ symbols in π_i and π_j encode i and j respectively, and $i \neq j$.
 3. Let $M \in \mathbb{F}^{1 \times \ell'}$ be the matrix that is equal to 0 everywhere, except the k -th column, which is equal to 1.
 4. Compute $\text{pf} = \text{Com.Open}(\text{crs}, \mathbf{c}_a, \text{aux}_a, M)$.
 5. Output $(M, \alpha, \text{pf}) = (M, \pi_a[k], \text{pf})$.

By function binding of the LMC,

$$\Pr \left[\begin{array}{l} \text{Com.Verify}(\text{crs}, \mathbf{c}, M, \alpha, \text{pf}) = 1, \\ (Mx) \neq \alpha \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ (\mathbf{c}, \text{aux}) \leftarrow \mathcal{A}_1(\text{crs}) \\ x \leftarrow \text{Ext}(\mathcal{A}_1, \text{crs}) \\ (M, \alpha, \text{pf}) \leftarrow \mathcal{A}'(\text{crs}, \text{aux}) \end{array} \right] \leq \varepsilon.$$

Observe that, by construction, whenever there is $\mathbf{c}_i = \mathbf{c}_j$, and we guess these indices correctly, it is always the case that: (1) by completeness of the linear map commitment, $\text{Com.Verify}(\text{crs}, \mathbf{c}, M, \alpha, \text{pf}) = 1$, and (2) with probability $1/\binom{t}{2}$ for at least one choice of a it holds that $(Mx) \neq M\pi_a$ for

$x \leftarrow \mathcal{A}'(\text{crs}, \text{aux})$. Therefore we conclude that:

$$\begin{aligned} & \Pr \left[\exists i, j \in [t], i \neq j \wedge \mathbf{c}_i = \mathbf{c}_j \mid \begin{array}{l} \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ \forall i \in [t], (\mathbf{c}_i, \text{aux}_i) = \text{Com.Commit}(\text{crs}, (\pi || i)) \end{array} \right] \\ & \leq \binom{t}{2} \cdot \Pr \left[\begin{array}{l} \text{Com.Verify}(\text{crs}, \mathbf{c}, M, \alpha, \text{pf}) = 1, \\ (Mx) \neq \alpha \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ (\mathbf{c}, \text{aux}) \leftarrow \mathcal{A}_1(\text{crs}) \\ x \leftarrow \text{Ext}(\mathcal{A}_1, \text{crs}) \\ (M, \alpha, \text{pf}) \leftarrow \mathcal{A}'(\text{crs}, \text{aux}) \end{array} \right] \\ & \leq t^2 \cdot \varepsilon(\lambda, \ell', 1, 1, O(\text{pt} + \text{t}_{\text{cc}} + \text{t}_{\text{co}})). \end{aligned}$$

□

Claim 5.16. *The SNARK has soundness error at most $(s+1) \cdot \beta + \varepsilon(\lambda, \ell', q, s+1, s + \text{vt} + \text{t}_\varphi)$ and knowledge error at most $(s+1) \cdot \kappa + \varepsilon(\lambda, \ell', q, s+1, s + \text{vt} + \text{t}_\varphi)$ against s -size adversaries. The extraction time is $O(\text{pt} + \text{et} + \text{t}_{\text{ext}}(s))$.*

Proof. We argue knowledge soundness. Soundness follows by an identical argument while removing the need for extraction. For an m -query prover $\mathbf{P}$ (so that $m \leq s$), let q_i be its i -th to the random oracle query and aux_i be its state immediately prior to making the i -th query. We use the notation $\mathbf{P} = (\mathbf{P}_1, \dots, \mathbf{P}_{m+1}, \mathbf{P}')$ as follows:

- For $i \in [m]$: $\mathbf{P}_i$ is the algorithm that, on input crs , aux_{i-1} , and a query answer ρ_{i-1} uses this information (by continuing to run $\mathbf{P}$) to output (q_i, aux_i) .
- $\mathbf{P}_{m+1}$ is the algorithm that, on input $\text{crs}, \text{aux}_m, \rho_m$ runs keeps running $\mathbf{P}$ until it outputs $(\mathbf{x}, \Pi = (\mathbf{c}, z, \text{pf}))$ and outputs $\mathbf{c}_{m+1} = \mathbf{c}$ and auxiliary state $\text{aux}_{m+1} = (\mathbf{x}, z, \text{pf})$.
- $\mathbf{P}'$ receives crs and $\text{aux}_{m+1} = (\mathbf{x}, z, \text{pf})$, computes $(M, S) \leftarrow \mathbf{V}(\mathbf{x}, \mathcal{O}(\mathbf{x}, \mathbf{c}))$ and outputs $(i, M, \varphi_S^{-1}(z), \text{pf})$

We can now present the SNARK extractor $\mathcal{E}$:

Construction 5.17. Let Ext be the extractor of the linear map commitment. Then $\mathcal{E}^\mathcal{O}$ on inputs $\mathbf{P} = (\mathbf{P}_1, \dots, \mathbf{P}_{m+1}, \mathbf{P}')$ and crs and runs as follows:

1. Run $(\mathbf{x}, \Pi = (\mathbf{c}, z, \text{pf})) \leftarrow \mathbf{P}^\mathcal{O}(\text{crs})$ and let i^* be the query index where $\mathbf{P}^\mathcal{O}$ queries $(\mathbf{x}, \mathbf{c})$, and $i^* = m+1$ if the query was not made.
2. Compute $\pi' \leftarrow \text{Ext}(\mathbf{P}_1^\mathcal{O}, \dots, \mathbf{P}_{i^*}^\mathcal{O}, \text{crs}) \in \mathbb{F}^{\ell'}$. Let π be the first ℓ symbols of π' .¹⁵
3. Derive $\mathbf{w} \leftarrow \mathbf{E}(\mathbf{x}, \pi)$.
4. Output $\mathbf{w}$.

¹⁵Formally, since Ext does not expect to receive an oracle circuit, by $\mathbf{P}_j^\mathcal{O}$ we mean the circuit that has the random oracle answers hard-coded. Observe that $\mathcal{E}$ can construct this circuit as it can collect all of the random oracle queries when it runs $\mathbf{P}^\mathcal{O}$. Furthermore, $\mathbf{P}_j^\mathcal{O}$ outputs a query q_j , which, in the case that we care about is equal to $(\mathbf{x}_j, \mathbf{c}_j)$ and not just a commitment as Ext expects. For simplicity of notation we ignore this discrepancy as the correct mapping is clear from context.

Using our notation and the above extractor:

$$\begin{aligned}
& \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n, \\ (\mathbb{x}, \mathbb{w}) \notin \mathcal{R}, \\ \mathcal{V}'(\mathbb{x}, \mathbf{c}, \rho, (z, \mathbf{pf})) = 1 \end{array} \middle| \begin{array}{l} \mathcal{O} \leftarrow \text{RO}(1^\lambda) \\ \text{crs} \leftarrow \mathcal{S}^\mathcal{O}(1^n, 1^\lambda) \\ (\mathbb{x}, \Pi) \leftarrow \mathbf{P}^\mathcal{O}(\text{crs}) \\ \mathbb{w} \leftarrow \mathcal{E}^\mathcal{O}(\mathbf{P}, \text{crs}) \end{array} \right] \\
&= \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n, \\ (\mathbb{x}, \mathbb{w}) \notin \mathcal{R}, \\ \text{Com.Verify}(\text{crs}, \mathbf{c}, M', \alpha, \mathbf{pf}) = 1, \\ \alpha \in S \end{array} \middle| \begin{array}{l} \mathcal{O} \leftarrow \text{RO}(1^\lambda) \\ \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ \text{aux}_0 := \perp \\ \rho_0 := \perp \\ \forall i \in [m+1] : \left(\begin{array}{l} (q_i, \text{aux}_i) \leftarrow \mathbf{P}_i(\text{crs}, \text{aux}_{i-1}) \\ \rho_i = \mathcal{O}(q_i) \end{array} \right) \\ (\alpha, \mathbf{pf}) \leftarrow \mathbf{P}'^\mathcal{O}(\text{crs}, \text{aux}_{m+1}) \\ (\mathbf{c}, \mathbb{x}) = q_{i^*} \\ (M, S) \leftarrow \mathbf{V}(\mathbb{x}, \rho_{i^*}) \\ \mathbb{w} \leftarrow \mathcal{E}(\mathbf{P}, \text{crs}) \end{array} \right]
\end{aligned}$$

We add to the experiment an extractor for every query that the prover makes. So far we have only added it to the experiment, so the probability is equal:

$$\begin{aligned}
&= \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n, \\ (\mathbb{x}, \mathbb{w}) \notin \mathcal{R}, \\ \text{Com.Verify}(\text{crs}, \mathbf{c}, M', \alpha, \mathbf{pf}) = 1, \\ \alpha \in S \end{array} \middle| \begin{array}{l} \mathcal{O} \leftarrow \text{RO}(1^\lambda) \\ \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ \text{aux}_0 := \perp \\ \rho_0 := \perp \\ \forall i \in [m+1] : \left(\begin{array}{l} (q_i, \text{aux}_i) \leftarrow \mathbf{P}_i(\text{crs}, \text{aux}_{i-1}) \\ \pi'_i \leftarrow \text{Ext}(\mathbf{P}_1^\mathcal{O}, \dots, \mathbf{P}_i^\mathcal{O}, \text{crs}) \\ \rho_i = \mathcal{O}(q_i) \end{array} \right) \\ (\alpha, \mathbf{pf}) \leftarrow \mathbf{P}'^\mathcal{O}(\text{crs}, \text{aux}_{m+1}) \\ (\mathbf{c}, \mathbb{x}) = q_{i^*} \\ (M, S) \leftarrow \mathbf{V}(\mathbb{x}, \rho_{i^*}) \\ \mathbb{w} \leftarrow \mathcal{E}(\mathbf{P}, \text{crs}) \end{array} \right]
\end{aligned}$$

Opening up the extractor $\mathcal{E}$ and noting that it computes π_{i^*} exactly as the way already computed in the experiment during the main loop, the above is equal to:

$$\begin{aligned}
&= \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n, \\ (\mathbb{x}, \mathbb{w}) \notin \mathcal{R}, \\ \text{Com.Verify}(\text{crs}, \mathbf{c}, M', \alpha, \mathbf{pf}) = 1, \\ \alpha \in S \end{array} \middle| \begin{array}{l} \mathcal{O} \leftarrow \text{RO}(1^\lambda) \\ \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ \text{aux}_0 := \perp \\ \rho_0 := \perp \\ \forall i \in [m+1] : \left(\begin{array}{l} (q_i, \text{aux}_i) \leftarrow \mathbf{P}_i(\text{crs}, \text{aux}_{i-1}) \\ \pi'_i \leftarrow \text{Ext}(\mathbf{P}_1^\mathcal{O}, \dots, \mathbf{P}_i^\mathcal{O}, \text{crs}) \\ \rho_i = \mathcal{O}(q_i) \end{array} \right) \\ (\alpha, \mathbf{pf}) \leftarrow \mathbf{P}'^\mathcal{O}(\text{crs}, \text{aux}_{m+1}) \\ (\mathbf{c}, \mathbb{x}) = q_{i^*} \\ (M, S) \leftarrow \mathbf{V}(\mathbb{x}, \rho_{i^*}) \\ \mathbb{w} \leftarrow \mathbf{E}(\mathbb{x}, \pi_{i^*}) \end{array} \right]
\end{aligned}$$

By multi-extraction of the linear map commitment scheme, α in the above check that $\alpha \in S$ can be replaced with $M\pi_{i^*}$ up to an added error of ε so that the above is bounded by:

$$\leq \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n, \\ (\mathbb{x}, \mathbb{w}) \notin \mathcal{R}, \\ \text{Com.Verify}(\text{crs}, \mathbf{c}, M', \alpha, \text{pf}) = 1, \\ \textcolor{red}{(M\pi_{i^*}) \in S} \end{array} \left| \begin{array}{l} \mathcal{O} \leftarrow \text{RO}(1^\lambda) \\ \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ \text{aux}_0 := \perp \\ \rho_0 := \perp \\ \forall i \in [m+1] : \begin{pmatrix} (q_i, \text{aux}_i) \leftarrow \mathbf{P}_i(\text{crs}, \text{aux}_{i-1}) \\ \pi'_i \leftarrow \text{Ext}(\mathbf{P}_1^\mathcal{O}, \dots, \mathbf{P}_i^\mathcal{O}, \text{crs}) \\ \rho_i = \mathcal{O}(q_i) \end{pmatrix} \\ (\alpha, \text{pf}) \leftarrow \mathbf{P}'^\mathcal{O}(\text{crs}, \text{aux}_{m+1}) \\ (\mathbf{c}, \mathbb{x}) = q_{i^*} \\ (M, S) \leftarrow \mathbf{V}(\mathbb{x}, \rho_{i^*}) \\ \mathbb{w} \leftarrow \mathbf{E}(\mathbb{x}, \pi_{i^*}) \end{array} \right. \right] \\ + \varepsilon(\lambda, \ell', q, m+1, s + \text{vt} + \text{t}_\varphi)$$

We can now remove the verification of the commitment scheme. This, in turn, allows us to remove $\mathbf{P}'$ from the experiment as its outputs are not used anymore:

$$\leq \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n, \\ (\mathbb{x}, \mathbb{w}) \notin \mathcal{R}, \\ (M\pi_{i^*}) \in S \end{array} \left| \begin{array}{l} \mathcal{O} \leftarrow \text{RO}(1^\lambda) \\ \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ \text{aux}_0 := \perp \\ \rho_0 := \perp \\ \forall i \in [m+1] : \begin{pmatrix} (q_i, \text{aux}_i) \leftarrow \mathbf{P}_i(\text{crs}, \text{aux}_{i-1}) \\ \pi'_i \leftarrow \text{Ext}(\mathbf{P}_1^\mathcal{O}, \dots, \mathbf{P}_i^\mathcal{O}, \text{crs}) \\ \rho_i = \mathcal{O}(q_i) \end{pmatrix} \\ (\mathbf{c}, \mathbb{x}) = q_{i^*} \\ (M, S) \leftarrow \mathbf{V}(\mathbb{x}, \rho_{i^*}) \\ \mathbb{w} \leftarrow \mathbf{E}(\mathbb{x}, \pi_{i^*}) \end{array} \right. \right] + \varepsilon(\lambda, \ell', q, m+1, s + \text{vt} + \text{t}_\varphi)$$

Observe now that we only need to run $\mathbf{P}$ up to the i^* -th query (recall that i^* is not efficiently computable, and so we are not done yet):

$$= \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n, \\ (\mathbb{x}, \mathbb{w}) \notin \mathcal{R}, \\ (M\pi_{i^*}) \in S \end{array} \left| \begin{array}{l} \mathcal{O} \leftarrow \text{RO}(1^\lambda) \\ \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ \text{aux}_0 := \perp \\ \rho_0 := \perp \\ \forall i \in [i^* - 1] : \begin{pmatrix} (q_i, \text{aux}_i) \leftarrow \mathbf{P}_i(\text{crs}, \text{aux}_{i-1}) \\ \pi'_i \leftarrow \text{Ext}(\mathbf{P}_1^\mathcal{O}, \dots, \mathbf{P}_i^\mathcal{O}, \text{crs}) \\ \rho_i = \mathcal{O}(q_i) \end{pmatrix} \\ (q_{i^*}, \text{aux}_{i^*}) \leftarrow \mathbf{P}_{i^*}(\text{crs}, \text{aux}_{i^*-1}) \\ \pi'_{i^*} \leftarrow \text{Ext}(\mathbf{P}_1^\mathcal{O}, \dots, \mathbf{P}_{i^*}^\mathcal{O}, \text{crs}) \\ (\mathbf{c}, \mathbb{x}) = q_{i^*} \\ \rho_{i^*} = \mathcal{O}(q_{i^*}) \\ (M, S) \leftarrow \mathbf{V}(\mathbb{x}, \rho_{i^*}) \\ \mathbb{w} \leftarrow \mathbf{E}(\mathbb{x}, \pi_{i^*}) \end{array} \right. \right] + \varepsilon(\lambda, \ell', q, m+1, s + \text{vt} + \text{t}_\varphi)$$

We now sample $j \leftarrow [m+1]$ instead of using i^* , getting $j = i^*$ with probability $1/(m+1)$. Thus, the above is bounded by:

$$\leq (m+1) \cdot \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n, \\ (\mathbb{x}, \mathbb{w}) \notin \mathcal{R}, \\ (M\pi_i) \in S \end{array} \left| \begin{array}{l} \mathcal{O} \leftarrow \text{RO}(1^\lambda) \\ j \leftarrow [m+1] \\ \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ \text{aux}_0 := \perp \\ \rho_0 := \perp \\ \forall i \in [j-1] : \left(\begin{array}{l} (q_i, \text{aux}_i) \leftarrow \mathbf{P}_i(\text{crs}, \text{aux}_{i-1}) \\ \pi'_i \leftarrow \text{Ext}(\mathbf{P}_1^\mathcal{O}, \dots, \mathbf{P}_i^\mathcal{O}, \text{crs}) \\ \rho_i = \mathcal{O}(q_i) \end{array} \right) \\ (q_j, \text{aux}_j) \leftarrow \mathbf{P}_j(\text{crs}, \text{aux}_{j-1}) \\ \pi'_j \leftarrow \text{Ext}(\mathbf{P}_1^\mathcal{O}, \dots, \mathbf{P}_j^\mathcal{O}, \text{crs}) \\ (\mathbf{c}, \mathbb{x}) = q_j \\ \rho_j = \mathcal{O}(q_j) \\ (M, S) \leftarrow \mathbf{V}(\mathbb{x}, \rho_j) \\ \mathbb{w} \leftarrow \mathbf{E}(\mathbb{x}, \pi_j) \end{array} \right. \right] + \varepsilon(\lambda, \ell', q, m+1, s + \mathbf{vt} + \mathbf{t}_\varphi)$$

We now notice that, as $\mathcal{O}(q_j)$ is queried only once (since $\mathbf{P}$ does not make the same query twice), we can sample it uniformly at random rather than from the random oracle:

$$\leq (m+1) \cdot \Pr \left[\begin{array}{l} |\mathbb{x}| \leq n, \\ (\mathbb{x}, \mathbb{w}) \notin \mathcal{R}, \\ (M\pi_i) \in S \end{array} \left| \begin{array}{l} \mathcal{O} \leftarrow \text{RO}(1^\lambda) \\ j \leftarrow [m+1] \\ \text{crs} \leftarrow \text{Com.Setup}(1^\lambda, \ell', q) \\ \text{aux}_0 := \perp \\ \rho_0 := \perp \\ \forall i \in [j-1] : \left(\begin{array}{l} (q_i, \text{aux}_i) \leftarrow \mathbf{P}_i(\text{crs}, \text{aux}_{i-1}) \\ \pi'_i \leftarrow \text{Ext}(\mathbf{P}_1^\mathcal{O}, \dots, \mathbf{P}_i^\mathcal{O}, \text{crs}) \\ \rho_i = \mathcal{O}(q_i) \end{array} \right) \\ (q_j, \text{aux}_j) \leftarrow \mathbf{P}_j(\text{crs}, \text{aux}_{j-1}) \\ \pi'_j \leftarrow \text{Ext}(\mathbf{P}_1^\mathcal{O}, \dots, \mathbf{P}_j^\mathcal{O}, \text{crs}) \\ (\mathbf{c}, \mathbb{x}) = q_j \\ \rho_j \leftarrow \{0, 1\}^\lambda \\ (M, S) \leftarrow \mathbf{V}(\mathbb{x}, \rho_j) \\ \mathbb{w} \leftarrow \mathbf{E}(\mathbb{x}, \pi_j) \end{array} \right. \right] + \varepsilon(\lambda, \ell', q, m+1, s + \mathbf{vt} + \mathbf{t}_\varphi)$$

Finally, by constructing a prover that runs the algorithms in in the experiment above up to the sampling of ρ_j (while emulating $\mathcal{O}$ with uniformly random answers) and outputting $\mathbb{x}$ and π'_i , and by knowledge soundness of the PCP, the above is bounded by $(m+1) \cdot \kappa + \varepsilon(\lambda, \ell', q, m+1, s + \mathbf{vt} + \mathbf{t}_\varphi)$ which is, in turn, bounded by $(s+1) \cdot \kappa + \varepsilon(\lambda, \ell', q, s+1, s + \mathbf{vt} + \mathbf{t}_\varphi)$. $\square$

$\square$

5.3 Proof of Theorems 5.1 and 5.2

In this section, we prove Theorem 5.1 and Theorem 5.2 by putting together a linear PCP and the linear map commitment described in Theorem 4.3 and deriving appropriate bounds from Lemma 5.11.

Note that for both theorems, when analyzing the completeness error we will use the tighter multi-extraction bound of $\varepsilon(\lambda, \ell, q, n, s) = \frac{60\ell^3 q^2 s + 3\ell s^2}{2^\lambda}$ for the linear map commitment (see proof of Theorem 4.3).

- Theorem 5.1: we use the PCP for GapMWSP_m of Theorem 5.7. Letting $t = \lambda / \log \log n$ in Lemma 5.11, we have the following:

– Completeness error:

$$\begin{aligned} & \sigma^t + t^2 \cdot \varepsilon(\lambda, n+1, 1, 1, n^2 + 2(n+1)) \\ &= \left(\frac{1}{\log n} \right)^{\frac{\lambda}{\log \log n}} + \left(\frac{\lambda}{\log \log n} \right)^2 \cdot \left(\frac{60(n+1)^3((n+1)^2 + 2(n+1)) + 3(n+1)((n+1)^2 + 2(n+1))^2}{2^\lambda} \right) \\ &< \frac{65n^5 \lambda^2}{2^\lambda \log \log(n)^2}, \end{aligned}$$

where the final inequality holds since $n \geq 256$.

– Soundness/knowledge error:

$$\begin{aligned} & (s+1) \cdot \beta + \varepsilon(\lambda, n+1, 1, s+1, s+3(n+1)) \\ &= \frac{s+1}{e^{\log^{c+1} n}} + \frac{s+1}{2^\lambda} + \frac{4(n+1) \cdot (s+3(n+1))^2}{2^\lambda} \\ &= \frac{s+1}{e^{\log^{c+1} n}} + \frac{4ns^2 + 36n^3 + 24n^2s + 108n^2 + 48ns + 108n + 4s^2 + 25s + 37}{2^\lambda} \\ &\leq \frac{s+1}{n^{\log^c n}} + \frac{5ns^2}{2^\lambda}, \end{aligned}$$

where the final inequality follows from $s > 60n^2$ and $n \geq 256$.

- Theorem 5.2: we use the PCP of Corollary 5.10. Letting $t = 2\lambda^2$ in Lemma 5.11, we have the following error bounds:

– Completeness error:

$$\begin{aligned} & \sigma^t + t^2 \cdot \varepsilon(\lambda, n+1, 1, 1, n^2 + 2(n+1)) \\ &= \left(1 - \frac{1}{2^\lambda} \right)^{2\lambda^2} + 4\lambda^4 \cdot \left(\frac{60(n+1)^3((n+1)^2 + 2(n+1)) + 3(n+1)((n+1)^2 + 2(n+1))^2}{2^\lambda} \right) \\ &\leq \frac{1}{2^\lambda} + 4\lambda^4 \cdot \left(\frac{60(n+1)^3((n+1)^2 + 2(n+1)) + 3(n+1)((n+1)^2 + 2(n+1))^2}{2^\lambda} \right) \\ &< \frac{260n^5 \lambda^4}{2^\lambda}, \end{aligned}$$

where the final inequality holds since $n \geq 256$.

– Soundness error:

$$\begin{aligned}
& (s+1) \cdot \beta + \varepsilon(\lambda, n+1, 1, s+1, s+3(n+1)) \\
&= \frac{(s+1)(\lambda+2)}{2^\lambda} + \frac{4(n+1) \cdot (s+3(n+1))^2}{2^\lambda} \\
&= \frac{4ns^2 + 36n^3 + 24n^2s + 108n^2 + 48ns + 108n + 4s^2 + (26+\lambda)s + \lambda + 38}{2^\lambda} \\
&\leq \frac{5ns^2}{2^\lambda}
\end{aligned}$$

Where the final inequality follows from $s > 60n^2$, $s > \lambda + 38$ and $n \geq 256$.

6 Lower bound for SNARGs with one group element

In previous sections, we showed the existence of two group element SNARGs in the generic group model with random oracles. In this section, we established a lower bound matching our constructions. With some caveats, we prove that in Maurer’s generic group model [Mau05] for asymmetric pairing groups with a random oracle, there does not exist a SNARG.

These caveats are that the setup algorithm does not have access to the random oracle and that the verifier queries are non-adaptive¹⁶. All group-based publicly verifiable SNARGs we are aware of fall into the model covered by our lower bound.

Our lower-bound holds for the full class of GGM + ROM as defined above. However, for convenience we describe it for a random oracles. That is, it is described for the class RO(1) of functions of the form

$$\mathcal{O}: (\{0, 1\} \cup \mathbb{G}_1 \cup \mathbb{G}_2 \cup \mathbb{G}_T)^* \rightarrow \{0, 1\} \times \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T.$$

This is without loss of generality, as all parties can always append each query with $\log \lambda$ bits to generate λ different outputs. This affects the running times and query complexity of the parties by a polynomial factor (as λ is polynomially related to all other parameters). These factors do not come into play in the proof of our lower bound, and so it is safe to work with the assumption that the output length is 1.

Theorem 6.1. *If one-way functions exist (in the plain model) then there does not exist a SNARG for NP in the Maurer bilinear GGM $(\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T) + \text{ROM}$ with the following properties:*

- *The proof consists of a single group element (of either $\mathbb{G}_1, \mathbb{G}_2$, or $\mathbb{G}_T$);*
- *The completeness and soundness errors are negligible in λ ;*
- *The verifier makes only non-adaptive ROM queries of the form $(\{0, 1\} \cup \mathbb{G}_1 \cup \mathbb{G}_2 \cup \mathbb{G}_T)^* \rightarrow \{0, 1\} \times \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T$ where ROM queries are done prior to any GGM equality checks (or, equivalently, do not depend on them);*
- *The setup algorithm does not make random oracle queries.*

Proof. Follows from the upcoming Lemmas 6.4, 6.7, 6.9 and 6.11. □

Remark 6.2 (Comparisson to [Gro16]). Groth [Gro16], proved a limitation on the structure of SNARGs: in the asymmetric pairing setting there are no SNARGs where

1. The proof only consists of (possibly multiple) $\mathbb{G}_1$ group elements; and
2. The output of the verifier is a conjunction of predefined pairing equation checks.

This result is incomparable to Theorem 6.1 as it also rules out SNARGs with multiple $\mathbb{G}_1$ elements, while we rule out SNARGs with much more expressive verifiers. Indeed, a variety of modern SNARGs (e.g., [GWC19; Lip24; DMS24]) have verifiers that follow a structure not covered by the lower bound in [Gro16] because they utilise a random oracle. As shown in these works, by using a random oracle (and thus bypassing the structure required by Groth) it is possible where the proof consists only of $\mathbb{G}_1$ elements.

¹⁶With non-adaptive, we mean that the verifier circuit $\mathcal{V}(\text{crs}, \mathbb{x}, \Pi)$ can be split into two parts $\mathcal{V}_1$ and $\mathcal{V}_2$. $\mathcal{V}_1(\text{crs}, \mathbb{x}, \Pi)$ does not do any equality check GGM calls or ROM calls outputs a state **State** and queries $(q_i)_{i \in [q]}$. $\mathcal{V}_2$ receives **State** and ROM response to these queries $(\mathcal{O}(q_i))_{i \in [q]}$ and accepts or rejects. $\mathcal{V}_2$ is allowed equality check GGM calls but no further ROM calls.

Remark 6.3. The proof in this section focuses on the prover’s message Π being a $\mathbb{G}_1$ element. By symmetry the proof is identical if it is a $\mathbb{G}_2$ element. If Π is a $\mathbb{G}_T$ element then small changes are required due to the difference in ways that the prover can generate a $\mathbb{G}_T$ element compared to a $\mathbb{G}_1$ element, but the proof remains essentially the same.

Matching of upper bound. The SNARG construction in [SW14], together with our results characterize the tightness of our lower bound. We rule out SNARGs with ≤ 1 group elements plus $O(\log \lambda)$ bits¹⁷. Feasibility of a construction requires either relaxing the overhead to > 1 group elements (our constructions) or allowing $\omega(\log \lambda)$ extra bits (as in [SW14]).

Roadmap. To prove Theorem 6.1 we first prove in Section 6.1 that the random oracle does not help to build one group element SNARGs. We do this by compiling every one group element SNARG in Maurer’s model with non-adaptive random oracle calls to a one group element SNARG in Maurer’s model *without the random oracle*. Then, in Section 6.2, we prove that a one group element SNARG in Maurer’s generic group model does not exist if one-way functions exist (in the plain model).

Notation. In this section we use additive notation for the cryptographic group. For example $[\mathbf{u}]_1$ are a vector of group elements in $\mathbb{G}_1$ and $\mathbf{u}$ is the vector of the discrete logarithms of $[\mathbf{u}]_1$. Similarly, $[\mathbf{v}]_2$ and $[\mathbf{w}]_T$ are vectors of group elements in $\mathbb{G}_2$ and $\mathbb{G}_T$, respectively. If the specific group of an element with discrete logarithm u is not relevant we will denote it by $[u]_*$.

6.1 Removing the ROM from one group element SNARGs

In this section, we show how to remove the random oracle from any one group element SNARG with non-adaptive random oracle calls. We achieve this in two steps. In the first step, we remove all the verifier calls to the random oracle which do not depend on the SNARG proof Π . Then, in a second step, we remove all the other calls to the random oracle. We discuss what it means for a group element to depend on the proof later in this section.

6.1.1 Making the verifier deterministic

To remove random oracle queries, we will need it to be the case that the verifier is deterministic (except for the responses of the random oracle). Here we argue that this holds without loss of generality. We amplify the completeness and soundness over the randomness of the verifier and then we fix a randomness that preserves completeness and soundness using an averaging argument.

Lemma 6.4. *Let $(\mathcal{S}', \mathcal{P}', \mathcal{V}')$ be a SNARG for relation $\mathcal{R}$ with completeness error $\sigma(\lambda)$ where $\sigma \leq 1 - 1/\text{poly}(\lambda)$ and (non-adaptive) soundness error $\beta(\lambda)$. Then there exists a SNARG for $\mathcal{R}$ with a deterministic verifier with completeness error $\sigma + 2^{-\lambda}$ and (non-adaptive) soundness error $4\beta/(1 - \sigma) + 2^{-\lambda}$. The argument size is preserved and all other parameters are polynomially related.*

Remark 6.5. While described for SNARGs in the standard model, the proof of Lemma 6.4 relativizes with respect to oracles, and so it also holds for SNARG constructions relative to oracles.

¹⁷We can handle these extra bits by having the verifier check all $2^{O(\log \lambda)}$ possibilities, which only reduces soundness polynomially.

Proof. Let $(\mathcal{S}', \mathcal{P}', \mathcal{V}')$ be the SNARG with randomized verifier and denote by $|\text{crs}'|$ its common reference string size and $|\Pi|$ its argument string size. Let $t = \frac{12 \log(e)(\lambda + |\text{crs}'| + |\Pi|)}{1 - \sigma}$ be a repetition parameter, and observe that t is polynomial in λ . We construct the following SNARG $(\mathcal{S}, \mathcal{P}, \mathcal{V})$ with deterministic verifier:

- $\mathcal{S}(1^\lambda)$:
 1. Let $\text{crs}' \leftarrow \mathcal{S}'(1^\lambda)$.
 2. For $i \in [t]$ sample verifier randomness r_i uniformly at random.
 3. Output $\text{crs} := (\text{crs}', (r_i)_{i \in [t]})$
- $\mathcal{P}(\text{crs}, \mathbb{x}, \mathbb{w})$: Output $\Pi \leftarrow \mathcal{P}'(\text{crs}, \mathbb{x}, \mathbb{w})$.
- $\mathcal{V}(\text{crs}, \mathbb{x}, \Pi)$:
 1. Let $\text{counter} \leftarrow 0$.
 2. For $i \in [t]$: Let $\text{counter} \leftarrow \text{counter} + \mathcal{V}'(\text{crs}, \mathbb{x}, \Pi; r_i)$.
 3. If $\text{counter} \geq \frac{(1 - \sigma) \cdot t}{2}$ output 1 otherwise output 0.

Completeness. Fix an $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$. For a common reference string crs' , let

$$p_{\text{crs}'} = \Pr_{r_{\mathcal{V}'}}[\mathcal{V}'_{\text{crs}'}(\mathbb{x}, \mathcal{P}'_{\text{crs}'}(\mathbb{x}, \mathbb{w}); r_{\mathcal{V}'}) = 1].$$

Fix a common reference string crs' with $p_{\text{crs}'} \geq 1 - \sigma$. Letting X_i be the 0/1 random variable where

$$X_i = 1 \iff \mathcal{V}'_{\text{crs}'}(\mathbb{x}, \mathcal{P}'_{\text{crs}'}(\mathbb{x}, \mathbb{w}); r_i) = 1,$$

so that $X_{\text{crs}'} = \sum_{i=1}^t X_i$ has $\mathbb{E}[\text{counter}] = (1 - \sigma) \cdot t$, and applying the multiplicative Chernoff bound, we have that (when crs' is chosen),

$$\Pr \left[X_{\text{crs}'} \leq \frac{(1 - \sigma) \cdot t}{2} \right] \leq 2^{-(1 - \sigma) \cdot t \cdot \log(e)/8} \leq 2^{-\lambda}.$$

Observe that, by completeness of the original SNARG, $\Pr_{\text{crs}'}[p_{\text{crs}'} \geq (1 - \sigma)] \geq 1 - \sigma$. Since $\mathcal{V}$ accepts if and only if $\text{counter} \geq (1 - \sigma)t/2$, which is equivalent to requiring that crs' is chosen such that $X_{\text{crs}'} \geq (1 - \sigma)t/2$, we have that $\mathcal{V}$ rejects with probability at most

$$\Pr[p_{\text{crs}'} < 1 - \sigma] + \Pr \left[X_{\text{crs}'} \leq \frac{(1 - \sigma) \cdot t}{2} \mid p_{\text{crs}'} \geq 1 - \sigma \right] \leq \sigma + 2^{-\lambda},$$

where above the probabilities are over the randomness of crs (which is the only randomness in the new SNARG).

Soundness. Fix an instance $\mathbb{x} \notin L$ and a malicious prover $\tilde{\mathcal{P}}$. For a common reference string crs' and proof string Π let

$$p_{\text{crs}', \Pi} = \Pr_{r_{\mathcal{V}}}[\mathcal{V}'(\text{crs}, \mathbb{x}, \Pi; r_{\mathcal{V}}) = 1].$$

Fix crs' and Π with $p_{\text{crs}', \Pi} < (1 - \sigma)/4$ and let X_i be the 0/1 random variable where

$$X_i = 1 \iff \mathcal{V}'_{\text{crs}'}(\mathbb{x}, \mathcal{P}'_{\text{crs}'}(\mathbb{x}, \mathbb{w}); r_i) = 1,$$

Then $X_{\text{crs}', \Pi} = \sum_{i=1}^t X_i$ has expectation $\mathbb{E}[X_{\text{crs}', \Pi}] < (1 - \sigma) \cdot t/4$. By the multiplicative Chernoff bound,

$$\Pr_{r_1, \dots, r_t} \left[X_{\text{crs}', \Pi} \geq \frac{(1 - \sigma) \cdot t}{2} \right] \leq 2^{-4(1 - \sigma) \cdot \log(e) \cdot t/3} \leq 2^{-(\lambda + |\text{crs}'| + |\Pi|)}.$$

By taking a union bound over all crs' and Π , we have that

$$\Pr_{r_1, \dots, r_t} \left[\exists \text{crs}', \Pi : p_{\text{crs}', \Pi} \leq (1 - \sigma) \cdot t/4 \wedge X_{\text{crs}', \Pi} \geq \frac{(1 - \sigma) \cdot t}{2} \right] \leq 2^{-\lambda}$$

Finally, observe that by soundness of the SNARG:

$$\begin{aligned} \beta &\geq \Pr_{r_{\mathcal{V}}}[\mathcal{V}'(\text{crs}', \mathbb{x}, \tilde{\mathcal{P}}(\text{crs}'); r_{\mathcal{V}}) = 1] = \sum_{\delta} \delta \cdot \Pr_{\text{crs}'}[p_{\text{crs}', \tilde{\mathcal{P}}(\text{crs}')} = \delta] \\ &\geq (1 - \sigma)/4 \cdot \Pr_{\text{crs}'}[p_{\text{crs}', \tilde{\mathcal{P}}(\text{crs}')} \geq (1 - \sigma)/4], \end{aligned}$$

and so $\Pr_{\text{crs}'}[p_{\text{crs}', \tilde{\mathcal{P}}(\text{crs}')} \geq (1 - \sigma)/4] \leq 4\beta/(1 - \sigma)$.

Since $\mathcal{V}$ accepts if and only if $\text{counter} \geq (1 - \sigma)t/2$, which is equivalent to requiring that crs' and $\Pi = \tilde{\mathcal{P}}(\text{crs}')$ is chosen such that $X_{\text{crs}', \tilde{\mathcal{P}}(\text{crs}')} \geq (1 - \sigma)t/2$, we have that $\mathcal{V}$ accepts with probability at most

$$\Pr_{\text{crs}'}[p_{\text{crs}', \tilde{\mathcal{P}}(\text{crs}')} \geq (1 - \sigma)/4] + \Pr_{\text{crs}', r_1, \dots, r_t} \left[X_{\text{crs}', \tilde{\mathcal{P}}(\text{crs}')} \geq \frac{(1 - \sigma) \cdot t}{2} \mid p_{\text{crs}', \tilde{\mathcal{P}}(\text{crs}')} \leq (1 - \sigma) \cdot t/4 \right],$$

which is bounded by $4\beta/(1 - \sigma) + 2^{-\lambda}$. $\square$

6.1.2 Defining proof-dependent and independent queries

In the generic group model for asymmetric pairing groups, any group element that an algorithm produces can be thought of as a polynomial in the algorithm's inputs. In our setting, for example, any group element the verifier receives can be thought of as a polynomial f in the formal variables $\hat{\Pi}$, crs_1 , crs_2 , and crs_T . These formal variables represent the proof Π , and the vector of $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ elements in the common reference string crs .

We consider the case where the proof is an element in $\mathbb{G}_1$. By symmetry between $\mathbb{G}_1$ and $\mathbb{G}_2$ this suffices also for the case that the proof is in $\mathbb{G}_2$. The proof for $\mathbb{G}_T$ is similar while accounting to the different structure of the possible ways the prover can generate a $\mathbb{G}_T$ proof. As the proof is otherwise identical, we focus on $\mathbb{G}_1$.

To indicate that the proof is in $\mathbb{G}_1$, we write $[\Pi]_1$ from here on.

Then in the verifier all elements in $\mathbb{G}_1$ can be expressed by a polynomial $a\hat{\Pi} + \langle \mathbf{b}, \text{crs}_1 \rangle$ where $a \in \mathbb{Z}_p$, and $\mathbf{b}$ is a vector over $\mathbb{Z}_p$ because $[\Pi]_1$ and the elements in $[\text{crs}_1]_1$ are in $\mathbb{G}_1$. Similarly, all elements in $\mathbb{G}_2$ can be expressed by a polynomial $\langle \mathbf{b}, \text{crs}_2 \rangle$ because $[\Pi]_1$ can not be moved to $\mathbb{G}_2$ and all $\mathbb{G}_T$ elements can be represented by $\langle \mathbf{a}, \hat{\Pi} \cdot \text{crs}_2 \rangle + \langle \mathbf{b}, \text{crs}_1 \otimes \text{crs}_2 \rangle + \langle \mathbf{c}, \text{crs}_T \rangle$.

These properties allow us to define a function **IsIndependent** which determines whether the output of such a polynomial is independent of the proof $[\Pi]_1$. Essentially, it is independent if all coefficients in monomials containing $\hat{\Pi}$ are 0. More precisely,

IsIndependent_{crs}($f(\hat{\Pi}, \text{crs}_1, \text{crs}_2, \text{crs}_T)$): where $\hat{\Pi}$, crs_1 , crs_2 , and crs_T are formal variables representing the proof and the $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_T$ elements in the common reference string, respectively. We

abuse notation and say that f outputs a $\mathbb{G}_1, \mathbb{G}_2$ or $\mathbb{G}_T$ element, meaning that the output of f represents the discrete logarithm of an element of that group. Whenever `IsIndependent` is called, it will be clear which group this is referring to, and so we do not specify this as an input, but leave it implicit.

1. If f outputs a $\mathbb{G}_1$ element:

(a) f has the form

$$a\hat{\Pi} + \langle \mathbf{b}, \hat{\mathbf{c}}\mathbf{s}_1 \rangle$$

where $a \in \mathbb{Z}_p$, and $\mathbf{b}$ is a vector over $\mathbb{Z}_p$.

(b) If $a = 0$ then output 1.

(c) If $a \neq 0$ then output 0.

2. If f outputs a $\mathbb{G}_2$ element then output 1.

3. If f outputs a $\mathbb{G}_T$ element:

(a) f has the form

$$\hat{\Pi} \cdot \langle \mathbf{a}, \hat{\mathbf{c}}\mathbf{s}_2 \rangle + \langle \mathbf{b}, \hat{\mathbf{c}}\mathbf{s}_1 \otimes \hat{\mathbf{c}}\mathbf{s}_2 \rangle + \langle \mathbf{c}, \hat{\mathbf{c}}\mathbf{s}_T \rangle$$

where $\mathbf{a}, \mathbf{b}, \mathbf{c}$ are vectors over $\mathbb{Z}_p$.

(b) If $\langle \mathbf{a}, [\mathbf{crs}_2]_2 \rangle = [0]_2$ then output 1.

(c) If $\langle \mathbf{a}, [\mathbf{crs}_2]_2 \rangle \neq [0]_2$ then output 0.

Crucial observation: Our proof relies on the fact that any algorithm that has access to `crs` and the structure of f can compute `IsIndependentcrs` on their own. For $\mathbb{G}_2$ elements this is trivial. For $\mathbb{G}_1$ elements this requires only knowing a , which is fixed in f . For $\mathbb{G}_T$ elements, since $\mathbf{a}$ is known, it is possible to compute the group element whose exponent is $\langle \mathbf{a}, [\mathbf{crs}_2]_2 \rangle$, and then check whether this exponent is 0 or not via a GGM equality check.

Claim 6.6. *Let f a function such that `IsIndependent`(f) = 0. Then for every `crs` there exists a function f_{crs}^{-1} such that $f_{\text{crs}}^{-1}(f(\hat{\Pi}, \text{crs})) = \hat{\Pi}$.*

Proof. The function f^{-1} will be different depending on to which group the output of the query that f matches. For a common reference string `crs`, let $[\text{crs}_1]_1$, $[\text{crs}_2]_2$, and $[\text{crs}_T]_T$ be the elements in $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ respectively.

f outputs a $\mathbb{G}_1$ element: then f has the form $a\hat{\Pi} + \langle \mathbf{b}, \hat{\mathbf{c}}\mathbf{s}_1 \rangle$ where $a \in \mathbb{Z}_p$ is non-zero, and $\mathbf{b}$ is a vector over $\mathbb{Z}_p$. Then

$$f_{\text{crs}}^{-1}(\hat{Y}) = \frac{\hat{Y} - \langle \mathbf{b}, \text{crs}_1 \rangle}{a}.$$

f outputs a $\mathbb{G}_2$ element: this cannot hold as `IsIndependent`(f) = 1 whenever f outputs a $\mathbb{G}_2$ element.

f outputs a $\mathbb{G}_T$ element: then f has the form

$$\langle \mathbf{a}, \hat{\Pi} \cdot \hat{\mathbf{c}}\mathbf{s}_2 \rangle + \langle \mathbf{b}, \hat{\mathbf{c}}\mathbf{s}_1 \otimes \hat{\mathbf{c}}\mathbf{s}_2 \rangle + \langle \mathbf{c}, \hat{\mathbf{c}}\mathbf{s}_T \rangle$$

where $\mathbf{a}, \mathbf{b}, \mathbf{c}$ are vectors over $\mathbb{Z}_p$, and $\langle \mathbf{a}, [\text{crs}_2]_2 \rangle \neq 0$. Then

$$f_{\text{crs}}^{-1}(\hat{Y}) = \frac{\hat{Y} - \langle \mathbf{b}, \text{crs}_1 \otimes \text{crs}_2 \rangle - \langle \mathbf{c}, \text{crs}_T \rangle}{\langle \mathbf{a}, \text{crs}_2 \rangle}.$$

□

6.1.3 Removing proof-independent queries

Lemma 6.7. *Let $(\mathcal{S}^{\mathcal{G}}, \mathcal{P}^{\mathcal{G}, \mathcal{O}}, \mathcal{V}^{\mathcal{G}, \mathcal{O}})$ be a single element SNARG for relation $\mathcal{R}$ in the Maurer Bilinear GGM $(\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T)$ with a random oracle with a deterministic verifier that makes non-adaptive ROM $\mathbb{G}_{\{1,2,T\}}^* \rightarrow \{0,1\} \times \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T$. Then there exists a single element SNARG for relation $\mathcal{R}$ in the Maurer Bilinear GGM with a random oracle with a deterministic verifier where for every crs in the image of the setup algorithm the polynomial f describing any verifier ROM query has $\text{IsIndependent}_{\text{crs}}(f) = 1$. Completeness and soundness are preserved, and all other parameters are polynomially related.*

Remark 6.8. Notice, the random oracle may take multiple group elements as input and outputs a bit and one group element from each group. This setting covers all possible ways of calling a random oracle. If an algorithm wants to hash bits, it can first encode them into group elements and then query the ROM with those group elements. With the proper domain separation, this does not cause any complications.

Proof. Given a SNARG $(\mathcal{S}^{\mathcal{G}}, \mathcal{P}^{\mathcal{G}, \mathcal{O}}, \mathcal{V}^{\mathcal{G}, \mathcal{O}})$ in the Maurer Bilinear GGM with a random oracle. Let $T_{\mathcal{V}}$ be a bound on the running time of $\mathcal{V}$ (which also bounds its query complexity) and n be an upper bound on the number of group elements that are used as input to the random oracle in a single query (observe that both parameters are polynomial in the instance size and security parameter). We design a SNARG $(\mathcal{S}, \mathcal{P}, \mathcal{V})$ where all verifier ROM queries are proof-dependent:

- $\mathcal{S}^{\mathcal{G}}(1^\lambda)$:
 1. Sample $\text{crs}' \leftarrow \mathcal{S}'^{\mathcal{G}}(1^\lambda)$.
 2. For $i \in [T_{\mathcal{V}}]$ sample $r_i \leftarrow \{0,1\}$ uniformly at random.
 3. For $i \in [T_{\mathcal{V}}]$, $j \in \{1,2,T\}$ sample $s_{i,j} \leftarrow \mathbb{Z}_p$ uniformly at random.
 4. Output $\text{crs} := (\text{crs}', (r_i)_{i \in [T_{\mathcal{V}}]}, ([s_{i,j}]_j)_{i \in [T_{\mathcal{V}}], j \in \{1,2,T\}})$
- $\mathcal{P}^{\mathcal{G}, \mathcal{O}}(\text{crs}, \mathbb{x}, \mathbb{w})$:
 1. Let $[\text{crs}_1]_1$ (resp. $[\text{crs}_2]_2$ and $[\text{crs}_T]_T$) be the group elements in crs' which are $\mathbb{G}_1$ elements (resp. $\mathbb{G}_2$ and $\mathbb{G}_T$).
 2. Simulate $\mathcal{V}^{\mathcal{G}, \mathcal{O}}(\text{crs}, \mathbb{x}, [\Pi]_1 := [1]_1)$. For each query $i \in [T_{\mathcal{V}}]$ and group element in that query $j \in [n]$ there is a polynomial $f_{i,j}(\hat{\Pi}, \text{crs}')$ outputting a group element from either $\mathbb{G}_1$, $\mathbb{G}_2$, or $\mathbb{G}_T$.
 3. Initialize a lazily sampled random oracle $\mathcal{SO}$.
 4. For each $i \in [T_{\mathcal{V}}]$ if for all $j \in [n]$ it holds that $\text{IsIndependent}_{\text{crs}}(f_{i,j}(\hat{\Pi}, \text{crs}')) = 1$ then program $\mathcal{SO}$ to output $(r_i, [s_{i,1}]_1, [s_{i,2}]_2, [s_{i,T}]_T)$ on input the group element $f_{i,j}([\Pi]_1 := [1]_1, \text{crs}')$.
 5. Output $[\Pi]_1 := \mathcal{P}^{\mathcal{G}, \mathcal{SO}}(\text{crs}, \mathbb{x}, \mathbb{w})$ where for a query X not programmed into $\mathcal{SO}$ we respond with $\mathcal{O}(X)$.
- $\mathcal{V}^{\mathcal{G}, \mathcal{O}}(\text{crs}, \mathbb{x}, [\Pi]_1)$:
 1. Let $[\text{crs}_1]_1$ (resp. $[\text{crs}_2]_2$ and $[\text{crs}_T]_T$) be the group elements in crs' which are $\mathbb{G}_1$ elements (resp. $\mathbb{G}_2$ and $\mathbb{G}_T$).
 2. Simulate $\mathcal{V}^{\mathcal{G}, \mathcal{O}}(\text{crs}, \mathbb{x}, [\Pi]_1 := [1]_1)$. For each query $i \in [T_{\mathcal{V}}]$ and group element in that query $j \in [n]$ there is a polynomial $f_{i,j}(\hat{\Pi}, \text{crs}')$ outputting a group element from either $\mathbb{G}_1$, $\mathbb{G}_2$, or $\mathbb{G}_T$.

3. For each $i \in [T_V]$ if for all $j \in [n]$ it holds that $\text{IsIndependent}_{\text{crs}}(f_{i,j}(\hat{\Pi}, \text{crs}')) = 1$ then program $\mathcal{SO}$ to output $(r_i, [s_{i,1}]_1, [s_{i,2}]_2, [s_{i,T}]_T)$ on input the group element $f_{i,j}([\Pi]_1 := [1]_1, \text{crs}')$.
4. Output $b := \mathcal{V}^{\mathcal{G}, \mathcal{SO}}(\text{crs}, \mathbb{x}, \mathbb{w})$ where for a query $[X]_*$ not programmed into $\mathcal{SO}$ we respond with $\mathcal{O}([X]_*)$.

First, note that since T_V is polynomial, the CRS length and all other efficiency parameters remain polynomial (the proof length remains identical).

Recall that $\mathcal{V}'$ is non-adaptive with respect to the random oracle queries, meaning that the executions $\mathcal{V}'^{\mathcal{G}, \mathcal{O}}(\text{crs}, \mathbb{x}, [1]_1)$ and $\mathcal{V}'^{\mathcal{G}, \mathcal{SO}}(\text{crs}, \mathbb{x}, [\Pi]_1)$ yield the same polynomials $f_{i,j}$ that produce the queries to the random oracle. Because $\mathcal{V}'$ is deterministic and IsIndependent is efficiently computable $\mathcal{P}$ and $\mathcal{V}$ will agree on which responses to retrieve from crs . Further, $(r_i, [s_{i,1}]_1, [s_{i,2}]_2, [s_{i,T}]_T)$ follow the same distribution as if drawn from a random oracle. Therefore, this transformation preserves completeness.

Soundness of $(\mathcal{S}', \mathcal{P}', \mathcal{V}')$ follows from the fact any adversary $\mathcal{A}'$ against its non-adaptive security can be transformed into an attack against $(\mathcal{S}, \mathcal{P}, \mathcal{V})$ by sampling $(r_i, [s_{i,1}]_1, [s_{i,2}]_2, [s_{i,T}]_T)$ from the random oracle at which the verifier is going to make proof independent queries. $\square$

6.1.4 Removing proof-dependent queries

In the next step, we remove all the queries that depend on the proof. We use the fact that any query that both prover and verifier have in common uniquely defines the proof because the query is a degree 1 polynomial in the proof.

Lemma 6.9. *Let $(\mathcal{S}^{\mathcal{G}}, \mathcal{P}'^{\mathcal{G}, \mathcal{O}}, \mathcal{V}'^{\mathcal{G}, \mathcal{O}})$ be a single element SNARG for a relation $\mathcal{R}$ in the Maurer Bilinear GGM $(\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T)$ with a random oracle, completeness error σ , soundness error β , and non-adaptive ROM $\mathbb{G}_{\{1,2,T\}}^* \rightarrow \{0,1\} \times \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T$ queries by the deterministic verifier where for every crs in the image of the setup algorithm the polynomial f describing any verifier ROM query has $\text{IsIndependent}_{\text{crs}}(f) = 1$. Then, there exists a single element SNARG for $\mathcal{R}$ $(\mathcal{S}^{\mathcal{G}}, \mathcal{P}^{\mathcal{G}}, \mathcal{V}^{\mathcal{G}})$ in the Maurer GGM (without a random oracle), with completeness error $\sigma + 1 - 1/\text{poly}(\lambda)$, soundness error β , and randomized verifier.*

Proof. Given a SNARG $(\mathcal{S}^{\mathcal{G}}, \mathcal{P}'^{\mathcal{G}, \mathcal{O}}, \mathcal{V}'^{\mathcal{G}, \mathcal{O}})$ in the Maurer GGM with ROM we prove that the following protocol in the Maurer GGM without ROM is also a SNARG.

- $\mathcal{S}^{\mathcal{G}}(1^\lambda)$:
 1. Output $\text{crs} \leftarrow \mathcal{S}^{\mathcal{G}}(1^\lambda)$.
- $\mathcal{P}^{\mathcal{G}}(\text{crs}, \mathbb{x}, \mathbb{w})$:
 1. Lazily sample a random oracle $\mathcal{SO}_{\mathcal{P}}$.
 2. Output $[\Pi]_1 \leftarrow \mathcal{P}'^{\mathcal{G}, \mathcal{SO}_{\mathcal{P}}}(\text{crs}, \mathbb{x}, \mathbb{w})$.
- $\mathcal{V}^{\mathcal{G}}(\text{crs}, \mathbb{x}, [\Pi]_1)$:
 1. Lazily sample a random oracle $\mathcal{SO}_{\mathcal{V}}$.
 2. If $\mathcal{V}'^{\mathcal{G}, \mathcal{SO}_{\mathcal{V}}}(\text{crs}, \mathbb{x}, [\Pi]_1) = 1$ then output 1.
 3. Otherwise, output 0.

The fact that soundness is preserved follows immediately from the verifier having the exact same distribution as in $(\mathcal{S}', \mathcal{P}', \mathcal{V}')$. Further, any adversary can not predict the response to a simulated random oracle query the verifier does because they are randomly sampled on the spot. We thus turn to proving completeness:

Claim 6.10. $(\mathcal{S}, \mathcal{P}, \mathcal{V})$ has completeness error at most $\sigma + 1 - 1/\text{poly}(\lambda)$.

Proof. Fix an instance $\mathbf{x}$ and a witness $\mathbf{w}$. We prove that

$$\begin{aligned} c &= \Pr_{r_{\mathcal{V}}, r_{\mathcal{P}}} \left[\mathcal{V}^{\mathcal{G}}(\text{crs}, \mathbf{x}, [\Pi]_1; r_{\mathcal{V}}) = 1 \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}^{\mathcal{G}}(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}^{\mathcal{G}}(\text{crs}, \mathbf{x}, \mathbf{w}; r_{\mathcal{P}}) \end{array} \right] \\ &= \Pr_{\mathcal{SO}_{\mathcal{V}}, \mathcal{SO}_{\mathcal{P}}} \left[b = 1 \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^{\mathcal{G}}(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^{\mathcal{G}, \mathcal{SO}_{\mathcal{P}}}(\text{crs}, \mathbf{x}, \mathbf{w}) \\ b \leftarrow \mathcal{V}'^{\mathcal{G}, \mathcal{SO}_{\mathcal{V}}}(\text{crs}, \mathbf{x}, [\Pi]_1) \end{array} \right] \\ &\geq 1/\text{poly}(\lambda) \end{aligned}$$

We define p_{cq} to be the probability of prover and verifier having a common query.

$$\begin{aligned} p_{\text{cq}} &= \Pr_{\mathcal{SO}_{\mathcal{V}}, \mathcal{SO}_{\mathcal{P}}} \left[[\mathcal{X}_{\mathcal{P}}]_* \cap [\mathcal{X}_{\mathcal{V}}]_* \neq \emptyset \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^{\mathcal{G}}(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^{\mathcal{G}, \mathcal{SO}_{\mathcal{P}}}(\text{crs}, \mathbf{x}, \mathbf{w}) \text{ with } \mathcal{SO}_{\mathcal{P}}\text{-queries } [\mathcal{X}_{\mathcal{P}}]_*, \\ \mathcal{V}'^{\mathcal{G}, \mathcal{SO}_{\mathcal{V}}}(\text{crs}, \mathbf{x}, [\Pi]_1) \text{ with } \mathcal{SO}_{\mathcal{V}}\text{-queries } [\mathcal{X}_{\mathcal{V}}]_* \end{array} \right] \\ &= \Pr_{\mathcal{SO}} \left[[\mathcal{X}_{\mathcal{P}}]_* \cap [\mathcal{X}_{\mathcal{V}}]_* \neq \emptyset \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^{\mathcal{G}}(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^{\mathcal{G}, \mathcal{SO}}(\text{crs}, \mathbf{x}, \mathbf{w}) \text{ with } \mathcal{SO}\text{-queries } [\mathcal{X}_{\mathcal{P}}]_*, \\ \mathcal{V}'^{\mathcal{G}, \mathcal{SO}}(\text{crs}, \mathbf{x}, [\Pi]_1) \text{ with } \mathcal{SO}\text{-queries } [\mathcal{X}_{\mathcal{V}}]_* \end{array} \right] \end{aligned}$$

This follows from the verifier only making non-adaptive queries, which means that the queries $[\mathcal{X}_{\mathcal{P}}]_*$ and $[\mathcal{X}_{\mathcal{V}}]_*$ only depend on $\mathcal{SO}_{\mathcal{P}}$.

First, we look at the case that it is probable that prover and verifier do not share queries.

Specifically, if $p_{\text{cq}} < 1/2$ we get that

$$\begin{aligned}
& \Pr_{\mathcal{SO}_V, \mathcal{SO}_P} \left[b = 1 \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^G(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^G, \mathcal{SO}_P(\text{crs}, \mathbb{x}, \mathbb{w}) \\ b \leftarrow \mathcal{V}'^G, \mathcal{SO}_V(\text{crs}, \mathbb{x}, [\Pi]_1) \end{array} \right] \\
& \geq \Pr_{\mathcal{SO}_V, \mathcal{SO}_P} \left[b = 1, \begin{array}{l} [\mathcal{X}_P]_* \cap [\mathcal{X}_V]_* = \emptyset \\ \text{crs} \leftarrow \mathcal{S}'^G(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^G, \mathcal{SO}_P(\text{crs}, \mathbb{x}, \mathbb{w}) \text{ with } \mathcal{SO}_P\text{-queries } [\mathcal{X}_P]_*. \\ b \leftarrow \mathcal{V}'^G, \mathcal{SO}_V(\text{crs}, \mathbb{x}, [\Pi]_1) \text{ with } \mathcal{SO}_V\text{-queries } [\mathcal{X}_V]_*. \end{array} \right] \\
& = \Pr_{\mathcal{SO}_V, \mathcal{SO}_P} \left[b = 1 \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^G(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^G, \mathcal{SO}_P(\text{crs}, \mathbb{x}, \mathbb{w}) \text{ with } \mathcal{SO}_P\text{-queries } [\mathcal{X}_P]_*. \\ b \leftarrow \mathcal{V}'^G, \mathcal{SO}_V(\text{crs}, \mathbb{x}, [\Pi]_1) \text{ with } \mathcal{SO}_V\text{-queries } [\mathcal{X}_V]_*. \\ [\mathcal{X}_P]_* \cap [\mathcal{X}_V]_* = \emptyset \end{array} \right] \cdot (1 - p_{\text{cq}}) \\
& \geq \Pr_{\mathcal{SO}} \left[b = 1 \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^G(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^G, \mathcal{SO}(\text{crs}, \mathbb{x}, \mathbb{w}) \text{ with } \mathcal{SO}\text{-queries } [\mathcal{X}_P]_*. \\ b \leftarrow \mathcal{V}'^G, \mathcal{SO}(\text{crs}, \mathbb{x}, [\Pi]_1) \text{ with } \mathcal{SO}\text{-queries } [\mathcal{X}_V]_*. \\ [\mathcal{X}_P]_* \cap [\mathcal{X}_V]_* = \emptyset \end{array} \right] \cdot (1 - p_{\text{cq}}) \tag{1} \\
& = \Pr_{\mathcal{SO}} \left[\begin{array}{l} b \neq 0, \\ [\mathcal{X}_P]_* \cap [\mathcal{X}_V]_* = \emptyset \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^G(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^G, \mathcal{SO}(\text{crs}, \mathbb{x}, \mathbb{w}) \text{ with } \mathcal{SO}\text{-queries } [\mathcal{X}_P]_*. \\ b \leftarrow \mathcal{V}'^G, \mathcal{SO}(\text{crs}, \mathbb{x}, [\Pi]_1) \text{ with } \mathcal{SO}\text{-queries } [\mathcal{X}_V]_*. \end{array} \right] \\
& \geq 1 - \sigma - p_{\text{cq}} \tag{2} \\
& > 1/4 > 1/\text{poly}(\lambda) - \sigma
\end{aligned}$$

Equality 1 follows from the fact that if prover and verifier queries are distinct then the $\mathcal{SO}_P$ and $\mathcal{SO}_V$ offer a consistent view of a random oracle. Inequality 2 follows by union bound over completeness of $(\mathcal{S}', \mathcal{P}', \mathcal{V}')$ and the event that $[\mathcal{X}_P]_* \cap [\mathcal{X}_V]_* = \emptyset$.

For the rest of the proof we consider the case that $p_{\text{cq}} \geq 1/2$. With T_P (respective T_V) being the number of ROM queries of the prover (respective verifier) there exist an $i^* \in [T_P]$ and $j^* \in [T_V]$ such that

$$p_{i^*, j^*} \tag{3}$$

$$\begin{aligned}
& = \Pr_{\mathcal{SO}_P, \mathcal{SO}_V} \left[\begin{array}{l} [X_{P, i^*}]_* = [X_{V, j^*}]_*, \\ \forall i < i^*, j \in [T_V] : [X_{P, i}]_* \neq [X_{V, j}]_* \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^G(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^G, \mathcal{SO}_P(\text{crs}, \mathbb{x}, \mathbb{w}) \\ \text{where } [X_{P, i}]_* \text{ is the } i\text{-th } \mathcal{SO}_P\text{-query.} \\ b \leftarrow \mathcal{V}'^G, \mathcal{SO}_V(\text{crs}, \mathbb{x}, [\Pi]_1) \\ \text{where } [X_{V, j}]_* \text{ is the } j\text{-th } \mathcal{SO}_V\text{-query.} \end{array} \right] \\
& = \Pr_{\mathcal{SO}} \left[\begin{array}{l} [X_{P, i^*}]_* = [X_{V, j^*}]_*, \\ \forall i < i^*, j \in [T_V] : [X_{P, i}]_* \neq [X_{V, j}]_* \end{array} \mid \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^G(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^G, \mathcal{SO}(\text{crs}, \mathbb{x}, \mathbb{w}) \\ \text{where } [X_{P, i}]_* \text{ is the } i\text{-th } \mathcal{SO}\text{-query.} \\ b \leftarrow \mathcal{V}'^G, \mathcal{SO}(\text{crs}, \mathbb{x}, [\Pi]_1) \\ \text{where } [X_{V, j}]_* \text{ is the } j\text{-th } \mathcal{SO}\text{-query.} \end{array} \right] \tag{4}
\end{aligned}$$

$$\geq 1/2T_P T_V. \tag{5}$$

Equality 4 follows from the verifier only making non-adaptive queries, which means that the queries $X_{P, i}$ and $X_{V, j}$ only depend on $\mathcal{SO}_P$. Inequality 5 follows from an averaging argument and $\mathcal{X}_P \cap \mathcal{X}_V$ being equivalent to the statement $\exists i^*, j^* : X_{P, i^*} = X_{V, j^*}$ and $\forall i < i^*, j \in [T_V] : X_{P, i} \neq X_{V, j}$.

Let n be the upper bound on how many elements a ROM query by the verifier takes as input. Each of query j of the T_V queries is defined by n polynomials over crs and Π . We call these polynomials $(f_{j,k}(\hat{\Pi}, \text{crs}))_{j \in [T_V], k \in [n]}$. For each $j \in [T_V]$ and $k \in [n]$ with $\text{IsIndependent}(f_{j,k}(\hat{\Pi}, \text{crs})) = 0$ we let $f_{j,k,\text{crs}}^{-1}$ be the function derived by Claim 6.6 (i.e., such that for every crs it holds that $f_{j,k,\text{crs}}^{-1}(f_{j,k}(\hat{\Pi}, \text{crs})) = \hat{\Pi}$).

We use these functions in the following analysis of the accepting probability. With the rules of probability we get

$$\begin{aligned}
& \Pr_{\mathcal{SO}_V, \mathcal{SO}_P} \left[b = 1 \left| \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^G(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^G, \mathcal{SO}_P(\text{crs}, \mathbb{x}, \mathbb{w}), \\ b \leftarrow \mathcal{V}'^G, \mathcal{SO}_V(\text{crs}, \mathbb{x}, [\Pi]_1) \end{array} \right. \right] \\
& \geq \Pr_{\mathcal{SO}_V, \mathcal{SO}_P} \left[\begin{array}{l} b = 1, \\ [X_{\mathcal{P}, i^*}]_* = [X_{\mathcal{V}, j^*}]_*, \\ \forall_{i < i^*, j \in [T_V]} : [X_{\mathcal{P}, i}]_* \neq [X_{\mathcal{V}, j}]_* \end{array} \left| \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^G(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^G, \mathcal{SO}_P(\text{crs}, \mathbb{x}, \mathbb{w}) \\ \text{where } [X_{\mathcal{P}, i}]_* \text{ is the } i\text{-th } \mathcal{SO}_P\text{-query,} \\ b \leftarrow \mathcal{V}'^G, \mathcal{SO}_V(\text{crs}, \mathbb{x}, [\Pi]_1) \\ \text{where } [X_{\mathcal{V}, j}]_* \text{ is the } j\text{-th } \mathcal{SO}_V\text{-query} \end{array} \right. \right] \\
& = \Pr_{\mathcal{SO}_V, \mathcal{SO}_P} \left[b = 1 \left| \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^G(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^G, \mathcal{SO}_P(\text{crs}, \mathbb{x}, \mathbb{w}) \\ \text{where } [X_{\mathcal{P}, i}]_* \text{ is the } i\text{-th } \mathcal{SO}_P\text{-query,} \\ b \leftarrow \mathcal{V}'^G, \mathcal{SO}_V(\text{crs}, \mathbb{x}, [\Pi]_1) \\ \text{where } [X_{\mathcal{V}, j}]_* \text{ is the } j\text{-th } \mathcal{SO}_V\text{-query,} \\ [X_{\mathcal{P}, i^*}]_* = [X_{\mathcal{V}, j^*}]_*, \\ \forall_{i < i^*, j \in [T_V]} : [X_{\mathcal{P}, i}]_* \neq [X_{\mathcal{V}, j}]_* \end{array} \right. \right] \cdot (1 - p_{i^*, j^*})
\end{aligned}$$

We introduce some notation. By $\text{st} \leftarrow \mathcal{P}'^G, \mathcal{SO}_P[0, i^*](\text{crs}, \mathbb{x}, \mathbb{w})$ we mean run $\mathcal{P}'^G, \mathcal{SO}_P(\text{crs}, \mathbb{x}, \mathbb{w})$ until its i^* -th $\mathcal{SO}_P$ -query and let st be the state of computation it is in before receiving the response to the i^* -th $\mathcal{SO}_P$ -query. Similarly, we define $\mathcal{P}'^G, \mathcal{SO}_P[i^*, T_P](\text{crs}, \mathbb{x}, \mathbb{w}, \text{st})$ is the continuation of the above mentioned execution until the end.

For $\text{st} \leftarrow \mathcal{P}'^G, \mathcal{SO}_P[0, i^*](\text{crs}, \mathbb{x}, \mathbb{w})$ we have $\mathcal{P}'^G, \mathcal{SO}_P[i^*, T_P](\text{crs}, \mathbb{x}, \mathbb{w}, \text{st}) = \Pi = f_{j^*, k^*, \text{crs}}^{-1}(X_{\mathcal{V}, j^*}[k^*])$ and $X_{\mathcal{V}, j^*} = X_{\mathcal{P}, i^*}$. Therefore, we can also compute Π from $f_{j^*, \text{crs}}^{-1}(X_{\mathcal{P}, i^*}[k^*])$.

$$= \Pr_{\mathcal{SO}_V, \mathcal{SO}_P} \left[b = 1 \left| \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^G(1^\lambda), \\ \text{Run } \mathcal{P}'^G, \mathcal{SO}_P[0, i^*](\text{crs}, \mathbb{x}, \mathbb{w}) \text{ where } [X_{\mathcal{P}, i}]_* \text{ is the } i\text{-th } \mathcal{SO}_P\text{-query,} \\ \text{Let } k^* \text{ be the smallest index s.t. } \text{IsIndependent}(f_{j^*, k^*}) = 1, \\ \Pi \leftarrow f_{j^*, k^*, \text{crs}}^{-1}(X_{\mathcal{P}, i^*}[k^*]), \\ b \leftarrow \mathcal{V}'^G, \mathcal{SO}_V(\text{crs}, \mathbb{x}, [\Pi]_1) \text{ where } [X_{\mathcal{V}, j}]_* \text{ is the } j\text{-th } \mathcal{SO}_V\text{-query,} \\ [X_{\mathcal{P}, i^*}]_* = [X_{\mathcal{V}, j^*}]_*, \\ \forall_{i < i^*, j \in [T_V]} : [X_{\mathcal{P}, i}]_* \neq [X_{\mathcal{V}, j}]_* \end{array} \right. \right] \cdot (1 - p_{i^*, j^*})$$

Now, we have that the queries on $\mathcal{SO}_V$ and $\mathcal{SO}_P$ are entirely distinct. Therefore, if $\mathcal{V}'$ uses $\mathcal{SO}_P$

the probabilities are identical.

$$= \Pr_{\mathcal{SO}_{\mathcal{P}}} \left[b = 1 \left| \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^{\mathcal{G}}(1^\lambda), \\ \text{Run } \text{st} \leftarrow \mathcal{P}'^{\mathcal{G}, \mathcal{SO}_{\mathcal{P}}[0, i^*]}(\text{crs}, \mathbb{x}, \mathbb{w}) \text{ where } [X_{\mathcal{P}, i}]_* \text{ is the } i\text{-th } \mathcal{SO}_{\mathcal{P}}\text{-query,} \\ \text{Let } k^* \text{ be the smallest index s.t. } \text{IsIndependent}(f_{j^*, k^*}) = 1, \\ \Pi \leftarrow f_{j^*, k^*}^{-1, \text{crs}}(X_{\mathcal{P}, i^*}[k^*]), \\ b \leftarrow \mathcal{V}'^{\mathcal{G}, \mathcal{SO}_{\mathcal{P}}}(\text{crs}, \mathbb{x}, [\Pi]_1) \text{ where } [X_{\mathcal{V}, j}]_* \text{ is the } j\text{-th } \mathcal{SO}_{\mathcal{P}}\text{-query,} \\ [X_{\mathcal{P}, i^*}]_* = [X_{\mathcal{V}, j^*}]_*, \\ \forall_{i < i^*, j \in [T_{\mathcal{V}}]} : [X_{\mathcal{P}, i}]_* \neq [X_{\mathcal{V}, j}]_* \end{array} \right. \right] \cdot (1 - p_{i^*, j^*})$$

We undo a previous step and compute Π using $\mathcal{P}'^{\mathcal{G}, \mathcal{SO}_{\mathcal{P}}[i^*, T_{\mathcal{P}}]}(\text{crs}, \mathbb{x}, \mathbb{w}, \text{st})$. We can do this because we have that $[X_{\mathcal{V}, j^*}]_* = [X_{\mathcal{P}, i^*}]_*$.

$$= \Pr_{\mathcal{SO}} \left[b = 1 \left| \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^{\mathcal{G}}(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^{\mathcal{G}, \mathcal{SO}}(\text{crs}, \mathbb{x}, \mathbb{w}) \text{ where } [X_{\mathcal{P}, i}]_* \text{ is the } i\text{-th } \mathcal{SO}\text{-query,} \\ b \leftarrow \mathcal{V}'^{\mathcal{G}, \mathcal{SO}}(\text{crs}, \mathbb{x}, [\Pi]_1) \text{ where } [X_{\mathcal{V}, j}]_* \text{ is the } j\text{-th } \mathcal{SO}\text{-query,} \\ [X_{\mathcal{P}, i^*}]_* = [X_{\mathcal{V}, j^*}]_*, \\ \forall_{i < i^*, j \in [T_{\mathcal{V}}]} : [X_{\mathcal{P}, i}]_* \neq [X_{\mathcal{V}, j}]_* \end{array} \right. \right] \cdot (1 - p_{i^*, j^*})$$

We use equality 4 to combine the probabilities.

$$\begin{aligned} &= \Pr_{\mathcal{SO}} \left[\begin{array}{l} b \neq 0, \\ [X_{\mathcal{P}, i^*}]_* = [X_{\mathcal{V}, j^*}]_*, \\ \forall_{i < i^*, j \in [T_{\mathcal{V}}]} : [X_{\mathcal{P}, i}]_* \neq [X_{\mathcal{V}, j}]_* \end{array} \left| \begin{array}{l} \text{crs} \leftarrow \mathcal{S}'^{\mathcal{G}}(1^\lambda), \\ [\Pi]_1 \leftarrow \mathcal{P}'^{\mathcal{G}, \mathcal{SO}}(\text{crs}, \mathbb{x}, \mathbb{w}) \\ \text{where } [X_{\mathcal{P}, i}]_* \text{ is the } i\text{-th } \mathcal{SO}\text{-query.} \\ b \leftarrow \mathcal{V}'^{\mathcal{G}, \mathcal{SO}}(\text{crs}, \mathbb{x}, [\Pi]_1) \\ \text{where } [X_{\mathcal{V}, j}]_* \text{ is the } j\text{-th } \mathcal{SO}\text{-query.} \end{array} \right. \right] \\ &\geq 1 - \sigma - (1 - 1/2T_{\mathcal{P}}T_{\mathcal{V}}) \\ &= 1/\text{poly}(\lambda) - \sigma \end{aligned}$$

These follow by union bound over completeness of $(\mathcal{S}', \mathcal{P}', \mathcal{V}')$ and the event that $[X_{\mathcal{P}, i^*}]_* = [X_{\mathcal{V}, j^*}]_* \wedge \forall_{i < i^*, j \in [T_{\mathcal{V}}]} : [X_{\mathcal{P}, i}]_* \neq [X_{\mathcal{V}, j}]_*$. $\square$

$\square$

6.2 Ruling out one group element SNARGs in the pure GGM

In this section, we prove that if one-way functions exist, a one-group element SNARG in Maurer's generic group model does not exist. The proof is a refinement of Groth's lower bound [Gro16]. He proved that if one-way functions exist, then there does not exist a SNARG where the proof consists of only $\mathbb{G}_1$ group elements, and the verifier has a particular structure. We focus on the setting where the proof is just one group element. This change in focus allows us to cover arbitrary verifiers in Maurer's generic group model.

Lemma 6.11. *If one-way functions exist (in the plain model) then there is no SNARG for NP in the Maurer Bilinear GGM with argument size one group element, completeness error $\sigma = 1 - 1/\text{poly}(\lambda)$, and soundness error $\beta = \text{negl}(\lambda)$.*

Proof. We prove the statement by contraposition. Assume $(\mathcal{S}, \mathcal{P}, \mathcal{V})$ is a SNARG in the Maurer Bilinear GGM as in the lemma statement for the NP relation $\mathcal{R} = \{(\mathbf{G}(s), s)\}$ where $\mathbf{G}: \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ is a PRG. Let L be the language derived from $\mathcal{R}$.

In this proof we will assume that the proof is in $\mathbb{G}_1$, however proving the lemma for $\mathbb{G}_2$ and $\mathbb{G}_T$ is analagous, the only changes accounting to the slightly different structure of $\mathbb{G}_2$ and $\mathbb{G}_T$ elements. Since the proof is a $\mathbb{G}_1$ element, the prover will output some group element $[\Pi]_1$ that is computed as some function $f(\text{crs}_1) = \langle \Pi, \text{crs}_1 \rangle$ where Π is a vector of $\mathbb{Z}_p$ elements. This means each verifier pairing equation check can be written as

$$\begin{aligned} & (d_i \cdot [\Pi]_1) \cdot \langle \mathbf{a}'_i, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle = [0]_T \\ \Leftrightarrow & [\Pi]_1 \cdot \langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle = [0]_T \end{aligned}$$

for some $\mathbb{Z}_p$ vectors $\mathbf{a}_i, \mathbf{a}'_i, \mathbf{b}_i, \mathbf{c}_i$, and $\mathbb{Z}_p$ elements d_i .

Using this proof system we show how to break the security game of the PRG: i.e., we construct a distinguisher $\mathcal{A}$ for the distributions

$$\mathcal{D}_G := \left\{ \mathbf{x} \mid \begin{array}{l} s \leftarrow \{0, 1\}^\lambda \\ \mathbf{x} \leftarrow \mathbf{G}(s) \end{array} \right\}, \quad \mathcal{D}_U := \{ \mathbf{x} \mid \mathbf{x} \leftarrow \{0, 1\}^{2\lambda} \}.$$

Note that $\mathcal{D}_G$ always outputs $\mathbf{x} \in L$ and that this happens only with negligible probability for $\mathcal{D}_U$. We describe first an algorithm **SolutionFinder**, and then use it to specify the distinguisher $\mathcal{A}$:

• **SolutionFinder** ^{$\mathcal{G}$} ($\text{crs}, \mathbf{a}, \mathbf{b}, \mathbf{c}$):

1. Let $[\text{crs}_1]_1$ (resp. $[\text{crs}_2]_2$ and $[\text{crs}_T]_T$) be the group elements in crs which are $\mathbb{G}_1$ elements (resp. $\mathbb{G}_2$ and $\mathbb{G}_T$).
2. Let **counter** $\leftarrow 0$.
3. Let $\mathcal{B} \leftarrow \emptyset$.
4. While **counter** $< \lambda/(1 - \sigma)$:
 - (a) Sample $s \leftarrow \{0, 1\}^\lambda$ uniformly at random.
 - (b) Run $[\Pi]_1 \leftarrow \mathcal{P}^{\mathcal{G}}(\text{crs}, \mathbf{x} := \mathbf{G}(s), \mathbf{w} := s)$ where Π is the $\mathbb{Z}_p$ -vector the prover uses to compute $[\Pi]_1 \leftarrow \langle \Pi, [\text{crs}_1]_1 \rangle$.
 - (c) Let $\mathcal{I} \leftarrow \emptyset$.
 - (d) Run $\mathcal{V}^{\mathcal{G}}(\text{crs}, \mathbf{x} := \mathbf{G}(s), [\Pi]_1)$ where for $i \in [T_V]$ we have $\mathbf{a}'_i, \mathbf{b}'_i$, and $\mathbf{c}'_i$ are the $\mathbb{Z}_p$ vectors such that the i -th query of the verifier checks the equation

$$[\Pi]_1 \cdot \langle \mathbf{a}'_i, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}'_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}'_i, [\text{crs}_T]_T \rangle \stackrel{?}{=} [0]_T.$$

If the equations holds let $\mathcal{I} \leftarrow \mathcal{I} \cup \{i\}$.

- (e) For $i \in \mathcal{I}$:

- i. Let

$$\mathbf{v} \leftarrow \begin{pmatrix} \Pi \otimes \mathbf{a}'_i \\ \mathbf{b}'_i \\ \mathbf{c}'_i \end{pmatrix}$$

- ii. If $\mathbf{v} \notin \text{span}(\mathcal{B})$ let $\mathcal{B} \leftarrow \mathcal{B} \cup \{\mathbf{v}\}$ and **counter** $\leftarrow 0$.

- (f) Let **counter** $\leftarrow \text{counter} + 1$.

- (g) If there exists a $\mathbf{\Pi}$ such that $\begin{pmatrix} \mathbf{\Pi} \otimes \mathbf{a} \\ \mathbf{b} \\ \mathbf{c} \end{pmatrix} \in \text{span}(\mathcal{B})$ then output such a uniformly random $\mathbf{\Pi}$ otherwise output $\perp$.

• $\mathcal{A}(\mathbb{x}^*)$:

1. Simulate a generic group under the name of $\mathcal{G}$.
2. Sample $\text{crs} \leftarrow \mathcal{S}^{\mathcal{G}}(1^\lambda)$.
3. Simulate $b^* \leftarrow \mathcal{V}^{\mathcal{G}}(\mathbb{x}^*, [\mathbf{\Pi}]_1 := [1])$ on the i -th pairing equation check respond as follows:
 - (a) Let the i -th pairing equation check be

$$[\mathbf{\Pi}]_1 \cdot \langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle \stackrel{?}{=} [0]_T.$$

- (b) If $\langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle = [0]_2$ respond with 1 if $\langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle = [0]_T$ and 0 otherwise.
 - (c) Let $\mathbf{\Pi}^* \leftarrow \text{SolutionFinder}^{\mathcal{G}}(\text{crs}, (\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i))$.
 - (d) If $\mathcal{V}^{\mathcal{G}}(\mathbb{x}, \langle \mathbf{\Pi}, [\text{crs}_1]_1 \rangle) = 1$ then $\mathcal{A}$ outputs 1. Otherwise respond to the query with 0.
4. Output 0.

First, we argue that the adversary terminates in polynomial time. Because the dimension of

$\begin{pmatrix} \mathbf{\Pi} \otimes \mathbf{a}_i \\ \mathbf{b}_i \\ \mathbf{c}_i \end{pmatrix}$ is polynomial in the security parameter, **counter** can only be reset to 0 a polynomial number of times and the loop in **SolutionFinder** is run $2\lambda/(1-\sigma) = \text{poly}(\lambda)$ many times.

For each i we define a linear space $S := \{(\mathbf{\Pi} \otimes \mathbf{a}) \mid \mathbf{\Pi} \in \mathbb{F}^{|\text{crs}_1|}\}$ and an affine space

$$B := \left\{ x \mid \begin{pmatrix} x \\ \mathbf{b} \\ \mathbf{c} \end{pmatrix} \in \text{span}(\mathcal{B}) \right\}.$$

A $\mathbf{\Pi}$ meets all the conditions if and only if $\mathbf{\Pi} \in S \cap B$. Therefore it is sufficient to compute said intersection and sample from it.

We analyse how the adversary's output behaves on samples from the PRG versus uniformly random samples:

Claim 6.12. *If $\mathbb{x} \notin L$ the adversary $\mathcal{A}$ outputs 0 with probability $1 - \text{negl}(\lambda)$.*

Proof. We define a new adversary, this time for the SNARG:

$\mathcal{A}_S^{\mathcal{G}}(\text{crs}, \mathbb{x})$:

1. Simulate $b^* \leftarrow \mathcal{V}^{\mathcal{G}}(\mathbb{x}, [\mathbf{\Pi}]_1 := [1])$ on the i -th pairing equation check respond as follows:
 - (a) Let the i -th pairing equation check be

$$[\mathbf{\Pi}]_1 \cdot \langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle \stackrel{?}{=} [0]_T.$$

- (b) If $\langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle = [0]_2$ respond with 1 if $\langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle = [0]_T$ and 0 otherwise.

- (c) Let $\Pi^* \leftarrow \text{SolutionFinder}^{\mathcal{G}}(\text{crs}, (\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i))$.
- (d) If $\mathcal{V}^{\mathcal{G}}(\mathbb{x}, \langle \Pi, [\text{crs}_1]_1 \rangle) = 1$ then $\mathcal{A}_S$ outputs $\langle \Pi, [\text{crs}_1]_1 \rangle$. Otherwise respond to the query with 0.

2. Output 0.

Observe that $\mathcal{A}_S$ is identical to $\mathcal{A}$ except that it doesn't sample crs and it has access to a real oracle $\mathcal{G}$ rather than simulating it by itself. However crs in the SNARG soundness experiment is sampled as in $\mathcal{A}$, and $\mathcal{G}$ is exactly the same as well. Thus, if $\mathcal{A}$ outputs 1 then $\mathcal{A}_S$ outputs a proof $[\Pi]_1$ such that $\mathcal{V}(\mathbb{x}, [\Pi]_1) = 1$. By soundness we get the probability that $\mathcal{A}_S$ outputs a proof $[\Pi]_1$ with $\mathcal{V}(\mathbb{x}, \langle \Pi, [\text{crs}_1]_1 \rangle) = 1$ is $\beta = \text{negl}(\lambda)$. It follows the probability that $\mathcal{A}(\mathbb{x})$ outputs 1 is $\text{negl}(\lambda)$. $\square$

Claim 6.13. *For $\mathbb{x} \leftarrow \mathcal{D}_{\mathcal{G}}$, the following occurs with negligible probability (over the choice of $\mathbb{x}$, crs and the verifier randomness): given $\mathbb{x}$ and crs , $\mathcal{V}$ accepts all proofs $[\Pi]_1$ where every pairing equation checks*

$$[\Pi]_1 \cdot \langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle \stackrel{?}{=} [0]_T$$

with $\langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle \neq [0]_2$ in $\mathcal{V}^{\mathcal{G}}(\text{crs}, \mathbb{x}, [\Pi]_1)$ are answered by 0.

Proof. We begin by moving from instances $\mathbb{x}$ sampled from $\mathcal{D}_{\mathcal{G}}$ to ones sampled from $\mathcal{D}_U$. Consider the following algorithm :

- $\mathcal{A}_{\mathcal{G}}(\mathbb{x})$:
 1. Simulate a generic group under the name of $\mathcal{G}$.
 2. Sample $\text{crs} \leftarrow \mathcal{S}^{\mathcal{G}}(1^\lambda)$.
 3. Simulate $b \leftarrow \mathcal{V}^{\mathcal{G}}(\mathbb{x}, [\Pi]_1 := [1])$ where on the i -th pairing equation check respond as follows:
 - (a) Let the i -th pairing equation check be

$$[\Pi]_1 \cdot \langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle \stackrel{?}{=} [0]_T.$$
 - (b) If $\langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle = [0]_2$ respond with 1 if $\langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle = [0]_T$ and 0 otherwise.
 - (c) Otherwise respond to the query with 0.
 4. Output b .

As we are in the Maurer GGM, without loss of generality, the view of $\mathcal{V}$ of the proof $[\Pi]_1$ is only in the pairing equation checks where $\langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle \neq [0]_2$ (if it makes an equality check in $\mathbb{G}_1$ then we can translate it into one in $\mathbb{G}_T$). Any the verifier's view when given $[\Pi]_1$ for which all of the are answered by 0 is identical to its view in $\mathcal{A}_{\mathcal{G}}(\mathbb{x})$. Thus the probability that $\mathcal{V}$ accepts all claims where the pairing equations output 0 is equal to the probability of $\mathcal{A}_{\mathcal{G}}$ outputting 1 for $\mathbb{x} \leftarrow \mathcal{D}_{\mathcal{G}}$.

By indistinguishability between $\mathcal{D}_{\mathcal{G}}$ and $\mathcal{D}_U$ and the fact that $\mathcal{A}_{\mathcal{G}}$ is efficient, we have that $\mathcal{A}_{\mathcal{G}}(\mathbb{x}) = 1$ with the same probability up to a $\text{negl}(\lambda)$ factor when $\mathbb{x} \leftarrow \mathcal{D}_{\mathcal{G}}$ as $\mathbb{x} \leftarrow \mathcal{D}_U$.

We now show that $\mathcal{A}_{\mathcal{G}}(\mathbb{x}) = 1$ with negligible probability when $\mathbb{x} \leftarrow \mathcal{D}_U$, thus completing the proof. First, note that when sampled from this distribution, $\mathbb{x} \notin L$ with overwhelming probability. Thus we henceforth consider some fixed $\mathbb{x} \notin L$. Suppose towards contradiction that $\mathcal{A}_{\mathcal{G}}(\mathbb{x})$ outputs 1 with noticeable probability $p \geq 1/\text{poly}(\lambda)$. We show that this contradicts soundness of the SNARG by defining an adversary:

- $\mathcal{A}_0^{\mathcal{G}}(\text{crs}, \mathbb{x})$: For $i \in [T_{\mathcal{V}} + 1]$ if $\mathcal{V}^{\mathcal{G}}(\text{crs}, \mathbb{x}, [\Pi]_1 := [i]_1) = 1$ output $[i]_1$.

We show that $\mathcal{A}_0$ will win the soundness experiment with probability p . Consider a crs such that $\mathcal{A}_G$ outputs 1, meaning that $\mathcal{V}$ accepts a proof $[\Pi]_1$ where all the relevant pairing equation checks in $\mathcal{V}^G(\text{crs}, \mathbf{x}, [\Pi]_1)$ return 0.

Each pairing equation check is a degree 1 polynomial in the proof $[\Pi]_1$. Therefore, by polynomial identity lemma, for every check there exists at most one $[\Pi]_1$ such that the pairing equation check outputs 1. Because the number of pairing equation checks the verifier is T_V it can reject at most T_V distinct group elements. $\mathcal{A}_0$ tests $T_V + 1$ distinct group elements and at most T_V elements can be rejected by the verifier, and whenever crs and the verifier randomness are chosen so that this happens, $\mathcal{A}_0$ will win. This happens with probability $p \geq 1/\text{poly}(\lambda)$ which contradicts the assumption that the protocol has negligible soundness error. $\square$

Claim 6.14. *If $\mathbf{x} \leftarrow \mathcal{D}_G$, then $\mathcal{A}(\mathbf{x})$ outputs 1 with probability $\geq 1/\text{poly}(\lambda)$.*

Proof. Let **GoodCRS** be the event that crs is such that

$$\Pr_{(\mathbf{x}', \mathbf{w}') \leftarrow \mathcal{D}_G} [\mathcal{V}^G(\text{crs}, \mathbf{x}', [\Pi']_1) = 1 \mid [\Pi']_1 \leftarrow \mathcal{P}^G(\text{crs}, \mathbf{x}', \mathbf{w}')] \geq 1 - \sigma,$$

where by $(\mathbf{x}', \mathbf{w}') \leftarrow \mathcal{D}_G$ we mean that $\mathbf{w}'$ is the preimage of the PRG. By completeness of the SNARG, $\Pr_{\text{crs}}[\text{GoodCRS}] \geq 1 - \sigma$. For the rest of the proof we assume **GoodCRS** holds.

We define another event, **GoodInstance**, representing when for the challenge $\mathbf{x}^*$ we have $\mathcal{V}(\text{crs}, \mathbf{x}^*, [\Pi]_1) = 1$ for $[\Pi]_1 \leftarrow \mathcal{P}^G(\text{crs}, \mathbf{x}^*, \mathbf{w}^*)$. By definition, $\Pr[\text{GoodInstance} \mid \text{GoodCRS}] \geq 1 - \sigma$.

Let $\mathbf{\Pi}^*$ be the vector that $\mathcal{P}^G(\text{crs}, \mathbf{x}^*, \mathbf{w}^*)$ uses to compute $[\Pi]_1 \leftarrow \langle \mathbf{\Pi}^*, [\text{crs}_1]_1 \rangle$. For notational convenience, we abuse notation and write $([\Pi]_1, \mathbf{\Pi}^*) \leftarrow \mathcal{P}^G(\text{crs}, \mathbf{x}^*, \mathbf{w}^*)$. By Claim 6.13 for $\mathbf{x} \leftarrow \mathcal{D}_G$ with probability $1 - \text{negl}(\lambda)$ there exists a query i such that $\langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle \neq 0$ and $[\Pi]_1 \cdot \langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle = [0]_T$. Let i^* be the random variable representing the smallest of these indices.

We now argue about the behavior of **SolutionFinder**. It will eventually use a basis $\mathcal{B}$ to check whether

$$\begin{pmatrix} \mathbf{\Pi}^* \otimes \mathbf{a}_{i^*} \\ \mathbf{b}_{i^*} \\ \mathbf{c}_{i^*} \end{pmatrix} \in \text{span}(\mathcal{B}).$$

Let $\mathcal{B}_j$ be $\mathcal{B}$ after adding the j -th vector to it. Before **SolutionFinder** uses this as its final basis it samples $2\lambda/(1 - \sigma)$ many independent $\mathbf{x}$ and $\mathbf{w}$. If for a fixed j

$$\Pr_{(\mathbf{x}, \mathbf{w}) \leftarrow \mathcal{R}_L} \left[\begin{array}{l} \exists i. \\ [\Pi]_1 \cdot \langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle \\ + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle = [0]_T, \\ \left(\begin{pmatrix} \mathbf{\Pi}^* \otimes \mathbf{a}_i \\ \mathbf{b}_i \\ \mathbf{c}_i \end{pmatrix} \right) \notin \mathcal{B}_j \end{array} \mid \begin{array}{l} ([\Pi]_1, \mathbf{\Pi}^*) \leftarrow \mathcal{P}^G(\text{crs}, \mathbf{x}, \mathbf{w}) \\ \mathcal{V}^G(\text{crs}, \mathbf{x}, \mathbf{w}) \text{ makes} \\ \text{paring equality checks} \\ (\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i)_{i \in [\ell]} \end{array} \right] > (1 - \sigma)/2,$$

then **SolutionFinder** will use it in the final computation with probability $(1 - (1 - \sigma)/2)^{2\lambda/(1 - \sigma)}$. Because in the **SolutionFinder** there are only polynomially many $\mathcal{B}_j$ we have that the final state of

$\mathcal{B}$ has the property

$$\Pr_{(\mathbf{x}, \mathbf{w}) \leftarrow \mathcal{R}_L} \left[\begin{array}{l} \exists i. \\ \left[\begin{array}{l} [\Pi]_1 \cdot \langle \mathbf{a}_i, [\text{crs}_2]_2 \rangle + \langle \mathbf{b}_i, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle \\ + \langle \mathbf{c}_i, [\text{crs}_T]_T \rangle = [0]_T, \\ \left(\begin{array}{c} \Pi^* \otimes \mathbf{a}_i \\ \mathbf{b}_i \\ \mathbf{c}_i \end{array} \right) \notin \mathcal{B} \end{array} \right] \mid \begin{array}{l} ([\Pi]_1, \Pi^*) \leftarrow \mathcal{P}^{\mathcal{G}}(\text{crs}, \mathbf{x}, \mathbf{w}) \\ \mathcal{V}^{\mathcal{G}}(\text{crs}, \mathbf{x}, \mathbf{w}) \text{ makes} \\ \text{paring equality checks} \\ (\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i)_{i \in [\ell]} \end{array} \right] \\ > (1 - \sigma)/2, \end{array} \right]$$

with probability $\leq \text{negl}(\lambda)$. We call this event **BadBasis**. For the rest of the proof we assume $\overline{\text{BadBasis}}$.

In this case, **SolutionFinder** outputs some vector Π such that $\left(\begin{array}{c} \Pi \otimes \mathbf{a}_{i^*} \\ \mathbf{b}_{i^*} \\ \mathbf{c}_{i^*} \end{array} \right) \in \text{span}(\mathcal{B})$ with probability $\geq 1 - (1 - \sigma)/2$. Because every vector $\mathbf{v} \in \mathcal{B}$ is such that $\left\langle \mathbf{v}, \left(\begin{array}{c} [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \\ [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \\ [\text{crs}_T]_T \end{array} \right) \right\rangle = [0]_T$ we get that

$$\begin{aligned} \left\langle \left(\begin{array}{c} \Pi \otimes \mathbf{a}_{i^*} \\ \mathbf{b}_{i^*} \\ \mathbf{c}_{i^*} \end{array} \right), \left(\begin{array}{c} [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \\ [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \\ [\text{crs}_T]_T \end{array} \right) \right\rangle &= [0]_T = \left\langle \left(\begin{array}{c} \Pi^* \otimes \mathbf{a}_{i^*} \\ \mathbf{b}_{i^*} \\ \mathbf{c}_{i^*} \end{array} \right), \left(\begin{array}{c} [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \\ [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \\ [\text{crs}_T]_T \end{array} \right) \right\rangle \Leftrightarrow \\ \langle \Pi \otimes \mathbf{a}_{i^*}, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle &= \langle \Pi^* \otimes \mathbf{a}_{i^*}, [\text{crs}_1]_1 \otimes [\text{crs}_2]_2 \rangle \Leftrightarrow \\ \langle \Pi, [\text{crs}_1]_1 \rangle \cdot \langle \mathbf{a}_{i^*}, [\text{crs}_2]_2 \rangle &= \langle \Pi^*, [\text{crs}_1]_1 \rangle \cdot \langle \mathbf{a}_{i^*}, [\text{crs}_2]_2 \rangle \Leftrightarrow \\ \langle \Pi, [\text{crs}_1]_1 \rangle &= \langle \Pi^*, [\text{crs}_1]_1 \rangle \\ &= [\Pi]_1 \end{aligned}$$

Therefore, $\mathcal{A}$ outputs 1 with probability

$$\begin{aligned} &\geq \Pr \left[\text{GoodCRS} \wedge \overline{\text{BadBasis}} \wedge \text{GoodInstance} \wedge \left(\begin{array}{c} \Pi \otimes \mathbf{a}_{i^*} \\ \mathbf{b}_{i^*} \\ \mathbf{c}_{i^*} \end{array} \right) \in \text{span}(\mathcal{B}) \right] \\ &\geq \Pr \left[\overline{\text{BadBasis}} \wedge \text{GoodInstance} \wedge \left(\begin{array}{c} \Pi \otimes \mathbf{a}_{i^*} \\ \mathbf{b}_{i^*} \\ \mathbf{c}_{i^*} \end{array} \right) \in \text{span}(\mathcal{B}) \mid \text{GoodCRS} \right] (1 - \sigma) \\ &\geq \Pr \left[\text{GoodInstance} \wedge \left(\begin{array}{c} \Pi \otimes \mathbf{a}_{i^*} \\ \mathbf{b}_{i^*} \\ \mathbf{c}_{i^*} \end{array} \right) \in \text{span}(\mathcal{B}) \mid \frac{\text{GoodCRS}}{\overline{\text{BadBasis}}} \right] (1 - \sigma)(1 - \text{negl}(\lambda)) \\ &\geq (1 - \sigma - (1 - \sigma)/2)(1 - \sigma)(1 - \text{negl}(\lambda)) \\ &\geq ((1 - \sigma)/2)(1 - \sigma)(1 - \text{negl}(\lambda)) \\ &\geq 1/\text{poly}(\lambda). \end{aligned}$$

□

Lemma 6.11 follows from Claim 6.14 and Claim 6.12. □

Acknowledgments

We would like to thank Aurore Guillevic for helpful discussions on symmetric pairing groups.

Gal Arnon is supported by the European Research Council (ERC) under the EU’s Horizon 2020 research and innovation programme (Grant agreement No. 101019547), and Stellar Foundation Grant. This work was done in part while Gal Arnon was a research fellow at the Simons Institute for the Theory of Computing. This work was done in part while Jesko Dujmovic was at CISA Helmholtz-Center for Information Security. Jesko Dujmovic was supported by the European Research Union (ERC, LACONIC, 101041207). Eylon Yogev is supported by the Israel Science Foundation (Grant No. 2302/22), and European Research Union (ERC, CRYPTOPROOF, 101164375). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

- [ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. “STIR: Reed–Solomon Proximity Testing with Fewer Queries”. In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO ’24. 2024, pp. 380–413.
- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. “WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification”. In: *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part IV*. Ed. by Serge Fehr and Pierre-Alain Fouque. Vol. 15604. Lecture Notes in Computer Science. Springer, 2025, pp. 214–243.
- [ACY23] Gal Arnon, Alessandro Chiesa, and Eylon Yogev. “IOPs with Inverse Polynomial Soundness Error”. In: *64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023*. IEEE, 2023, pp. 752–761.
- [ADI25] Gal Arnon, Jesko Dujmovic, and Yuval Ishai. “Designated-verifier SNARGs with one group element”. In: *Annual International Cryptology Conference*. Springer, 2025, pp. 192–224.
- [AFLN24] Martin R. Albrecht, Giacomo Fenzi, Oleksandra Lapiha, and Ngoc Khanh Nguyen. “SLAP: Succinct Lattice-Based Polynomial Commitments from Standard Assumptions”. In: *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Ed. by Marc Joye and Gregor Leander. Vol. 14656. Lecture Notes in Computer Science. Springer, 2024, pp. 90–119.
- [AHIV17] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkatasubramanian. “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup”. In: *Proceedings of the 24th ACM Conference on Computer and Communications Security*. CCS ’17. 2017, pp. 2087–2104.
- [ALMSS98] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. “Proof verification and the hardness of approximation problems”. In: *Journal of the ACM* 45.3 (1998). Preliminary version in FOCS ’92., pp. 501–555.
- [BBBPWM18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. “Bulletproofs: Short Proofs for Confidential Transactions and More”. In: *Proceedings of the 39th IEEE Symposium on Security and Privacy*. S&P ’18. 2018, pp. 315–334.

- [BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. “Fast Reed–Solomon Interactive Oracle Proofs of Proximity”. In: *Proceedings of the 45th International Colloquium on Automata, Languages and Programming*. ICALP ’18. 2018, 14:1–14:17.
- [BBHR19] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. “Scalable Zero Knowledge with No Trusted Setup”. In: *Proceedings of the 39th Annual International Cryptology Conference*. CRYPTO ’19. 2019, pp. 733–764.
- [BCCGP16] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. “Efficient Zero-Knowledge Arguments for Arithmetic Circuits in the Discrete Log Setting”. In: *Proceedings of the 35th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’16. 2016, pp. 327–357.
- [BCIOP13] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. “Succinct Non-Interactive Arguments via Linear Interactive Proofs”. In: *Proceedings of the 10th Theory of Cryptography Conference*. TCC ’13. 2013, pp. 315–333.
- [BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. “Interactive Oracle Proofs”. In: *Proceedings of the 14th Theory of Cryptography Conference*. TCC ’16-B. 2016, pp. 31–60.
- [BGJT13] Razvan Barbulescu, Pierrick Gaudry, Antoine Joux, and Emmanuel Thomé. “A quasi-polynomial algorithm for discrete logarithm in finite fields of small characteristic”. In: *IACR Cryptol. ePrint Arch.* (2013), p. 400.
- [BHIRW24] Nir Bitansky, Prahladh Harsha, Yuval Ishai, Ron D Rothblum, and David J Wu. “Dot-product proofs and their applications”. In: *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE. 2024, pp. 806–825.
- [BIOW20] Ohad Barta, Yuval Ishai, Rafail Ostrovsky, and David J Wu. “On succinct arguments and witness encryption from groups”. In: *Annual International Cryptology Conference*. Springer. 2020, pp. 776–806.
- [BISW17] Dan Boneh, Yuval Ishai, Amit Sahai, and David J. Wu. “Lattice-Based SNARGs and Their Application to More Efficient Obfuscation”. In: *Proceedings of the 36th Annual International Conference on Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’17. 2017, pp. 247–277.
- [BS23] Ward Beullens and Gregor Seiler. “LaBRADOR: Compact Proofs for R1CS from Module-SIS”. In: *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V*. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14085. Lecture Notes in Computer Science. Springer, 2023, pp. 518–548.
- [CGKY25] Alessandro Chiesa, Ziyi Guan, Christian Knabenhans, and Zihan Yu. “On the Fiat-Shamir Security of Succinct Arguments from Functional Commitments”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 902. URL: <https://eprint.iacr.org/2025/902>.
- [CY24] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024. URL: <https://github.com/hash-based-snargs-book>.
- [DFGK14] George Danezis, Cedric Fournet, Jens Groth, and Markulf Kohlweiss. “Square Span Programs with Applications to Succinct NIZK Arguments”. In: *Proceedings of the 20th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’14. 2014, pp. 532–550.
- [DMS24] Michel Dellepère, Pratyush Mishra, and Alireza Shirzad. “Garuda and Pari: Faster and Smaller SNARKs via Equiefficient Polynomial Commitments”. In: *IACR Cryptol. ePrint Arch.* (2024), p. 1245.

- [FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. “The Algebraic Group Model and its Applications”. In: *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part II*. Ed. by Hovav Shacham and Alexandra Boldyreva. Vol. 10992. Lecture Notes in Computer Science. Springer, 2018, pp. 33–62.
- [FS86] Amos Fiat and Adi Shamir. “How to prove yourself: practical solutions to identification and signature problems”. In: *Proceedings of the 6th Annual International Cryptology Conference. CRYPTO ’86*. 1986, pp. 186–194.
- [GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. “Quadratic Span Programs and Succinct NIZKs without PCPs”. In: *Advances in Cryptology - EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings*. Ed. by Thomas Johansson and Phong Q. Nguyen. Vol. 7881. Lecture Notes in Computer Science. Springer, 2013, pp. 626–645.
- [GLSTW23] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. “Brakedown: Linear-time and field-agnostic SNARKs for R1CS”. In: *Proceedings of the 43rd Annual International Cryptology Conference. CRYPTO ’23*. 2023.
- [GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. Cryptology ePrint Archive, Report 2019/953. 2019.
- [Gal14] Steven Galbraith. *New discrete logarithm records and the death of Type 1 pairings*. EllipticNews Blog. <https://ellipticnews.wordpress.com/2014/02/01/new-discrete-logarithm-records-and-the-death-of-type-1-pairings/>. 2014. URL: <https://ellipticnews.wordpress.com/2014/02/01/new-discrete-logarithm-records-and-the-death-of-type-1-pairings/>.
- [Gro10] Jens Groth. “Short Pairing-Based Non-interactive Zero-Knowledge Arguments”. In: *Proceedings of the 16th International Conference on the Theory and Application of Cryptology and Information Security. ASIACRYPT ’10*. 2010, pp. 321–340.
- [Gro16] Jens Groth. “On the Size of Pairing-Based Non-interactive Arguments”. In: *Proceedings of the 35th Annual International Conference on Theory and Applications of Cryptographic Techniques. EUROCRYPT ’16*. 2016, pp. 305–326.
- [HNY22] Iftach Haitner, Daniel Nukrai, and Eylon Yogev. “Lower Bound on SNARGs in the Random Oracle Model”. In: *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part III*. Ed. by Yevgeniy Dodis and Thomas Shrimpton. Vol. 13509. Lecture Notes in Computer Science. Springer, 2022, pp. 97–127.
- [IKO05] Yuval Ishai, Eyal Kushilevitz, and Rafail Ostrovsky. “Sufficient Conditions for Collision-Resistant Hashing”. In: *Proceedings of the 2nd Theory of Cryptography Conference. TCC ’05*. 2005, pp. 445–456.
- [IKO07] Yuval Ishai, Eyal Kushilevitz, and Rafail Ostrovsky. “Efficient Arguments without Short PCPs”. In: *Proceedings of the Twenty-Second Annual IEEE Conference on Computational Complexity. CCC ’07*. 2007, pp. 278–291.
- [JLLW23] Aayush Jain, Huijia Lin, Ji Luo, and Daniel Wichs. “The Pseudorandom Oracle Model and Ideal Obfuscation”. In: *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part IV*. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14084. Lecture Notes in Computer Science. Springer, 2023, pp. 233–262.

- [Kho02] Subhash Khot. “On the Power of Unique 2-Prover 1-Round Games”. In: *Proceedings of the 17th Annual IEEE Conference on Computational Complexity, Montréal, Québec, Canada, May 21-24, 2002*. IEEE Computer Society, 2002, p. 25.
- [Kil92] Joe Kilian. “A note on efficient zero-knowledge proofs and arguments”. In: *Proceedings of the 24th Annual ACM Symposium on Theory of Computing*. STOC ’92. 1992, pp. 723–732.
- [LM19] Russell WF Lai and Giulio Malavolta. “Subvector commitments with application to succinct arguments”. In: *Advances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part I 39*. Springer. 2019, pp. 530–560.
- [Lip12] Helger Lipmaa. “Progression-Free Sets and Sublinear Pairing-Based Non-Interactive Zero-Knowledge Arguments”. In: *Proceedings of the 9th Theory of Cryptography Conference on Theory of Cryptography*. TCC ’12. 2012, pp. 169–189.
- [Lip24] Helger Lipmaa. “Polymath: Groth16 Is Not the Limit”. In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*. Ed. by Leonid Reyzin and Douglas Stebila. Vol. 14929. Lecture Notes in Computer Science. Springer, 2024, pp. 170–206.
- [Mau05] Ueli Maurer. “Abstract models of computation in cryptography”. In: *Cryptography and Coding: 10th IMA International Conference, Cirencester, UK, December 19-21, 2005. Proceedings 10*. Springer. 2005, pp. 1–12.
- [Mic00] Silvio Micali. “Computationally Sound Proofs”. In: *SIAM Journal on Computing* 30.4 (2000). Preliminary version appeared in FOCS ’94., pp. 1253–1298.
- [Nec94] Vassiliy Ilyich Nechaev. “Complexity of a determinate algorithm for the discrete logarithm”. In: *Mathematical Notes* 55 (2 1994), pp. 165–172.
- [SSEKYZ24] Ron Steinfeld, Amin Sakzad, Muhammed F. Esgin, Veronika Kuchta, Mert Yassi, and Raymond K. Zhao. “LUNA: Quasi-Optimally Succinct Designated-Verifier Zero-Knowledge Arguments from Lattices”. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*. Ed. by Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie. ACM, 2024, pp. 3167–3181.
- [ST06] Alex Samorodnitsky and Luca Trevisan. “Gowers uniformity, influence of variables, and PCPs”. In: *Proceedings of the thirty-eighth annual ACM symposium on Theory of Computing*. 2006, pp. 11–20.
- [STW23] Srinath Setty, Justin Thaler, and Riad Wahby. “Customizable constraint systems for succinct arguments”. In: *IACR Cryptol. ePrint Arch.* (2023), p. 552.
- [SW14] Amit Sahai and Brent Waters. “How to use indistinguishability obfuscation: deniable encryption, and more”. In: *STOC*. 2014.
- [Sho97] Victor Shoup. “Lower bounds for discrete logarithms and related problems”. In: *Proceedings of the 16th International Conference on the Theory and Application of Cryptographic Techniques*. EUROCRYPT ’97. 1997, pp. 256–266.
- [WW24] Brent Waters and David J. Wu. “Adaptively-Sound Succinct Arguments for NP from Indistinguishability Obfuscation”. In: *Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024*. Ed. by Bojan Mohar, Igor Shinkar, and Ryan O’Donnell. ACM, 2024, pp. 387–398.
- [WW25] Brent Waters and David J. Wu. “A Pure Indistinguishability Obfuscation Approach to Adaptively-Sound SNARGs for sNP”. In: *Proceedings of the 45th Annual International Cryptology Conference*. Ed. by Yael Tauman Kalai and Seny F. Kamara. Vol. 16006. CRYPTO ’25. Springer, 2025, pp. 292–326.

- [WZ24] Brent Waters and Mark Zhandry. “Adaptive Security in SNARGs via iO and Lossy Functions”. In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*. Ed. by Leonid Reyzin and Douglas Stebila. Vol. 14929. Lecture Notes in Computer Science. 2024, pp. 72–104.
- [ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. “BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes”. In: *Proceedings of the 44th Annual International Cryptology Conference*. Vol. 14929. CRYPTO ’24. 2024, pp. 138–169.